박 사 학 위 논 문
Ph.D. Dissertation

동형암호와 전호화, 동형암호 친화적인 대칭키 암호에 관하여

On Homomorphic Encryption, Transciphering Frameworks, and HE-friendly Ciphers

2022

KOREA ADVANCED INSTITUTE OF SCIENCE AND TECHNOLOGY
KAIST
1971
한국과학기술원

김 성 광 (金 成 光 Kim, Seongkwang)

한 국 과 학 기 술 원

Korea Advanced Institute of Science and Technology

박 사 학 위 논 문

동형암호와 전호화, 동형암호 친화적인 대칭키 암호에 관하여

2022

김 성 광

한 국 과 학 기 술 원

전산학부 (정보보호대학원)

동형암호와 전호화, 동형암호 친화적인 대칭키 암호에 관하여

김 성 광

위 논문은 한국과학기술원 박사학위논문으로
학위논문 심사위원회의 심사를 통과하였음

KAIST

2021년 11월 26일

심사위원장 이 주 영 (인)

심 사 위 원 한 상 근 (인)

심 사 위 원 윤 아 람 (인)

심 사 위 원 이 형 태 (인)

심 사 위 원 윤 인 수 (인)

On Homomorphic Encryption, Transciphering Frameworks, and HE-friendly Ciphers

Seongkwang Kim

Advisor: Jooyoung Lee

A dissertation submitted to the faculty of
Korea Advanced Institute of Science and Technology in
partial fulfillment of the requirements for the degree of
Doctor of Philosophy in Computer Science (Information Security)

Daejeon, Korea
November 26, 2021

Approved by

Jooyoung Lee
Professor of School of Computing

The study was conducted in accordance with Code of Research Ethics[1].

[1] Declaration of Ethical Conduct in Research: I, as a graduate student of Korea Advanced Institute of Science and Technology, hereby declare that I have not committed any act that may damage the credibility of my research. This includes, but is not limited to, falsification, thesis written by someone else, distortion of research findings, and plagiarism. I confirm that my thesis contains honest conclusions based on my own careful research under the guidance of my advisor.

DIS 김성광. 동형암호와 전호화, 동형암호 친화적인 대칭키 암호에 관하여. 전산학부 (정보보호대학원) . 2022년. 86+vi 쪽. 지도교수: 이주영. (영문 논문)
Seongkwang Kim. On Homomorphic Encryption, Transciphering Frameworks, and HE-friendly Ciphers. School of Computing (Graduate School of Information Security) . 2022. 86+vi pages. Advisor: Jooyoung Lee. (Text in English)

초 록

동형암호는 데이터가 암호화된 상태로 계산할 수 있는 암호이며, 그 특성으로 인해 의료, 유전자, 금융 등 다양한 분야에 쓰이고 있다. 그럼에도 불구하고, 현재의 동형암호 스킴들은 느린 암호화 속도와 큰 암호문 확장이라는 단점을 가지고 있다. 전호화(transciphering framework)는 대칭키 암호문을 동형암호 암호문으로 전환해주는 기법으로, 앞서 설명한 동형암호의 단점을 해결하기 위해 제안되었다.

이 논문에서는 RtF와 BtE 라는 두 가지 새로운 전호화 기법을 제안하였다. RtF는 최초로 제안된 실수 데이터를 전호화하는 기법이며, CKKS와 FV의 구조적 유사성을 이용하여 혼합한 것이 주요 아이디어이다. 결과적으로, 이후에 CKKS를 통해 계산될 실수 데이터가 큰 암호문 확장이나 계산력의 과부하 없이 암호화될 수 있게 되었다. BtE는 이진 연산이 가능한 동형암호 암호문 슬롯의 확대체 구조를 활용하여 효율성을 높인 전호화 기법이다. 단순히 BtE만 사용하면 이진 확대체 연산 밖에 지원하지 않지만, 이후 자리수 추출 함수를 사용하여 이진 연산이 가능한 형태의 암호문으로 만들 수 있다.

각각의 전호화는 전호화 내부에서 사용할 스트림 암호가 필요하기 때문에, 우리는 HERA와 Rubato, eHERA라는 세 개의 동형암호 친화적인 대칭키 암호를 제안했다. HERA는 간단하고 랜덤한 키 스케줄과 성분 값이 작은 선형 연산 특징을 지녔는데, 이는 한 번에 여러 연산을 처리하는 동형암호의 특성을 잘 활용하여 효율적으로 HERA를 계산하기 위함이다. Rubato는 블록암호와 LWE의 혼합 형태를 가진 암호이다. 암호화 마지막에 더해지는 노이즈가 대수적 차수를 높이는 역할을 하여, 암호의 곱셈 깊이(multiplicative depth)를 낮추는데 도움이 된다. 우리는 또한 HERA의 변형인 eHERA를 제안했다. eHERA는 BtE를 사용하기 위해 필요한 이진 확대체 위에서의 스트림 암호이다. eHERA를 사용한 BtE를 통해 기존보다 더 효율적인 이진 전호화 기법을 구성할 수 있음을 실험을 통해 확인하였다.

핵 심 낱 말 스트림 암호, 동형암호, 동형암호 친화적인 대칭키 암호, 전호화

Abstract

Homomorphic encryption (HE) is a promising cryptographic primitive that enables computation over encrypted data, with a variety of applications including medical, genomic, and financial tasks. Current HE schemes, nevertheless, still suffer from slow encryption speed and large ciphertext expansion compared to symmetric cryptography. Transciphering framework is a hybrid framework converting symmetric ciphertext to HE ciphertext, which is proposed to address the issue of the ciphertext expansion and the client-side computational overload.

In this paper, we propose two transciphering frameworks: RtF (Real-to-Finite-field) framework, and BtE (Binary-to-Extension-field) framework. The RtF framework is the first transciphering framework for real numbers. The main idea behind this construction is to combine the CKKS and the FV homomorphic encryption schemes, and use a stream cipher using modular arithmetic in between. As a result, real numbers can be encrypted without significant ciphertext expansion or computational overload on the client side. The BtE framework fully exploits the structure of extension fields in order to use the whole inner-slot storage inside HE ciphertexts. Although the BtE framework only supports homomorphic

operation on a binary extension field, the digit extraction function at the end of the BtE framework makes the resulting HE ciphertexts operate on a usual binary space homomorphically.

As an instantiation of the stream cipher in our framework, we propose three HE-friendly ciphers: HERA, Rubato, and eHERA. The HERA cipher features a simple randomized key schedule and sparse linear layers in order to be efficiently evaluated with batching technique. The Rubato cipher is a mixture of block cipher and LWE encryption. The added noise has an effect of increasing the algebraic degree so that the multiplicative depth becomes lower than before. We also propose a new HE-friendly cipher eHERA, a variant of HERA, as an instantiation of the stream cipher operating on a binary extension field for the BtE framework. The BtE framework combined with eHERA outperforms existing HE-friendly ciphers in terms of both throughput and latency in the client-side.

Keywords Stream cipher, homomorphic encryption, HE-friendly cipher, transciphering framework

KAIST

Contents

Contents i
List of Tables iv
List of Figures vi

Chapter 1. Introduction **1**
1.1 Transciphering Framework 1
1.2 Scenarios for Transciphering Framework 2
1.3 Our Results 4
1.3.1 Transciphering Framework for Real Numbers 4
1.3.2 Efficient Transciphering of Binary Numbers 5

Chapter 2. Preliminaries **6**
2.1 Notations 6
2.2 Homomorphic Encryption 6
2.2.1 Lattice Background 6
2.2.2 LWE and RLWE 7
2.2.3 Homomorphic Encryption Schemes 7
2.2.4 Packing Methods for Cipher Evaluation 10
2.2.5 Some Notable Homomorphic Operations 10
2.3 Symmetric Cryptanalysis 11
2.3.1 Trivial Linearization 11
2.3.2 GCD Attack 12
2.3.3 Gröbner Basis Attack 12
2.3.4 Interpolation Attack 13
2.3.5 Linear Cryptanalysis on Non-binary Ciphers 13
2.3.6 Differential Cryptanalysis 14
2.4 HE-friendly Ciphers 14
2.4.1 LowMC 14
2.4.2 Kreyvium 15
2.4.3 FLIP and FiLIP 16
2.4.4 Rasta and Its Variants 18

Chapter 3. Transciphering of Real Numbers **21**
3.1 RtF Transciphering Framework 21
3.1.1 Overview of the Framework 21

KAIST

3.1.2 Specification . . . 23
3.1.3 Message Precision . . . 26
3.2 HERA: Batching-friendly Ciphers . . . 27
3.2.1 Specification . . . 28
3.2.2 Design Rationale . . . 29
3.3 Security Analysis of HERA . . . 31
3.3.1 On the Number of Monomials in HERA . . . 32
3.3.2 Trivial Linearization . . . 33
3.3.3 GCD Attack . . . 34
3.3.4 Gröbner Basis Attack . . . 34
3.3.5 Interpolation Attack . . . 36
3.3.6 Linear Cryptanalysis . . . 36
3.3.7 Differential Cryptanalysis and Its Variants . . . 37
3.4 Rubato: Noisy Ciphers for the RtF Framework . . . 38
3.4.1 Specification . . . 39
3.4.2 Design Rationale . . . 41
3.4.3 Parameter Selection . . . 42
3.5 Security Analysis of Rubato . . . 42
3.5.1 Cryptanalysis Based on Symmetric Primitive Analysis . . . 43
3.5.1.1 On the Number of Monomials in Rubato . . . 43
3.5.1.2 Trivial Linearization . . . 44
3.5.1.3 GCD Attack . . . 44
3.5.1.4 Gröbner Basis Attack . . . 44
3.5.1.5 Interpolation Attack . . . 45
3.5.1.6 Linear Cryptanalysis . . . 45
3.5.1.7 Differential Cryptanalysis and Its Variants . . . 46
3.5.2 Cryptanalysis Based on LWE Analysis . . . 47
3.5.2.1 Exhaustive Search . . . 48
3.5.2.2 Lattice Attacks . . . 48
3.5.2.3 BKW Attack . . . 50
3.5.2.4 Arora-Ge Attack . . . 51
3.6 Performance Evaluation of the RtF Framework . . . 52
3.6.1 Parameter Selection . . . 52
3.6.2 Benchmark of **HERA** . . . 53
3.6.3 Benchmark of **Rubato** . . . 54
3.6.4 Performance Comparison . . . 55

KAIST

Chapter 4. Efficient Transciphering of Binary Numbers **56**
4.1 BtE Transciphering Framework 56
4.1.1 Digit Extraction 57
4.1.2 Specification of the Framework 58
4.2 eHERA: Ciphers over Extension Fields 58
4.2.1 Design Change of **eHERA** from **HERA** 59
4.2.2 Specification 59
4.2.3 Parameter Selection 61
4.3 Security Analysis of eHERA 61
4.3.1 On the Number of Monomials in eHERA 62
4.3.2 Trivial Linearization 65
4.3.3 Interpolation Attack 66
4.3.4 GCD Attack 66
4.3.5 Gröbner Basis Attack. 66
4.3.6 Linear Cryptanalysis 69
4.3.7 Differential Cryptanalysis 69
4.4 Performance Evaluation of the BtE Framework 70
4.4.1 Benchmark 70
4.4.2 Comparison to Other Binary Ciphers 70

Chapter 5. Conclusion **73**

Chapter A. Additional Information on Security Analysis **81**
A.1 Branch Number of the Linear Layer in Rubato 81
A.2 Additional Plots on Security Analysis 83

Acknowledgments in Korean **85**

Curriculum Vitae in Korean **86**

KAIST

List of Tables

3.1 This table presents experimental error of HalfBoot for various Δ'/Q_0. The value ε is the mean error occurred by HalfBoot. The experiment is done by using parameter `Par-128` in Table 3.10 except Δ'/Q_0. 27

3.2 Comparisons of different types of maps in terms of evaluation time and noise consumption. 30

3.3 Recommended number of rounds with respect to each attack. 31

3.4 $d_{reg}(r,k)$ for a semi-regular system of $16rk$ cubic equations in $16(r-1)k+16$ variables. . 35

3.5 $d'_{reg}(r,g)$ for a semi-regular system of $16r$ cubic equations in $16r$ variables with g guesses. 35

3.6 Comparison of HE-friendly ciphers operating on modular domains, where the modulus q is set to 25 bits. "#(Key words)" is the number of key words in $\mathbb{Z}_q$ and "#(Multiplications)" (resp. "Random bits") is the number of multiplications (resp. random bits) required to generate a single component of a ciphertext. 38

3.7 Selected parameters of Rubato. 42

3.8 The log of the complexity of the attacks on Rubato. The upward sign (↑) implies that the complexity is larger than 2^{1000}. The linear algebra constant ω is assumed to be 2. 43

3.9 The expected time complexity of the BKZ algorithm in terms of BKZ block size β and root-Hermite factor δ. 49

3.10 Selected sets of parameters used in our implementation. The rest of the bootstrapping parameters is set to be $(h, K, R, d_{\sin}, d_{\arcsin}) = (192, 25, 2, 63, 0/7)$. 53

3.11 Performance of the RtF transciphering framework with HERA. 54

3.12 Client-side performance of the RtF transciphering framework with Rubato. 54

3.13 Server-side performance of the RtF transciphering framework with Rubato. 54

3.14 Comparison of the RtF transciphering framework with Rubato to previous measures supporting homomorphic encryption of approximate numbers. All the experiments are done with 128-bit security. Parameter N in parentheses implies the dimension of LWE. The parameter p stands for the bits of precision. 55

4.1 Some selected parameters of eHERA. The ciphertext moduli are tightly chosen, which means that further computation after transciphering requires larger moduli. "Ctxt. Mod." stands for the log size of ciphertext modulus. 61

4.2 Recommended number of rounds with respect to each attack. 61

4.3 Values of $\log S$ according to the number of rounds when $n = 16$, $e = 21$ and $t = 17$. S is a lower bound of the number of monomials. 64

4.4 The log of the time complexity according to the the number of rounds r and the number of guessed variables g. 65

4.5 Lower bound of log of the time complexity of the Gröbner basis attack for a semi-regular system of the key variables according the the number of rounds r and g guesses. 67

4.6 $d_{reg}(r,k)$ for a semi-regular system of $16rk$ equations of degree t in $16(r-1)k+16$ variables and log of the time complexity of the Gröbner basis attack for the system with $w = 2$. . . 68

KAIST

4.7 $d'_{reg}(r, g)$ for a semi-regular system of $16r$ equations of degree t in $16r$ variables with g guesses and log of the time complexity of the Gröbner basis attack for the system with $w = 2$. 68
4.8 The client-side and the server-side performance of the BtE framework equipped with eHERA. 70
4.9 Performance comparison between HE-friendly ciphers over $\mathbb{Z}_2$. The choice of parameters can be found in Table 4.10. 71
4.10 Parameters used in the comparison. These parameters are originally used in [43]. 72

KAIST

List of Figures

1.1 The (basic) transciphering framework. Homomorphic operations are performed in the boxes with thick lines. 2

2.1 Transciphering framework when a stream cipher is used. The keys are omitted when the usage is clear. 16

3.1 The RtF transciphering framework. Homomorphic encryption and evaluation is performed in the boxes with thick lines. Operations in the boxes with rounded corners do not use any secret information. The vertical dashed line distinguishes the client-side and the server-side computation, while the horizontal dashed line distinguishes the offline and the online computation. The client sends ciphertexts block by block, while the server gathers B ciphertext blocks and recovers the CKKS-encrypted message of the ciphertexts. 22
3.2 The round function of HERA. Operations in the box with dotted (resp. thick) lines are public (resp. secret). "MC" and "MR" represent MixColumns and MixRows, respectively. . 28
3.3 State of HERA. Each square stands for the component in $\mathbb{Z}_t$. 29
3.4 Definition of MixColumns and MixRows in HERA. For $c \in \{0, 1, 2, 3\}$, x_{ij} and y_{ij} are defined as in Figure 3.3. 29
3.5 Complexity of the Gröbner basis attack with $\omega = 2$ and $t = 2^{16} + 1$. 35
3.6 The round function of Rubato. Operations in the box with dotted (resp. thick) lines are public (resp. secret). "MC" and "MR" represent MixColumns and MixRows, respectively. . 39
3.7 State of Rubato. Each square stands for the component in $\mathbb{Z}_q$. 40
3.8 Definition of MixColumns and MixRows. For $c \in \{1, 2, \ldots, v\}$, x_{ij} and y_{ij} are defined as in Figure 3.7. 40

4.1 Pictorial description of DigitExt. Each 16-cell bar stands for a homomorphic ciphertext, 4-cell bar stands for a single slot of $\mathbb{F}_{2^e}$, and each cell stands for the digit (bit for binary). Each digit is extracted and moved to the LSB of the slot. 56
4.2 The round function of eHERA. Operations in the box with dotted (resp. thick) lines are public (resp. secret). "MC" and "MR" represent MixColumns and MixRows, respectively. . 60
4.3 Definition of MixColumns and MixRows in eHERA. For $c \in \{0, 1, 2, 3\}$, x_{ij} and y_{ij} are defined as in Figure 3.3. 60
4.4 Client-side and server-side performances of HE-friendly ciphers. 72

A.1 Diagram of state change in Rubato. 82
A.2 Pictorial representation of an example satisfying $B_d = 2v$. 83
A.3 The appropriate choice of the number of rounds r and the width of discrete Gaussian distribution αq for a given security level. 83
A.4 The log of time complexity of the Arora-Ge attack as a function in $t\alpha q$. 84

KAIST

Chapter 1. Introduction

1.1 Transciphering Framework

Cryptography has been extensively used to protect data when they are stored (data-at-rest) or when they are being transmitted (data-in-transit). We also see increasing needs that data should be protected while they are being used, since they are often processed within untrusted environments. For example, organizations might want to migrate their computing environment from on-premise to public cloud, and to collaborate with their data without necessarily trusting each other. If data are protected by an encryption scheme which is *homomorphic*, then the cloud would be able to perform meaningful computations on the encrypted data, supporting a wide range of applications such as machine learning over a large amount of data preserving its privacy.

Homomorphic Encryption. An encryption scheme that enables addition *or* multiplication over encrypted data without decryption key is called a *homomorphic encryption* (HE) scheme. An HE scheme is called partially homomorphic if it supports only single operation among addition and multiplication. If an HE scheme supports both addition and multiplication for restricted (resp. indefinite) class of circuits, it is called somewhat (resp. fully) homomorphic encryption. In this paper, homomorphic encryption refers to the one supports both addition and multiplication. Since the emergence of Gentry's blueprint [53], there has been a large amount of research in this area [25, 49, 37, 56]. Various applications of HE to medical, genomic, and financial tasks have also been proposed [36, 33, 71, 87].

Unfortunately, current HE schemes including CKKS [35] commonly suffer from heavy computational and memory overload. The encryption/decryption speed is relatively slow compared to conventional encryption schemes, and it implies that HE is inadequate for bulk encryption. Also, ciphertext expansion seems to be an intrinsic problem of homomorphic encryption due to the noise used in the encryption algorithm. Although the ciphertext expansion has been significantly reduced down to the order of hundreds in terms of the ratio of a ciphertext size to its plaintext size since the invention of the batching technique [55], it does not seem to be acceptable from a practical viewpoint. Furthermore, this ratio becomes even worse when it comes to encryption of a short message; encryption of a single bit might result in a ciphertext of a few megabytes.

Transciphering Framework. To address the issue of the ciphertext expansion and the client-side computational overload, a hybrid framework, also called a *transciphering framework*, has been proposed [87] (see Figure 1.1). A transciphering framework converts a symmetric ciphertext into a homomorphic ciphertext on the server-side, reducing computational and communication overload on the client-side. In the client-sever model, a client generates a secret-public-evaluation key triple (sk, pk, evk) for HE. The client sends the public key and the evaluation key to the server. The client encrypts a message $\mathbf{m}$ using a symmetric cipher E with a symmetric key $\mathbf{k}$; this symmetric key is encrypted using an HE algorithm $\mathsf{Enc}^{\mathsf{HE}}_{pk}$. The resulting ciphertexts $\mathbf{c} = \mathsf{E}_{\mathbf{k}}(\mathbf{m})$ and $\mathsf{Enc}^{\mathsf{HE}}_{pk}(\mathbf{k})$ are stored in the server.

When the server wants to compute $\mathsf{Enc}^{\mathsf{HE}}_{pk}(\mathbf{m})$ (for computation over encrypted data), it first computes $\mathsf{Enc}^{\mathsf{HE}}_{pk}(\mathbf{c})$ for the corresponding ciphertext $\mathbf{c}$. Then the server homomorphically evaluates E^{-1} over $\mathsf{Enc}^{\mathsf{HE}}_{pk}(\mathbf{c})$ and $\mathsf{Enc}^{\mathsf{HE}}_{pk}(\mathbf{k})$, securely obtaining $\mathsf{Enc}^{\mathsf{HE}}_{pk}(\mathbf{m})$.

Given a symmetric cipher with low multiplicative depth and complexity, this framework has the

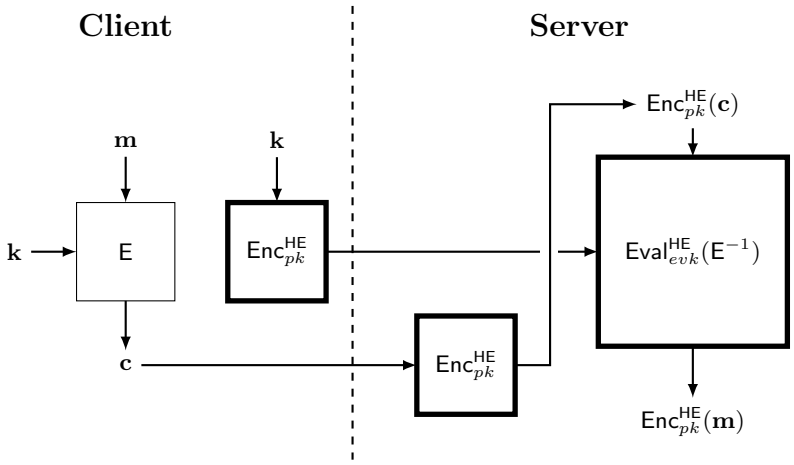


Figure 1.1: The (basic) transciphering framework. Homomorphic operations are performed in the boxes with thick lines.

following advantages on the client side.

- A client does not need to encrypt all its data using an HE algorithm (except the symmetric key). All the data can be encrypted using only a symmetric cipher, significantly saving computational resources in terms of time and memory.
- Symmetric encryption does not result in ciphertext expansion, so the communication overload between the client and the server will be significantly low compared to using any homomorphic encryption scheme alone.

KAIST

All these merits come at the cost of computational overload on the server side. That said, this trade-off would be worth considering in practice since servers are typically more powerful than clients.

HE-friendly Ciphers. Symmetric ciphers are built on top of linear and non-linear layers, and in a conventional environment, there has been no need to take different design principles for the two types of layers with respect to their implementation cost. However, when a symmetric cipher is combined with BGV/FV-style HE schemes [25, 49] in a transciphering framework, homomorphic addition becomes way cheaper than homomorphic multiplication in terms of computation time and noise growth. With this observation, the efficiency of an HE-friendly cipher is evaluated by its multiplicative complexity and depth. In an arithmetic circuit, its multiplicative complexity is represented by the number of multiplications (ANDs in the binary case). Multiplicative depth is the depth of multiplications in the tree representing the arithmetic circuit, closely related to the noise growth in the HE-ciphertexts. These two metrics have brought a new direction in the design of symmetric ciphers: to use simple nonlinear layers at the cost of highly randomized linear layers as adopted in the design of FLIP [85] and Rasta [44].

1.2 Scenarios for Transciphering Framework

Although transciphering framework can be considered at any place that HE is used, it is not a panacea for every privacy problem since it takes more time than HE-only until $\mathsf{Enc}^{\mathsf{HE}}(\mathbf{m})$ is finally obtained. In this section, we suggest two appropriate scenarios for transciphering frameworks.

Two-party Scenario. There are two parties processing transciphering framework: client and server. In this scenario, the main purpose of the framework is outsourcing computation of the client's data to the

server while the privacy of data is protected against the server. Since the data should be protected against the server, the symmetric key and the secret key of HE are generated by the client. The corresponding evaluation key (and public key if needed) is initially transferred to the server.

As the current HE schemes still suffer from a large amount of computational overhead, we do not recommend to use HE or transciphering framework when the function to be evaluated is public and simple. When the server has secret functions to evaluate (e.g., ML inference as a service) or evaluating the function requires too much resources compared to capacity of the client, transciphering framework is a plausible idea for (oblivious) evaluation of those functions in the following sense.

- When the data are planned to be stored for a long time, transciphering framework can reduce the memory consumption caused by the encrypted data.
- If the client has presumably constrained resources (e.g., Internet of Things), transciphering framework can alleviate the load of the client in terms of computational power, memory, and entropy gathering.
- If the data are not planned to be packed in a fixed way, since any (limited-size) collection of symmetric ciphertexts can be transciphered into a single HE ciphertext, transciphering framework is more forward-compatible to process the data.

Multi-party Scenario. In this scenario, there are three types of parties: data providers, an evaluator, and key holder(s). The data providers provide data, wishing privacy of the data is protected. The evaluator processes the data received from the data providers without knowing about the data. The key holders – may be singular or plural – have the secret key of HE. If there are two or more key holders, then each key holder has a share of the secret key and the evaluation key and the public key are generated by multi-party computation. It is called multi-party homomorphic encryption (MHE) [86]. Assuming no conspiracy of the evaluator and all the key holders, the privacy of the data will be protected. Transciphering framework in the multi-party scenario proceeds as follows.

1. The key holders generate secret key sk, public key pk, and evaluation key evk.
2. The key holders send pk to the data providers and (pk, evk) to the evaluator.
3. Each data provider generates its own symmetric key $\mathbf{k}_i$, and sends HE-encrypted symmetric key $\mathcal{K}_i = \mathsf{Enc}^{\mathsf{HE}}_{pk}(\mathbf{k}_i)$ to the evaluator.
4. Each data provider encrypts the data using symmetric cipher E, and sends the symmetric ciphertext $\mathbf{c} = \mathsf{E}_{\mathbf{k}_i}(\mathbf{m})$.
5. The evaluator transciphers $\mathbf{c}$ to $\mathsf{Enc}^{\mathsf{HE}}_{pk}(\mathbf{m})$ using $\mathcal{K}_i$.
6. The evaluator homomorphically processes $\mathsf{Enc}^{\mathsf{HE}}_{pk}(\mathbf{m})$ and sends the result $\mathsf{Enc}^{\mathsf{HE}}_{pk}(f(\mathbf{m}))$ to the key holders.
7. The key holders decrypt $\mathsf{Enc}^{\mathsf{HE}}_{pk}(f(\mathbf{m}))$ and send it to a recipient.

This process can be used for secure data collection. The evaluator can collect data from the public preserving their privacy, and process the data in an offline manner. Similarly as the two-party scenario, transciphering framework can offer better usability compared to HE-only case when a small amount of data is steadily collected for a long time.

KAIST

1.3 Our Results

In this dissertation, we present two categories of results: transciphering framework for real numbers, and efficient transciphering of binary numbers. In the literature, there has been no transciphering framework for real numbers. As wide range of real-world applications involve computation of real numbers, it narrows down the applicability of transciphering framework. We present new transciphering framework for approximate homomorphic encryption, dubbed RtF (Real-to-Finite-field) framework, and two ciphers HERA and Rubato for the RtF framework.

We then present a new transciphering framework BtE which efficiently transcipher binary numbers, and an HE-friendly cipher eHERA for the BtE framework. Previous HE-friendly ciphers have usually been designed by focusing on AND depth and the number of ANDs, while these measures are not always directly connected to homomorphic evaluation time. We focus on the structure of homomorphic encryption to reduce the essential homomorphic evaluation time of the cipher.

1.3.1 Transciphering Framework for Real Numbers

RtF Transciphering Framework. The CKKS scheme and the FV scheme have similar plaintext spaces; roughly speaking, plaintext space of both schemes corresponds to $\mathbb{Z}[X]/(\Phi(X))$ with bounded coefficients, where $\Phi(X)$ is a cyclotomic polynomial. It implies that real data can be encoded into the FV-plaintext space. Furthermore, ciphertexts of two schemes can converted to each other by bootstrapping. The main idea of the RtF framework is to transcipher real data by FV and convert it to CKKS-ciphertext by CKKS-bootstrapping. The details differ from the main idea slightly for the efficient encryption of small message. As a result, real numbers can be encrypted without significant ciphertext expansion or computational overload on the client side.

Stream Cipher HERA. As an instantiation of the stream cipher in our framework, we propose a new HE-friendly cipher, dubbed HERA, and extensively analyze its security and efficiency. When designing HERA, we focused on the non-multiplicative components in HE-friendly ciphers. Contrary to popular facts, homomorphically evaluating linear function is turned out to be quite time-consuming. We adopt fixed low-weight (or sparse) linear layer and a simple randomized key schedule which is component-wise multiplication by random elements to master key. Furthermore, we designed HERA to use a smaller number of random bits compared to recent HE-friendly ciphers using randomized linear layers, and HERA features a competitive encryption speed on the client side.

With the RtF transciphering framework combined with HERA at the 128-bit security level, we achieve small ciphertext expansion ratio with a range of 1.23 to 1.54, which is at least 23 times smaller than using (symmetric) CKKS-only, assuming the same precision bits and the same level of ciphertexts at the end of the framework. We also achieve 1.6 μs and 21.7 MB/s for latency and throughput on the client side, which are 9085 times and 17.8 times faster than the CKKS-only environment, respectively.

Noisy Cipher Rubato. Rubato is a family of *noisy* ciphers with a novel design strategy of introducing noise to a symmetric cipher of a low algebraic degree. With this strategy, the multiplicative depth and complexity of the cipher is significantly reduced, compared to existing HE-friendly ciphers, without degrading the overall security. More precisely, given a moderate block size (16 to 64), Rubato enjoys a low multiplicative depth (2 to 5) and a small number of multiplications per encrypted word (2.1 to 6.25) at the cost of slightly larger ciphertext expansion (1.26 to 1.31). The security of Rubato is supported by comprehensive analysis including symmetric and LWE cryptanalysis. Compared to HERA within the

KAIST

RtF framework, client-side and server-side throughput is improved by 22.9% and 32.2%, respectively, at the cost of only 1.6% larger ciphertext expansion.

1.3.2 Efficient Transciphering of Binary Numbers

BtE Transciphering Framework. Recent RLWE-based HE schemes usually support SIMD (single instruction, multiple data) operation, which is called batching. For *good* enough positive integers q and N, q-ary data are computed in ring $\mathbb{Z}_q^N$ with respect to component-wise addition and multiplication. When the data are binary, the data will reside not in $\mathbb{Z}_2^N$, but in $\mathbb{F}_{2^e}^d$ for $ed = N$. The BtE framework fully exploits the structure of extension fields in order to use the whole inner-slot storage inside HE ciphertexts. The BtE framework transciphers the element in a binary extension field, and extracts all the digits (in the extension field element) to distinct HE-ciphertexts. It allows to evaluate AND to the resulting HE ciphertexts.

Stream Cipher eHERA. As an instantiation of the stream cipher operating on a binary extension field for the BtE framework, we proposed a new HE-friendly cipher eHERA, an extension field variant of HERA. Although the design of eHERA mostly follows HERA, we had to re-analyze the security because of the field extension structure. The BtE framework combined with eHERA outperforms existing HE-friendly ciphers for binary data in terms of both throughput and latency in the client-side. It delivers up to 64258 times higher throughput and up to 24671 times faster latency in the client-side. It also enjoys up to 750000 times higher throughput in the server-side without significantly degrading server-side latency.

KAIST

Chapter 2. Preliminaries

2.1 Notations

Throughout the paper, bold lowercase letters (resp. bold uppercase letters) denote vectors (resp. matrices). For a real number r, $\lfloor r \rceil$ denotes the nearest integer to r, rounding upwards in case of a tie. For an integer q, we identify $\mathbb{Z}_q$ with $\mathbb{Z} \cap (-q/2, q/2]$, and for any real number z, $[z]_q$ denotes the mod q reduction of z into $(-q/2, q/2]$. The notation $\lfloor \cdot \rceil$ and $[\cdot]_q$ are extended to vectors (resp. polynomials) to denote their component-wise (resp. coefficient-wise) reduction. For a complex vector $\mathbf{x}$, its ℓ_p-norm is denoted by $\|\mathbf{x}\|_p$. When we say ℓ_p-norm of a polynomial, it means that the ℓ_p-norm of the coefficient vector of the polynomial.

Usual dot products of vectors are denoted by $\langle \cdot, \cdot \rangle$. Throughout the paper, ζ and ξ denote a $2N$-th primitive root of unity over the complex field $\mathbb{C}$, and the finite field $\mathbb{Z}_p$, respectively, for fixed parameters N and p. We denote the multiplicative group of $\mathbb{Z}_p$ by $\mathbb{Z}_p^\times$. The set of strings of arbitrary length over a set S is denoted by S^*. For two vectors (strings) $\mathbf{a}$ and $\mathbf{b}$, their concatenation is denoted by $\mathbf{a} \| \mathbf{b}$. For an integer n, $\phi(n)$ denotes the Euler's totient function. For a set S, we will write $a \leftarrow S$ to denote that a is chosen from S uniformly at random. For a probability distribution $\mathcal{D}$, $a \leftarrow \mathcal{D}$ will denote that a is sampled according to the distribution $\mathcal{D}$. Unless stated otherwise, all logarithms are to the base 2.

KAIST

2.2 Homomorphic Encryption

2.2.1 Lattice Background

Let $\mathbf{B} \in \mathbb{R}^{m \times n}$ be a full rank matrix. The lattice $L(\mathbf{B})$ generated by $\mathbf{B}$ is defined by $L(\mathbf{B}) = \{\mathbf{B} \cdot \mathbf{x} : \mathbf{x} \in \mathbb{Z}^n\}$. The matrix $\mathbf{B}$ is called a basis of $L(\mathbf{B})$. The i-th successive minimum $\lambda_i(L)$ of a lattice L is the smallest value t such that at least i linearly independent lattice vectors of length $\leq t$ exist in L. The shortest vector problem (SVP) is finding a shortest non-zero vector of L from a given basis. The γ-unique shortest vector problem (γ-uSVP) is finding the shortest non-zero vector of L provided that $\lambda_2(L) > \gamma\lambda_1(L)$. The γ-bounded distance decoding (BDD_γ) problem is finding a lattice point in L closest to a target vector $\mathbf{t}$ provided that $\mathrm{dist}(\mathbf{t}, L) \coloneqq \min_{\mathbf{x} \in L} \mathrm{dist}(\mathbf{t}, \mathbf{x}) \leq \gamma \cdot \lambda_1(L)$.

Hermite Factor. Given an n-dimensional lattice L with a basis $\mathbf{B}$, a *root-Hermite factor* δ of the basis $\mathbf{B}$ is defined by $\delta^{n-1} = \|\mathbf{b}_1\| / (\det(L(\mathbf{B}))^{1/n}$ where $\mathbf{b}_1$ is the shortest vector of the basis and $\det(L(\mathbf{B})) = \sqrt{\mathbf{B}^\intercal \mathbf{B}}$. If δ is smaller, then the basis includes a shorter vector in the lattice.

Gaussian Heuristic. Gaussian heuristic (GH) is a heuristic on how many lattice points are contained in a *nice* object. Given a measurable set $S \subset \mathbb{R}^n$ and a full-rank lattice $L \subset \mathbb{R}^n$, the number of lattice points of L in S is approximated by $\#(S \cap L) = \mathrm{vol}(S)/\det(L)$ where $\mathrm{vol}(S)$ is the volume of S. If S is an n-dimension ball of radius R, then the equation becomes $\#(S \cap L) = v_n R^n / \det(L)$ where v_n is the volume of the unit n-ball. With this heuristic, the norm of the shortest vector of L can be approximated by

$$GH(L) = \left(\frac{v_n R^n}{\det(L)} \right)^{\frac{1}{n}} \approx \sqrt{\frac{n}{2\pi e}} \cdot (\pi n)^{\frac{1}{2n}}$$

by Stirling's formula. For random lattices, GH is precise within the error at most 5% [52].

Geometric Series Assumption. Schnorr claimed that the Gram-Schmidt orthogonalized norm of a BKZ-reduced basis behaves as a geometric series, which is called geometric series assumption (GSA) [96]. For a BKZ-reduced basis $\mathbf{B} = [\mathbf{b}_1, \cdots, \mathbf{b}_n]$ and its orthogonalization $\mathbf{B}^* = [\mathbf{b}_1^*, \cdots, \mathbf{b}_n^*]$, it satisfies that $\|\mathbf{b}_1^*\|/\|\mathbf{b}_i^*\| = r^{i-1}$ for all $1 \leq i \leq n$ where r is a constant.

2.2.2 LWE and RLWE

Let n and q be positive integers. Let χ be a probability distribution over $\mathbb{Z}$. For an unknown vector $\mathbf{s} \in \mathbb{Z}_q^n$, the LWE (learning with errors) distribution $L_{\mathbf{s},\chi}$ over $\mathbb{Z}_q^n \times \mathbb{Z}_q$ is obtained by sampling a vector $\mathbf{a} \leftarrow \mathbb{Z}_q^n$ and an error $e \leftarrow \chi$, and outputting

$$(\mathbf{a}, b = [\langle \mathbf{a}, \mathbf{s} \rangle + e]_q) \in \mathbb{Z}_q^n \times \mathbb{Z}_q.$$

The search-LWE problem is to find $\mathbf{s} \in \mathbb{Z}_q^n$ when independent samples $(\mathbf{a}_i, b_i)$ are obtained according to $L_{\mathbf{s},\chi}$. The decision-LWE problem is to distinguish the distribution $L_{\mathbf{s},\chi}$ from the uniform distribution over $\mathbb{Z}_q^n \times \mathbb{Z}_q$.

For a positive real $\alpha > 0$, the discrete Gaussian distribution $D_{\alpha q}$ is a probability distribution on $\mathbb{Z}$ defined by

$$\Pr[y \leftarrow D_{\alpha q} : y = x] \propto \exp\left(-\pi x^2/(\alpha q)^2\right)$$

for each $x \in \mathbb{Z}$. The discrete Gaussian distribution is a popular candidate of the distribution χ.

Lyubashevsky et al. introduced the ring version of the LWE problem, which is also called ring-LWE (RLWE) [80]. For a positive integer M, let $\Phi_M(X)$ be the M-th cyclotomic polynomial of degree $N = \phi(M)$. Let $\mathcal{R} = \mathbb{Z}[X]/(\Phi_M(X))$ and let $\mathcal{R}_q = \mathcal{R}/q\mathcal{R} = \mathbb{Z}_q[X]/(\Phi_M(X))$. The (decisional) RLWE problem is to distinguish the distribution of

$$(a, b = [a \cdot s + e]_q) \in \mathcal{R}_q^2$$

from the uniform distribution over $\mathcal{R}_q^2$, where $s \in \mathcal{R}$ is a secret polynomial, a is sampled uniformly at random from $\mathcal{R}_q$ and e is sampled according to a certain error distribution over $\mathcal{R}$. The security of the BGV, FV and CKKS schemes is based on the hardness assumption of the RLWE problem.

2.2.3 Homomorphic Encryption Schemes

From the seminal work of Gentry [53], many homomorphic encryption schemes have been proposed: leveled homomorphic encryption schemes [25, 24, 49], homomorphic encryption schemes with frequent bootstrapping [56, 46, 37], and approximate homomorphic encryption schemes [35]. As designing our transciphering framework and HE-friendly ciphers for the BGV [25], FV [49], and CKKS [35] schemes, we briefly review those schemes in this section. For more details, we refer to the original papers.

It is remarkable that those three schemes use the same ciphertext space; for a positive integer Q, an integer M, and $N = \phi(M)$, both schemes use

$$\mathcal{R}_Q^2 = \mathbb{Z}_Q[X]/(\Phi_M(X)) \times Z_Q[X]/(\Phi_M(X))$$

as their ciphertext spaces, where $\Phi_M(X)$ denotes the M-th cyclotomic polynomial. They also use similar

algorithms for key generation, encryption, decryption, and homomorphic addition and multiplication. However, the BGV and the FV schemes support *exact* computation, while the CKKS scheme supports *approximate* computation over the real numbers by taking different strategies to efficiently encode messages.

Encoders and Decoders. In BGV and FV, plaintexts are elements of the ring $\mathcal{R}_p$. Assuming that p does not divide M, we note that the image $\overline{\Phi}_M(X)$ in $\mathbb{Z}_p[X]$ of cyclotomic polynomial $\Phi_M(X)$ factors into ℓ irreducible factors,

$$\overline{\Phi}_M(X) = F_1(X) \dots F_\ell(X),$$

where $F_i(X) \in \mathbb{Z}_p[X]$ are distinct degree-d polynomials where d is the multiplicative order of $p \pmod M$. By the Chinese remainder theorem, we have a $\mathbb{Z}_p$-algebra isomorphism

$$\begin{aligned} \mathcal{R}_p &\longrightarrow \mathbb{Z}_p[X]/(F_1(X)) \times \cdots \times \mathbb{Z}_p[X]/(F_\ell(X)) \\ f(X) &\longmapsto (f(X) \bmod F_1(X), \dots, f(X) \bmod F_\ell(X)) \,. \end{aligned}$$

Now, for a given irreducible polynomial $F(X)$ of degree d over $\mathbb{Z}_p$, we define

$$\mathbb{E} = \mathbb{Z}_p[X]/(F(X)).$$

Since $\mathbb{E}$ is isomorphic to $\mathbb{Z}_p[X]/(F_i(X))$ for all i, we can get a $\mathbb{Z}_p$-algebra isomorphism $\mathcal{R}_p \longrightarrow \mathbb{E}^\ell$. By the isomorphism, component-wise additions and multiplications on vectors in $\mathbb{E}^\ell$ is allowed on elements of $\mathcal{R}_p$. The isomorphism $\mathcal{R}_p \to \mathbb{E}^\ell$ is called the decoder. Specifically, if M is a power of two and $p \equiv 1 \pmod M$, then the decoder $\mathsf{Dcd}_\ell^{\{\mathsf{BGV},\mathsf{FV}\}}$ is defined by, for $m(X) = \sum_{k=0}^{\ell-1} a_k X^{kN/\ell} \in \mathcal{R}_p$,

$$\mathsf{Dcd}_\ell^{\{\mathsf{BGV},\mathsf{FV}\}}(m(X)) = (m(\alpha_0), \cdots, m(\alpha_{\ell-1})) \in \mathbb{Z}_p^\ell,$$

where $\alpha_i = \xi^{5^i \cdot N/\ell} \pmod t$ for $0 \le i \le \ell/2 - 1$ and $\alpha_i = \xi^{-5^{i-\ell/2} \cdot N/\ell} \pmod t$ for $\ell/2 \le i \le \ell - 1$[1]. The encoder $\mathbb{E}^\ell \to \mathcal{R}_p$ is defined by the inverse function of the decoder.

On the other hand, to support approximate computation, CKKS encoder maps a complex vector to a polynomial over $\mathbb{Z}$. Let Δ_{CKKS} be a positive real number (called a scaling factor in [35]). The CKKS encoder $\mathsf{Ecd}_{\ell/2}^{\mathsf{CKKS}} : \mathbb{C}^{\ell/2} \to \mathcal{R}$ is the (approximate) inverse of the decoder $\mathsf{Dcd}_{\ell/2}^{\mathsf{CKKS}} : \mathcal{R} \to \mathbb{C}^{\ell/2}$, where for $m(X) = \sum_{k=0}^{\ell-1} a_k X^{kN/\ell} \in \mathcal{R}$,

$$\mathsf{Dcd}_{\ell/2}^{\mathsf{CKKS}}(m(X)) = \Delta_{\mathsf{CKKS}}^{-1} \cdot (m(\beta_0), m(\beta_1), \cdots, m(\beta_{\ell/2-1})) \in \mathbb{C}^{\ell/2},$$

where $\beta_j = \zeta^{5^j \cdot N/\ell} \in \mathbb{C}$ for $0 \le j \le \ell/2 - 1$.

Algorithms. The descriptions of those three schemes have also been merged, so that one can easily compare the differences between those.

- Key generation: given a security parameter $\lambda > 0$, fix integers N, P, and $Q_0, \dots, Q_L$ such that Q_i divides Q_{i+1} for $0 \le i \le L-1$, and distributions $\mathcal{D}_{key}$, $\mathcal{D}_{err}$ and $\mathcal{D}_{enc}$ over $\mathcal{R}$ in a way that the resulting scheme is secure against any adversary with computational resource of $O(2^\lambda)$.

  1. Sample $a \leftarrow \mathcal{R}_{Q_L}$, $s \leftarrow \mathcal{D}_{key}$, and $e \leftarrow \mathcal{D}_{err}$.

[1] A primitive root of unity ξ exists if the characteristic p of the message space is an odd prime such that $p \equiv 1 \pmod M$.

2. The secret key is defined by $sk = (1, s) \in \mathcal{R}^2$, and the corresponding public key is defined by $pk = (b, a) \in \mathcal{R}_{Q_L}^2$, where
   - $b = [-a \cdot s + pe]_{Q_L}$ for BGV;
   - $b = [-a \cdot s + e]_{Q_L}$ for FV and CKKS.
3. Sample $a' \leftarrow \mathcal{R}_{P \cdot Q_L}$ and $e' \leftarrow \mathcal{D}_{err}$.
4. The evaluation key is defined by $evk = (b', a') \in \mathcal{R}_{P \cdot Q_L}^2$, where
   - $b' = [-a' \cdot s + pe' + Ps']_{P \cdot Q_L}$ for BGV;
   - $b' = [-a' \cdot s + e' + Ps']_{P \cdot Q_L}$ for FV and CKKS

   with $s' = [s^2]_{Q_L}$.

- Encryption: given a public key $pk \in \mathcal{R}_{Q_L}^2$ and a plaintext $m \in \mathcal{R}$,
  1. Sample $r \leftarrow \mathcal{D}_{enc}$ and $e_0, e_1 \leftarrow \mathcal{D}_{err}$.
  2. Compute $\mathsf{EncZero} = [r \cdot pk]_{Q_L}$.

  - For BGV, $\mathsf{Enc}^{\mathsf{BGV}}(pk, m) = [\mathsf{EncZero} + ([m]_p, 0) + (pe_0, pe_1)]_{Q_L}$
  - For FV, $\mathsf{Enc}^{\mathsf{FV}}(pk, m) = [\mathsf{EncZero} + (\Delta_{\mathsf{FV}} \cdot [m]_p, 0) + (e_0, e_1)]_{Q_L}$, where $\Delta_{\mathsf{FV}} = \lfloor Q_L/p \rceil$.
  - For CKKS, $\mathsf{Enc}^{\mathsf{CKKS}}(pk, m) = [\mathsf{EncZero} + (m, 0) + (e_0, e_1)]_{Q_L}$.

- Decryption: given a secret key $sk \in \mathcal{R}^2$ and a ciphertext $ct \in \mathcal{R}_{Q_l}^2$,

$$\mathsf{Dec}^{\mathsf{BGV}}(sk, ct) = [[\langle sk, ct \rangle]_{Q_l}]_p;$$
$$\mathsf{Dec}^{\mathsf{FV}}(sk, ct) = \left\lfloor \frac{t}{Q_l} [\langle sk, ct \rangle]_{Q_l} \right\rceil;$$
$$\mathsf{Dec}^{\mathsf{CKKS}}(sk, ct) = [\langle sk, ct \rangle]_{Q_l}.$$

- Addition: given ciphertexts ct_1 and ct_2 in $\mathcal{R}_{Q_l}^2$, their sum is defined by

$$ct_{add} = [ct_1 + ct_2]_{Q_l}.$$

- Multiplication: given ciphertexts $ct_1 = (b_1, a_1)$ and $ct_2 = (b_2, a_2)$ in $\mathcal{R}_{Q_l}^2$ and an evaluation key evk, their product is defined by
  - $ct_{mult} = \left[\left[(d_0, d_1) + \lfloor P^{-1} \cdot d_2 \cdot evk \rceil \right]_{Q_l} \right]_p$ where $(d_0, d_1, d_2) = [(b_1 b_2, a_1 b_2 + a_2 b_1, a_1 a_2)]_{Q_l}$ for BGV;
  - $ct_{mult} = \left[(d_0, d_1) + \lfloor P^{-1} \cdot d_2 \cdot evk \rceil \right]_{Q_l}$ where $(d_0, d_1, d_2) = \left[\left\lfloor \frac{t}{Q_l} (b_1 b_2, a_1 b_2 + a_2 b_1, a_1 a_2) \right\rceil \right]_{Q_l}$ for FV;
  - $ct_{mult} = \left[(d_0, d_1) + \lfloor P^{-1} \cdot d_2 \cdot evk \rceil \right]_{Q_l}$ where $(d_0, d_1, d_2) = [(b_1 b_2, a_1 b_2 + a_2 b_1, a_1 a_2)]_{Q_l}$ for CKKS.

- Rescaling (Modulus switching): given a ciphertext $ct \in \mathcal{R}_{Q_l}^2$ and $l' < l$, its rescaled ciphertext is defined by

$$\mathsf{Rescale}_{l \to l'}(ct) = \left[\left\lfloor \frac{Q_{l'}}{Q_l} \cdot ct \right\rceil \right]_{Q_{l'}}.$$

KAIST

2.2.4 Packing Methods for Cipher Evaluation

When homomorphically evaluating a symmetric cipher, an appropriate packing method will lead to better performance. Suppose that we want to evaluate a substitution-permutation cipher $\mathsf{E} : \mathbb{Z}_p^n \to \mathbb{Z}_p^n$ with matrix $\mathbf{A} \in \mathbb{Z}_p^{n \times n}$ in the permutation layer and $S : \mathbb{Z}_p^n \to \mathbb{Z}_p^n$ in the S-box layer using the FV scheme, where $n \mid \ell$ for the number of slots ℓ. Typically, encryption of long messages involves repeated application of fixed $\mathbf{A}$ and S for different data. To homomorphically multiply by matrix $\mathbf{A}$ to a large amount of data, we can consider two different packing methods on the server side: row-wise, and column-wise packing.

A straightforward approach is to evaluate ℓ input vectors by homomorphically computing

$$\mathbf{A} \cdot \begin{pmatrix} \mathcal{C}_0 \\ \mathcal{C}_1 \\ \vdots \\ \mathcal{C}_{n-1} \end{pmatrix}$$

for encrypted arrays $\mathcal{C}_i = \mathsf{Enc}^{\mathsf{FV}}(\mathsf{Ecd}^{\mathsf{FV}}([m_0^{(i)}, \ldots, m_{\ell-1}^{(i)}]))$, $i = 0, \ldots, n-1$. This is to semantically evaluate

$$\mathbf{A} \cdot \begin{pmatrix} m_0^{(0)} & m_1^{(0)} & \cdots & m_{\ell-1}^{(0)} \\ m_0^{(1)} & m_1^{(1)} & \cdots & m_{\ell-1}^{(1)} \\ & \vdots & \ddots & \vdots \\ m_0^{(n-1)} & m_1^{(n-1)} & \cdots & m_{\ell-1}^{(n-1)} \end{pmatrix}$$

by applying homomorphic addition and multiplication, where $m_j^{(i)} \in \mathbb{Z}_p$ for $0 \le j \le \ell - 1$. We call this method *row-wise packing*.

KAIST

Alternatively, we can evaluate ℓ/n input vectors by $\mathsf{Mul}^{\mathsf{FV}}(\mathbf{A}, \mathcal{C})$ where it semantically evaluates

$$\begin{pmatrix} \mathbf{A} & \mathbf{0} & \ldots & \mathbf{0} \\ \mathbf{0} & \mathbf{A} & \ldots & \mathbf{0} \\ & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{A} \end{pmatrix} \cdot \begin{pmatrix} m_0 \\ m_1 \\ \vdots \\ m_{\ell-1} \end{pmatrix}$$

in a single ciphertext by applying rotation as well as homomorphic operations to an encrypted array

$$\mathcal{C} = \mathsf{Enc}^{\mathsf{FV}}(\mathsf{Ecd}^{\mathsf{FV}}([m_0, \ldots, m_{\ell-1}])).$$

We call this method *column-wise packing*. Not only linear layers, but also nonlinear layers can be evaluated by any of these methods.

As row-wise packing inputs and outputs n HE-ciphertexts, column-wise packing outperforms row-wise packing in terms of latency. On the other hand, matrix multiplication with column-wise packing requires additional rotations so row-wise packing outperforms column-wise packing in terms of throughput. One can choose an appropriate packing method given its application.

2.2.5 Some Notable Homomorphic Operations

Bootstrapping for CKKS. The bootstrapping procedure for CKKS has been actively studied recently [63, 22, 75, 34]. Let ct be a CKKS-ciphertext of $m(Y) \in \mathbb{Z}[Y]/(Y^\ell + 1)$ with respect to the secret

key sk and the ciphertext modulus q, where $Y = X^{N/\ell}$, namely, $m(Y) = [\langle ct, sk \rangle]_q$. In this case, $m(Y)$ has $\ell/2$ slots. The CKKS bootstrapping aims to find a larger modulus $Q > q$ and a ciphertext ct' such that $m(Y) = [\langle ct', sk \rangle]_Q$. It consists of five steps: ModRaise, SubSum, CoeffToSlot^{CKKS}, EvalMod, and SlotToCoeff^{CKKS}.

- ModRaise: If we set $t(X) = \langle ct, sk \rangle \in \mathcal{R}$, then $t(X) = q \cdot I(X) + m(Y)$ for some $I(X) \in \mathcal{R}$. ModRaise raises the ciphertext modulus to $Q \gg q$ so that ct is regarded as an encryption of $t(X)$ with respect to modulus Q.

- SubSum: If $N \neq \ell$, then SubSum maps $I(X)$ to a polynomial in Y, that is, $q \cdot I(X) + m(Y)$ to $(N/\ell) \cdot (q \cdot \tilde{I}(Y) + m(Y))$.

- CoeffToSlot^{CKKS}: Since the message $q \cdot I(X) + m(Y)$ is in the coefficient domain, it requires homomorphic evaluation of the encoding algorithm to enable slot-wise modulo q operation. CoeffToSlot^{CKKS} performs homomorphic evaluation of the inverse discrete Fourier transform (DFT) to obtain the ciphertext(s) of $\mathsf{Ecd}^{\mathsf{CKKS}}(q \cdot I(X) + m(Y))$.

- EvalMod: To approximate the modulo q operation, EvalMod homomorphically evaluates a polynomial approximation of $f(t) = \frac{q}{2\pi} \sin\left(\frac{2\pi t}{q}\right)$. In recent works [22, 63], Chebyshev polynomial approximations are used.

- SlotToCoeff^{CKKS}: SlotToCoeff^{CKKS} performs homomorphic evaluation of DFT to output a ciphertext of $m(Y)$ back in its coefficient domain.

KAIST

Operations in FV. In the FV scheme, there are two operations between slots and coefficients.

- CoeffToSlot^{FV}: It is a homomorphic evaluation of FV-encoding function. It semantically puts the coefficients of a plaintext polynomial into the vector of slots. It is done by multiplying the inverse number theoretic transform (NTT) matrix.

- SlotToCoeff^{FV}: It is a homomorphic evaluation of FV-decoding function. It semantically puts the slot vector of a message into the coefficients of the plaintext polynomial. It is also done by multiplying the NTT matrix.

2.3 Symmetric Cryptanalysis

2.3.1 Trivial Linearization

Trivial linearization is to make a system of polynomial equations *linear* by replacing all monomials by new variables, and solve it. When the cipher is represented by a system of polynomial equations of degree d over $\mathbb{Z}_q$ in n unknowns (and $d < q$), the number of monomials appearing in this system is upper bounded by

$$S = \sum_{i=0}^{d} \binom{n+i-1}{i}.$$

Therefore, at most S equations will be enough to solve this system of equations. If the system is sparse, then it would require less equations to solve the system.

An adversary might take the *guess and determine* strategy before trivial linearization. By guessing g variables, the number of possible monomials is reduced down to

$$S_g = \sum_{i=0}^{d} \binom{n-g+i-1}{i}.$$

This approach will be useful in particular when almost every monomial appears in the system. In this case, the overall time complexity becomes $O(q^g S_g^\omega)$.

2.3.2 GCD Attack

The GCD (greatest common divisor) attack is one of the major threats to symmetric ciphers over large fields (e.g., MiMC [1]). Let $\mathbb{F}$ be a finite field, and $E : \mathbb{F} \times \mathbb{F} \to \mathbb{F}$ be an encryption function, where $\mathbb{F}$ is the key and the plaintext/ciphertext spaces. Given two plaintext-ciphertext pairs $(x_1, y_1), (x_2, y_2) \in \mathbb{F}_{2^e}$, one can establish univariate polynomial equations $E(K, x_1) = y_1$ and $E(K, x_2) = y_2$ in an unknown K. Then,

$$\gcd(E(K, x_1) - y_1, E(K, x_2) - y_2)$$

includes $(K - k)$ as a factor, where k denotes the actual secret key. It is well-known that the complexity of finding the GCD of two univariate polynomials of degree d is $O(d \log^2 d)$.

2.3.3 Gröbner Basis Attack

KAIST

The Gröbner basis attack is an attack by solving a system of equations by computing a Gröbner basis of the system. If such a Gröbner basis is found, then the variables can be eliminated one by one after carefully converting the order of monomials. We refer to [9] for details. In the literature, security against Gröbner basis attack is bounded by the time complexity for Gröbner basis computing.

Suppose that an attacker wants to solve a system of m polynomial equations in n variables over a field $\mathbb{F}_q$,

$$f_1(x_1, \ldots, x_n) = f_2(x_1, \ldots, x_n) = \cdots = f_m(x_1, \ldots, x_n) = 0.$$

The complexity of computing a Gröbner basis of such system is known to be

$$O\left(\binom{n + d_{reg}}{d_{reg}}^\omega\right)$$

in terms of the number of operations over the base field, where $2 \leq \omega \leq 3$ is the linear algebra constant and d_{reg} is the *degree of regularity* [18]. With the degree of regularity, one can see how much degree of polynomial multiples will be needed to find the Gröbner basis. Unfortunately, it is hard to compute the exact degree of regularity for a generic system of equations. On the other hand, there is the Macaulay bound

$$d_{reg} = 1 + \sum_{i=1}^{m} (d_i - 1)$$

for a *regular sequence* (with $m = n$) where d_i is the degree of f_i [15]. When the number of equations is larger than the number of variables, the degree of regularity of a *semi-regular sequence* can be computed

as the degree of the first non-positive coefficient in the Hilbert series

$$\mathrm{HS}(z) = \frac{1}{(1-z)^n} \times \prod_{i=1}^{m}(1-z^{d_i}).$$

As it is conjectured that most sequences are semi-regular [50], we analyze the security of our ciphers against the Gröbner basis attack under the (semi-)regular assumption.

Hybrid Approach. One can take a hybrid approach between the guess-and-determine attack and the algebraic attack [17]. Guessing some variables makes the system of equations overdetermined. An overdetermined system becomes easier to solve; the complexity of the hybrid approach after g guesses is given as

$$O\left(q^g \binom{n-g+d_g}{d_g}^{\omega}\right)$$

where d_g is the degree of regularity after g guesses.

2.3.4 Interpolation Attack

The interpolation attack is to establish an encryption polynomial in plaintext variables without any information on the secret key and to distinguish it from a random permutation [69].

Suppose that a cipher with a fixed key can be represented as a permutation on $\mathbb{F}_q^n$ (i.e., the plaintext and the ciphertext spaces are $\mathbb{F}_q^n$). Then a single word of the ciphertext can be written as a function, say $E : \mathbb{F}_q^n \to \mathbb{F}_q$. We can also consider its polynomial representation as follows.

KAIST

$$E(\mathbf{X}) = \sum_{\mathbf{e}\in\mathcal{I}} a_{\mathbf{e}}\mathbf{X}^{\mathbf{e}},$$

where $\mathcal{I} \subset \mathbb{Z}_{\geq 0}^n$ denotes the index set, and $\mathbf{X}^{\mathbf{e}} = \prod_{i=1}^n X_i^{e_i}$ for $\mathbf{X} = (X_1, \ldots, X_n) \in \mathbb{F}_q^n$ and $\mathbf{e} = (e_1, \ldots, e_n) \in \mathcal{I}$. Given plaintext-ciphertext pairs $(\mathbf{X}_j, Y_j)_j$ such that $Y_j = E(\mathbf{X}_j)$, this attack seeks to recover all the (key-related) coefficients $a_{\mathbf{e}}$'s i.e., a polynomial equivalent to E. It is known that the data complexity of this attack depends on the number of monomials in the polynomial representation of this cipher.

2.3.5 Linear Cryptanalysis on Non-binary Ciphers

Linear cryptanalysis measures how close a function is from linear [81]. Linear cryptanalysis is typically applied to block ciphers operating on binary spaces. However, linear cryptanalysis can be extended to non-binary spaces [13]; similarly to binary ciphers, for a prime q, the linear probability of a cipher $\mathsf{E} : \mathbb{Z}_q^n \to \mathbb{Z}_q^n$ with respect to input and output masks $\mathbf{a}, \mathbf{b} \in \mathbb{Z}_q^n$ can defined by

$$\mathrm{LP}^{\mathsf{E}}(\mathbf{a}, \mathbf{b}) = \left| \mathbb{E}_{\mathbf{m}} \left[\exp\left\{ \frac{2\pi i}{q} \Big(-\langle \mathbf{a}, \mathbf{m} \rangle + \langle \mathbf{b}, \mathsf{E}(\mathbf{m}) \rangle \Big) \right\} \right] \right|^2,$$

where $\mathbf{m}$ follows the uniform distribution over $\mathbb{Z}_q^n$. When E is a keyed function, the expected linear probability is defined by

$$\mathrm{ELP}^{\mathsf{E}}(\mathbf{a}, \mathbf{b}) = \mathbb{E}_{\mathbf{k}}[\mathrm{LP}^{\mathsf{E}_{\mathbf{k}}}(\mathbf{a}, \mathbf{b})].$$

Then the number of samples required for linear cryptanalysis is known to be

$$\frac{1}{\mathrm{ELP}^{\mathsf{E}}(\mathbf{a},\mathbf{b})}.$$

In order to ensure the security against linear cryptanalysis, it is sufficient to bound the maximum linear probability $\max_{\mathbf{a}\neq\mathbf{0},\mathbf{b}} \mathrm{ELP}^{\mathsf{E}}(\mathbf{a},\mathbf{b})$.

2.3.6 Differential Cryptanalysis

Differential cryptanalysis is an algorithm distinguishing the distribution of input-output differential pairs from uniform random [20]. It usually has been applied to binary ciphers, while it can be naturally generalized to a cipher over an abelian group. Let $\mathsf{E} : G \times G \to G$ be a function where G be an abelian group. Then, differential probability of input-output pair $(a, b) \in G^2$ is defined by

$$\mathrm{DP}^{\mathsf{E}}(a,b) = \Pr\left[x \in G : \mathsf{E}(x+a) - \mathsf{E}(x) = b\right].$$

Similarly to linear cryptanalysis, the expected differential probability is defined by

$$\mathrm{EDP}^{\mathsf{E}}(a,b) = \mathbb{E}_{\mathbf{k}}[\mathrm{DP}^{\mathsf{E}_{\mathbf{k}}}(a,b)].$$

Then the number of chosen samples required for differential cryptanalysis is known to be

$$\frac{1}{\mathrm{EDP}^{\mathsf{E}}(a,b)}.$$

KAIST

In order to ensure the security against differential cryptanalysis, it is sufficient to bound the maximum differential probability $\max_{a\neq 0,b} \mathrm{EDP}^{\mathsf{E}}(a,b)$.

2.4 HE-friendly Ciphers

Since the transciphering framework has been introduced [87], early works have been focused on homomorphic evaluation of popular symmetric ciphers (e.g., AES [55], SIMON [76], and PRINCE [45]). Such ciphers have been designed without any consideration on their arithmetic complexity, so the performance of their homomorphic evaluation was not satisfactory. To reduce the time of homomorphically evaluating ciphers, several dedicated ciphers (which are called HE-friendly ciphers) have been proposed. In this section, we introduce some HE-friendly ciphers proposed earlier.

2.4.1 LowMC

Albrecht et al. proposed the first HE-friendly cipher LowMC [4], which aims to minimize the depth and the number of AND gates. As it was designed not only for homomorphic encryption but also for multi-party computation and zero-knowledge proof, there is a family of parameter sets for each purpose. Nowadays, the LowMC cipher have been used for a component of a digital signature scheme [29].

LowMC is a substitution-permutation network (SPN) based block cipher using random affine layer and partial nonlinear layer of 3-bit S-boxes. The nonlinear layer is chosen to a 3-bit quadratic function

$$S(a,b,c) = (a \oplus bc, a \oplus b \oplus ac, a \oplus b \oplus c \oplus ab)$$

to minimize the depth and the number of ANDs while the differential probability and linear correlation are not so much sacrificed. This S-box only involves 3 AND gates and 1 AND depth. However, since S-box and the inverse of S-box are all quadratic, the LowMC cipher is required to have large AND depth against algebraic attacks [40, 41].

The nonlinear layer is not fully filled with S-boxes, which is called partial nonlinear layer. This partial-SPN structure is adopted to reduce the number of ANDs, while too smaller number of S-boxes may leads to insecurity. As more cryptanalysis on partial-SPN have been proposed [14, 92, 58, 72], partial-SPN becomes an unpopular construction in HE applications since the multiplicative depth is more critical to efficiency than the multiplicative complexity.

The affine layer of LowMC consists of multiplication by random matrices and random round constant addition. The affine layers are once chosen in specification, then they are fixed. Denoting the block size n, the random matrices are chosen uniformly at random from all the $n \times n$ invertible matrices, and the round constants are chosen uniformly at random from all the binary vectors of length n when the cipher is instantiated. Random affine layers complement the security of 3-bit S-box against statistical attacks at the cost of heavy linear algebra complexity. This part is a bottleneck of the client-side computation.

As the key length k and the block size n are usually different, the LowMC cipher uses an expanding key schedule. The key schedule is multiplication by key schedule matrices which are chosen uniformly at random from all the $n \times k$ full-rank matrices. In order to reduce the number of ANDs, the key schedule does not involve any multiplication between key-related materials.

2.4.2 Kreyvium

KAIST

Canteaut et al. [27] claimed that stream ciphers would be advantageous in terms of online complexity compared to block ciphers, and proposed a new stream cipher Kreyvium. When a block cipher is used for transciphering purpose, the server can homomorphically compute decryption circuit of the block cipher after receiving symmetric ciphertexts. On the other hand, if a stream cipher is used, homomorphic operations computing keystream can be done before receiving symmetric ciphertexts. In Figure 2.1, we give a pictorial description of transciphering framework when an IV-based stream cipher is used.

The authors claimed to use Trivium [39] for transciphering purpose since it has a small number of nonlinear operations. Although Trivium is a synchronous stream cipher, it should be computed as IV-based bounded-length keystream generator since the multiplicative depth (with respect to the secret key) keeps growing as keystream is generated. For a multiplicative depth $d \geq 4$, the output keystream length $n(d)$ of Trivium is

$$n(d) = -1152 + 282 \times \left\lfloor \frac{d}{3} \right\rfloor + \begin{cases} 81 & \text{if } d \equiv 0 \mod 3 \\ 160 & \text{if } d \equiv 1 \mod 3 \\ 269 & \text{if } d \equiv 2 \mod 3 \end{cases}.$$

As Trivium cannot become 128-bit secure by increasing the size of key and IV without additional modification [83], the authors proposed a new 128-bit secure stream cipher Kreyvium which is made by adding two more register to Trivium. The detailed algorithm is in Algorithm 1. For a multiplicative

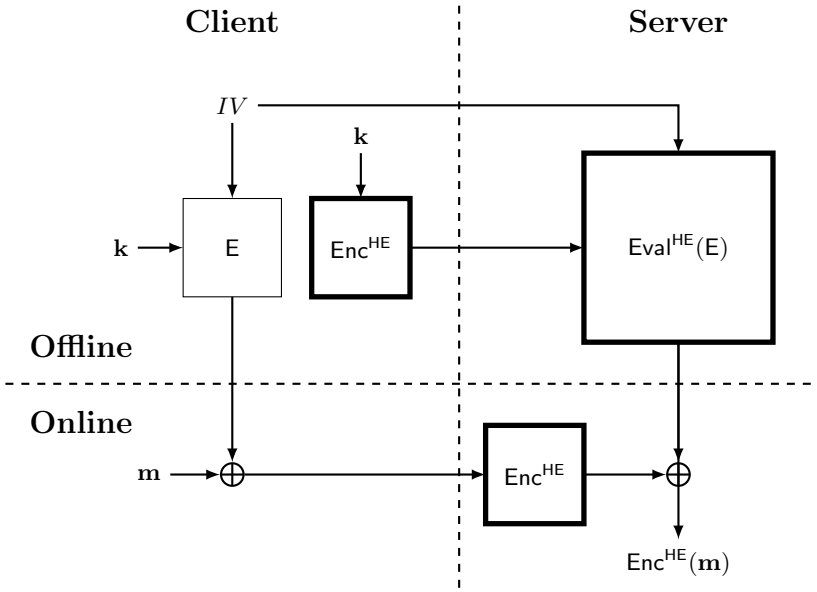


Figure 2.1: Transciphering framework when a stream cipher is used. The keys are omitted when the usage is clear.

depth $d \geq 4$, the output keystream length $n(d)$ of Kreyvium is

$$n(d) = -1152 + 282 \times \left\lfloor \frac{d}{3} \right\rfloor + \begin{cases} 70 & \text{if } d \equiv 0 \mod 3 \\ 149 & \text{if } d \equiv 1 \mod 3 \\ 258 & \text{if } d \equiv 2 \mod 3 \end{cases}.$$

KAIST

There have been some attacks on reduced-round Kreyvium: cube attacks [73, 98, 99, 68], and conditional differential cryptanalysis [100]. The best known key recovery attack is mounted to 894-round Kreyvium and the best known distinguishing attack is mounted to 899-round Kreyvium out of total 1152 rounds.

2.4.3 FLIP and FiLIP

Méaux et al. proposed a new stream cipher construction, dubbed filter permutator, and specify FLIP which is a family of filter permutator [85]. The filter permutator consists of key register, bit permutation, and filtering function. We can express the keystream z_{IV} by a formula

$$z_{IV} = \mathsf{FF} \circ \mathsf{BP}_{IV}(\mathbf{k})$$

where IV is an initial vector, FF is the filtering function, BP_{IV} is a bit permutation generated by IV, and $\mathbf{k}$ is the key.

The key register is where the key is stored. As FLIP is not an LFSR-based stream cipher, the key register will not be updated. The bit permutation is randomly generated by a permutation generator (e.g., PRNG initialized by IV) so that the bit permutation is expected to be different for every IV. Algebraic attacks in the chosen-plaintext attack (CPA) model (e.g., higher order differential attack, cube attack, and interpolation attack) cannot be applied to FLIP since the polynomial representation is randomized for distinct IVs. It implies that a cipher is not required to achieve the degree of $\lambda - 1$ for

Algorithm 1: Kreyvium

Input:

- Key $(K_0, \ldots, K_{127})$
- Initial vector $(IV_0, \ldots, IV_{127})$
- Keystream length n

Output:

- Keystream z_i

$(s_1, \ldots, s_{93}) \leftarrow (K_0, \ldots, K_{92})$
$(s_{94}, \ldots, s_{177}) \leftarrow (IV_0, \ldots, IV_{83})$
$(s_{178}, \ldots, s_{288}) \leftarrow (IV_{84}, \ldots, IV_{127}, 1, \ldots, 1, 0)$
$(K^*_{127}, K^*_{126}, \ldots, K^*_0) \leftarrow (K_0, \ldots, K_{127})$
$(IV^*_{127}, IV^*_{126}, \ldots, IV^*_0) \leftarrow (IV_0, \ldots, IV_{127})$
for $i \leftarrow 1$ *to* $1152 + n$ **do**
  $t_1 \leftarrow s_{66} + s_{93}$
  $t_2 \leftarrow s_{162} + s_{177}$
  $t_3 \leftarrow s_{243} + s_{288} + K^*_0$
  if $i > 1152$ **then**
    output $z_{i-1152} \leftarrow t_1 + t_2 + t_3$
  $t_1 \leftarrow t_1 + s_{91} \cdot s_{92} + s_{171} + IV^*_0$
  $t_2 \leftarrow t_2 + s_{175} \cdot s_{176} + s_{264}$
  $t_3 \leftarrow t_3 + s_{286} \cdot s_{287} + s_{69}$
  $t_4 \leftarrow K^*_0$
  $t_3 \leftarrow IV^*_0$
  $(s_1, \ldots, s_{93}) \leftarrow (t_3, s_1, \ldots, s_{92})$
  $(s_{94}, \ldots, s_{177}) \leftarrow (t_1, s_{94}, \ldots, s_{176})$
  $(s_{178}, \ldots, s_{288}) \leftarrow (t_2, s_{178}, \ldots, s_{287})$
  $(K^*_{127}, K^*_{126}, \ldots, K^*_0) \leftarrow (t_4, K^*_{127}, \ldots, K_1)$
  $(IV^*_{127}, IV^*_{126}, \ldots, IV^*_0) \leftarrow (t_5, IV^*_{127}, \ldots, IV_1)$

KAIST

λ-bit security.

The filtering function is a fixed low-degree polynomial. Specifically, it is sum of linear functions L_n, quadratic functions Q_n, and triangular functions T_k where they are defined by

$$L_n(x_0, \ldots, x_{n-1}) = \sum_{i=0}^{n-1} x_i$$
$$Q_n(x_0, \ldots, x_{2n-1}) = \sum_{i=0}^{n-1} x_{2i}x_{2i+1}$$
$$T_k(x_0, \ldots, x_{k(k+1)/2-1}) = \sum_{i=0}^{k} \left(\prod_{j=0}^{i-1} x_{j+i(i-1)/2} \right).$$

The firstly proposed parameters contain only one triangular function in the filtering function. It leads to a guess-and-determine attack [47], which cancels high-degree monomials by guessing it. Now revised version of FLIP contains multiple triangular functions.

Méaux et al. proposed improved filter permutators, called FiLIP [84]. It consists of two more

components: subset selection, and whitening. We can express the keystream z_{IV} by a formula

$$z_{IV} = \mathsf{FF}((\mathsf{SS}_{IV} \circ \mathsf{BP}_{IV})(\mathbf{k}) \oplus \mathsf{wtn}_{IV})$$

where SS_{IV} is a subset selection generated by IV, and wtn_{IV} is the whitening vector generated by IV.

The subset selection is a random subset of the key register determined by a PRNG initialized by IV. By adding the subset selection, the authors can enlarge the key register, which increases the security without significant increase of homomorphic computation. However, the enlargement of key register is equivalent to the large key size, where the key size of FiLIP reaches 0.5 to 2 KB for 128-bit security. The whitening is addition by a random binary vector (which is also generated by PRNG with IV).

As FiLIP requires the small number of (both linear and nonlinear) operations, it has been reported that FiLIP can be efficiently evaluated with the TFHE scheme [66].

2.4.4 Rasta and Its Variants

Rasta. Rasta [44] is a stream cipher aiming at higher throughput at the cost of high latency by minimizing both AND depth and ANDs per bit. Rasta is an ASASA-like construction which consists of affine layers, nonlinear layers, and key additions. We can express a keystream $z_{\mathsf{nc},i}$ of Rasta by a formula

$$z_{\mathsf{nc},i} = (A_{r,\mathsf{nc},i} \circ S \circ A_{r-1,\mathsf{nc},i} \circ \cdots \circ S \circ A_{0,\mathsf{nc},i})(\mathbf{k}) \oplus \mathbf{k}$$

where $A_{j,\mathsf{nc},i}$ are affine layers generated by a nonce nc, S is the nonlinear layer, and $\mathbf{k}$ is the key.

KAIST

The affine layers is composed of multiplication by random invertible matrices and addition by random vectors. The random matrices and the random vectors are generated by extendable-output function (XOF) seeded with nonce and block counter i. Similarly to FLIP, distinct permutations for every encryption block prevent many algebraic attacks. However, it turns out that the multiplication by random matrices cannot be efficiently computed in HE setting. Furthermore, it is too slow on the client side since too much random bits should be generated to evaluate a single block.

The nonlinear layer is the χ-transformation [38]. For $\mathbf{x} = (x_1, \ldots, x_n) \in \mathbb{Z}_2^n$, the i-th component of $\chi(\mathbf{x})$, denoted $\chi(\mathbf{x})_i$, is defined by

$$\chi(\mathbf{x})_i = x_i \oplus x_{i+2} \oplus x_{i+1}x_{i+2}$$

taking the indices modulo n. Since the χ-transformation features high-degree inverse function, it prevents algebraic meet-in-the-middle attack (i.e., solving equations which are set up at a middle state). Because of it, Rasta achieves the significantly small number of rounds. Recently, Liu et al. exploited the sparsity of the χ-transformation and mounted the algebraic attack to 3-round Rasta and full round instances of Agrasta (tight parameter version of Rasta) [78].

Dasta. Hebborn and Leander proposed a variant Dasta of Rasta [64]. The main difference from Rasta is that affine layers do not rely on XOF. Instead, affine layers of Dasta consist of bit permutations and fixed linear transformations. The round function of Dasta is

$$L \circ P_{r,i} \circ \chi$$

where L is the fixed linear transformation and $P_{r,i}$ is the bit permutation determined by round counter

r and block counter i. Given a fixed bit permutation π and a simple function α of the round counter and the block counter, the bit permutation is of the form

$$P_{r,i} = \pi^{\alpha(r,i)}.$$

Since Dasta does not rely on XOF, it achieves hundreds of times faster client-side throughput compared to Rasta. Because of this structured affine layers, the recent attack on reduced-round Rasta broke an instance of full-round Dasta [78]. Fortunately, this attack can be prevented by swapping the last $P_{r,i}$ and L.

Masta. Masta is another variant of Rasta operating on a modular domain [61]. It is well-known that the M-th cyclotomic polynomial $\Phi_M(X)$ has $\phi(M)$ distinct zeros over $\mathbb{Z}_p$ if $p \equiv 1 \mod M$. Previous ciphers have been designed over $\mathbb{Z}_2$, and the batching technique cannot be fully utilized when the ciphers are homomorphically computed. On the other hand, since Masta is operating over $\mathbb{Z}_p$, a proper choice of p allows full utilization of the batching technique. It leads to a drastic improvement on the server-side efficiency.

Another feature of Masta is to reduce the amount of required pseudorandom bits per encryption. Generating random affine layers of r-round Rasta requires $O(rn^2)$ bits for block size n. In Masta, the random matrices are replaced by multiplication by random elements in $\mathbb{F}_{p^n}$. It reduces the required pseudorandom bits to $O(rn \log p)$ bits. As evaluation of XOF is the bottleneck of computing Rasta on the client side, the client-side efficiency of Masta is significantly better than Rasta.

Pasta. Dobraunig et al. [43] formally defined hybrid homomorphic encryption and proposed an HE-friendly cipher Pasta operating on a modular domain, which is inspired by Rasta. Unlike the similarity between their names, components of Pasta is quite different from those of Rasta. The main differences are truncation, two kinds of S-box layers, and cheaper linear layers.

Pasta is also an ASASA-like structure. We can express the i-th keystream block $z_{\mathsf{nc},i}$ of Pasta by a formula

$$z_{\mathsf{nc},i} = \mathsf{Tr}_{n,t} \circ A_{r,\mathsf{nc},i} \circ S_{\mathsf{Cube}} \circ A_{r-1,\mathsf{nc},i} \circ S_{\mathsf{Feistel}} \circ A_{r-2,\mathsf{nc},i} \circ \cdots \circ S_{\mathsf{Feistel}} \circ A_{0,\mathsf{nc},i}(\mathbf{k})$$

where

- $\mathbf{k}$ is the key;
- r is the number of rounds;
- n is the block size in terms of the number of words in $\mathbb{Z}_p$;
- t is the output size satisfying $n = 2t$;
- nc is the nonce;
- for $\mathbf{x} = (x_1, x_2, \ldots, x_n) \in \mathbb{Z}_p^n$,

$$S_{\mathsf{Feistel}}(\mathbf{x})_i = \begin{cases} x_1 & \text{if } i = 1 \\ x_i + x_{i-1}^2 & \text{if } i \geq 2 \end{cases},$$
$$S_{\mathsf{Cube}}(\mathbf{x}) = (x_1^3, \ldots, x_n^3),$$
$$\mathsf{Tr}_{n,t}(\mathbf{x}) = (x_1, \ldots, x_t);$$

KAIST

- for $\mathbf{x} = \mathbf{x}_L \| \mathbf{x}_R \in \mathbb{Z}_p^t \times \mathbb{Z}_p^t$ and $k \in \{L, R\}$,

$$A_{j,\mathsf{nc},i}(\mathbf{x}) = \begin{bmatrix} 2 \cdot I & I \\ I & 2 \cdot I \end{bmatrix} \cdot \begin{bmatrix} A_{j,\mathsf{nc},i,L}(\mathbf{x}_L) \\ A_{j,\mathsf{nc},i,R}(\mathbf{x}_R) \end{bmatrix},$$

$$A_{j,\mathsf{nc},i,k}(\mathbf{x}_k) = (\mathbf{M}_{j,\mathsf{nc},i,k})^t \mathbf{x}_k + \mathbf{c}_{j,\mathsf{nc},i,k},$$

$$\mathbf{M}_{j,\mathsf{nc},i,k} = \begin{bmatrix} 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 1 & \dots & 0 \\ & \vdots & & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 1 \\ \alpha_1 & \alpha_2 & \alpha_3 & \dots & \alpha_t \end{bmatrix}$$

  where $\alpha_i \in \mathbb{Z}_p^\times$ and $\mathbf{c}_{j,\mathsf{nc},i,k} \in \mathbb{Z}_p^t$ are randomly generated by XOF.

The S-box layer of Pasta is inherently vulnerable to algebraic meet-in-the-middle attack since it has quadratic inverse. However, the last truncation prevent these kinds of attacks which was used in [42] for the same purpose. So Pasta can have both small multiplicative depth and invertible S-box layers. The linear layers of Pasta has PHOTON-like [60] matrix multiplication. It requires $O(rn \log p)$ random bits similarly to Masta, but requires less additions and multiplications compared to the linear layers of Masta.

KAIST

Chapter 3. Transciphering of Real Numbers

3.1 RtF Transciphering Framework

Real-world data typically contain some errors from their true values since they are represented by real numbers rather than bits or integers. Even in the case that input data are represented by exact numbers without approximation, one might have to approximate intermediate values during data processing for efficiency. Therefore, it would be practically relevant to support approximate computation over encrypted data. The CKKS encryption scheme [35] provides the desirable feature using an efficient encoder for real numbers. Due to this feature, CKKS achieves good performance in various applications, for example, to securely evaluate machine learning algorithms on a real dataset [23, 89].

The transciphering framework in Figure 1.1 does not directly apply to the CKKS scheme. The main reason is that it is infeasible to design an HE-friendly (deterministic) symmetric cipher E operating on real (or complex) numbers; if an HE-friendly symmetric cipher E over the real field exists, then E is given as a real polynomial map, and any ciphertext will be represented by a polynomial in the corresponding plaintext and the secret key over $\mathbb{R}$. Then, for given plaintext-ciphertext pairs $(\mathbf{m}_i, \mathbf{c}_i)$, an adversary will be able to establish a system of polynomial equations in the unknown key $\mathbf{k}$. The sum of $\|\mathsf{E}_{\mathbf{k}}(\mathbf{m}_i) - \mathbf{c}_i\|_2^2$ over the plaintext-ciphertext pairs also becomes a real polynomial, where the actual key is the zero of this function. Since this polynomial is differentiable, its (approximate) zeros will be efficiently found by using iterative algorithms such as the gradient descent algorithm. By taking multiple plaintext-ciphertext pairs, the probability of finding any false key will be negligible.

KAIST

In order to overcome this problem, we combine CKKS with FV which is a homomorphic encryption scheme using modular arithmetic [49], obtaining a novel hybrid framework, dubbed the RtF (Real-to-Finite-field) transciphering framework. This framework inherits a wide range of usability from the previous transciphering framework, such as efficient short message encryption or flexible repacking of data on the server side. Additionally, our framework does not require to use the complex domain for message spaces (as in the CKKS scheme), or any expertise of the CKKS parameter setting on the client side.

3.1.1 Overview of the Framework

Our RtF transciphering framework aims to replace the (basic) transciphering framework in Fig. 1.1 to support CKKS, when equipped with any suitable stream cipher. The overall design is depicted in Fig. 3.1. At a high level, we propose to use a stream cipher operating on $\mathbb{Z}_q^n$ to encrypt real number messages on the client side and to convert the ciphertexts into the corresponding CKKS ciphertexts on the server side. In this regard, it is required to employ an additional HE scheme which provides homomorphic evaluation of keystreams of the stream cipher over the modulo q spaces efficiently, and we use FV for this purpose.

The main idea of the RtF framework is to inject real messages into the coefficients of plaintext polynomials of FV and to delegate encoding/decoding to the server via SlotToCoeff and CoeffToSlot for FV and CKKS which is described more precisely as follows.

First, a message of real numbers $\mathbf{m}_{\mathsf{ctr}} \in \mathbb{R}^n$ is scaled into $\mathbb{Z}_q^n$ by multiplying by a constant and rounding, and encrypted to $\mathbf{c}_{\mathsf{ctr}}$ on the client side. After gathering symmetric ciphertexts $\mathbf{c}_{\mathsf{ctr}}$'s from the

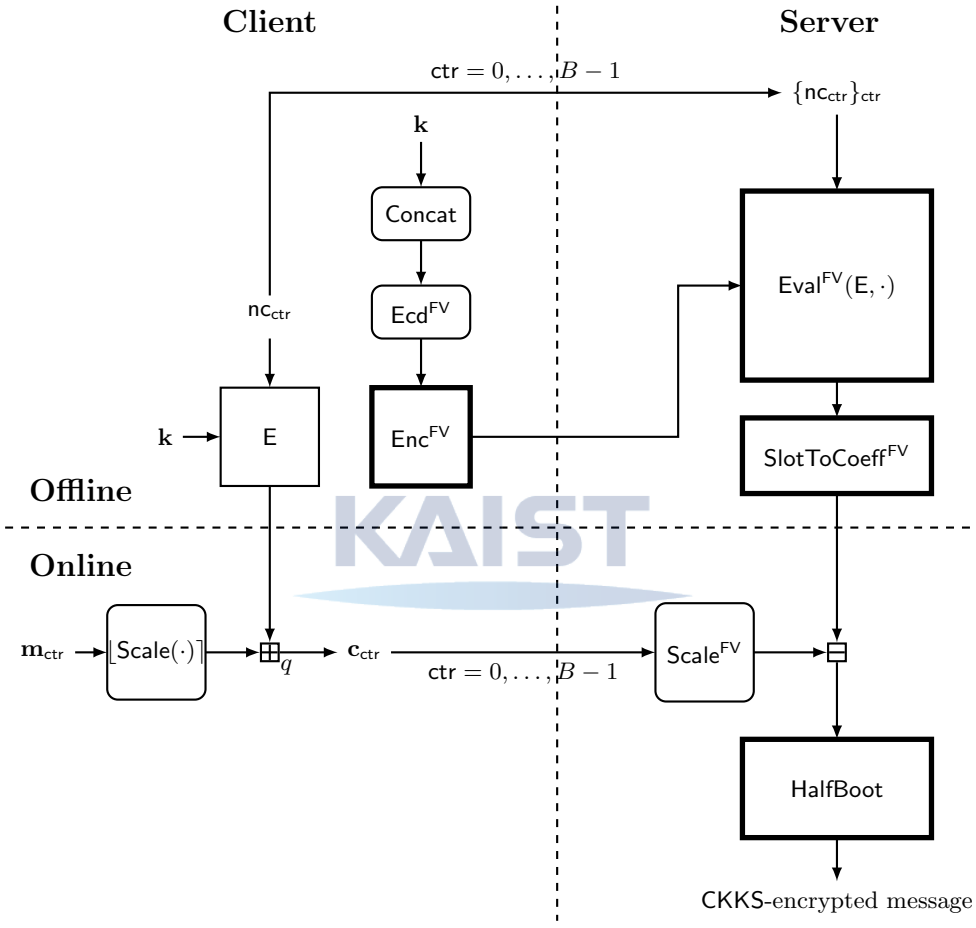


Figure 3.1: The RtF transciphering framework. Homomorphic encryption and evaluation is performed in the boxes with thick lines. Operations in the boxes with rounded corners do not use any secret information. The vertical dashed line distinguishes the client-side and the server-side computation, while the horizontal dashed line distinguishes the offline and the online computation. The client sends ciphertexts block by block, while the server gathers B ciphertext blocks and recovers the CKKS-encrypted message of the ciphertexts.

client, the server generates a polynomial $C \in \mathcal{R}_q$ whose coefficients are components of $\mathbf{c}_{\mathsf{ctr}}$'s. Then the polynomial is scaled up into the FV ciphertext space by multiplying Δ_{FV}, say $\mathcal{C} = (\Delta_{\mathsf{FV}} \cdot C, 0)$.[1] On the other hand, when the server evaluates the symmetric cipher, a bunch of the keystream is FV-encrypted in slots. In order to match the domain of computation, the server evaluates

$$\mathsf{SlotToCoeff}^{\mathsf{FV}} : \mathsf{Enc}^{\mathsf{FV}}(\mathsf{Ecd}^{\mathsf{FV}}(z_0, \ldots, z_{N-1})) \mapsto \mathsf{Enc}^{\mathsf{FV}}(z_0 + \cdots + z_{N-1}X^{N-1})$$

after evaluation of the cipher, where $(z_0, \ldots, z_{N-1})$ is the concatenated keystream. Then, homomorphically computing

$$(\Delta_{\mathsf{FV}} \cdot C, 0) - \mathsf{Enc}^{\mathsf{FV}}(z_0 + \cdots + z_{N-1}X^{N-1}),$$

we have $\mathsf{Enc}^{\mathsf{FV}}(m_0 + \cdots + m_{N-1}X^{N-1})$, where $(m_0, \ldots, m_{N-1})$ is the concatenated message. The next step is to convert the type of encryption to CKKS and then to put the messages into slots, which can be done by HalfBoot.

In the bootstrapping procedure, there are five steps as follows:

$$\mathsf{ModRaise} \to \mathsf{SubSum} \to \mathsf{CoeffToSlot}^{\mathsf{CKKS}} \to \mathsf{EvalMod} \to \mathsf{SlotToCoeff}^{\mathsf{CKKS}}.$$

HalfBoot basically follows the procedure of CKKS bootstrapping, except the final $\mathsf{SlotToCoeff}^{\mathsf{CKKS}}$ step. Since the input ciphertext of HalfBoot contains the original message $(m_0, \ldots, m_{N-1})$ in coefficients rather than slots, it does not require to move data in slots back to coefficients after EvalMod. Furthermore, with an appropriate rescaling, HalfBoot gives an effect of full bootstrapping to enable further approximate computations on the output CKKS ciphertexts.

KAIST

3.1.2 Specification

For a fixed security parameter λ, all the other parameters for the FV and the CKKS schemes will be set accordingly, including the degree of the polynomial modulus N, the ciphertext moduli $\{Q_i\}_{i=0}^{L}$ (used for both FV and CKKS), and the FV plaintext modulus q. With these parameters fixed, we will describe how the framework works, distinguishing four parts; initialization, client-side computation, and offline/online server-side computation (see Figure 3.1). The client-side and server-side computations are described in Algorithm 2 and Algorithm 3, respectively.

Initialization. We use FV and CKKS with the same cyclotomic polynomial of degree N, and the same public-private key pair (pk, sk). The public key pk is shared by the server and the client. Let ℓ be the number of used slots per FV-ciphertext to encrypt $\mathbf{k} \in \mathbb{Z}_q^n$ which satisfies $n \,|\, \ell$ and $\ell \,|\, N$. To enable SIMD evaluation for keystreams, we consider the following matrix of B duplications of $\mathbf{k}$.

$$\mathsf{Concat}(\mathbf{k}) \coloneqq \underbrace{(\mathbf{k} \| \mathbf{k} \| \cdots \| \mathbf{k})}_{B\text{-times}} \in \mathbb{Z}_q^{n \times B}.$$

The client can pack the coefficients of matrix $\mathsf{Concat}(\mathbf{k})$ column-wisely into one glued column vector in $\mathbb{Z}_q^{nB}$ or row-by-row manner, which are called column-wise and row-wise packing, respectively. The number of keystreams calculated in a single ciphertext (resp. n ciphertexts) is $B = \ell/n$ for column-wise packing (resp. $B = \ell$ for row-wise packing). We refer to Section 2.2.4 for more details.

[1] We note that $\mathbf{c}_{\mathsf{ctr}}$'s are in coefficients, not in slots.

Algorithm 2: Client-side symmetric key encryption of the RtF transciphering framework

Input:

- Nonce $\mathsf{nc}_{\mathsf{ctr}} \in \{0,1\}^{\lambda}$
- Symmetric key $\mathbf{k} \in \mathbb{Z}_q^n$
- Tuple of messages $\mathbf{m}_{\mathsf{ctr}} \in \mathbb{R}^n$
- Scaling factor δ

Output:

- Symmetric ciphertext $\mathbf{c}_{\mathsf{ctr}} \in \mathbb{Z}_q^n$

$\mathbf{z}_{\mathsf{ctr}} \leftarrow \mathsf{E}(\mathbf{k}_{\mathsf{ctr}}, \mathsf{nc}_{\mathsf{ctr}})$
$\widetilde{\mathbf{m}}_{\mathsf{ctr}} \leftarrow \lfloor \delta \cdot \mathbf{m}_{\mathsf{ctr}} \rceil$
$\mathbf{c}_{\mathsf{ctr}} \leftarrow [\widetilde{\mathbf{m}}_{\mathsf{ctr}} + \mathbf{z}_{\mathsf{ctr}}]_q$
return $\mathbf{c}_{\mathsf{ctr}}$

KAIST

Algorithm 3: Server-side homomorphic evaluation of decryption of the RtF transciphering framework

Input:

- Set of nonces $\mathsf{nc}_0, \ldots, \mathsf{nc}_{B-1} \in \{0,1\}^{\lambda}$
- Homomorphically encrypted keys $\mathcal{K} = \mathsf{Enc}^{\mathsf{FV}}\left(\mathsf{Ecd}^{\mathsf{FV}}(\mathsf{Concat}(\mathbf{k}))\right)$
- Tuple of symmetric ciphertexts $\mathbf{c} = (\mathbf{c}_0, \ldots, \mathbf{c}_{B-1}) \in (\mathbb{Z}_q^n)^B$

Output:

- CKKS-encrypted message $\mathcal{M}$

$\mathcal{V} \leftarrow \mathsf{Eval}^{\mathsf{FV}}(\mathsf{E}, \mathcal{K}, \{\mathsf{nc}_{\mathsf{ctr}}\}_{\mathsf{ctr}})$
$\mathcal{Z} \leftarrow \mathsf{SlotToCoeff}^{\mathsf{FV}}(\mathcal{V})$
$C \leftarrow \mathsf{VecToPoly}(\mathbf{c})$
$\mathcal{C} \leftarrow (\Delta_{\mathsf{FV}} \cdot C, 0)$
$\mathcal{X} \leftarrow [\mathcal{C} - \mathcal{Z}]_Q$
$\mathcal{X} \leftarrow \mathsf{Rescale}_{\to 0}(\mathcal{X})$ `// Rescale to the lowest level`
$\mathcal{M} \leftarrow \mathsf{HalfBoot}(\mathcal{X})$
return $\mathcal{M}$

To summarize, the client computes

$$\mathcal{K} \coloneqq \mathsf{Enc}^{\mathsf{FV}}(pk, \mathsf{Ecd}^{\mathsf{FV}}(\mathsf{Concat}(\mathbf{k}))),$$

and sends $\mathcal{K}$ to the server. We note that this initialization phase can be done only once at the beginning of the RtF framework. The client also generates a random value $\mathsf{nc} \in \{0,1\}^\lambda$ and sends it to the server.

Client-side Computation. Given a nonce $\mathsf{nc} \in \{0,1\}^\lambda$, a secret key $\mathbf{k} \in \mathbb{Z}_q^n$ of E, an n-tuple of real messages $\mathbf{m} = (m_0, \ldots, m_{n-1}) \in \mathbb{R}^n$, and a scaling factor $\delta > 0$, the client executes the following encryption algorithm as described in Algorithm 2.

The client computes keystream $\mathbf{z} = \mathsf{E}_{\mathbf{k}}(\mathsf{nc}) \in \mathbb{Z}_q^n$. Then, the client scales the message $\mathbf{m}$ by multiplying δ to every component of $\mathbf{m}$. Rounding it off gives a vector $\widetilde{\mathbf{m}} \in \mathbb{Z}^n$. If q and δ are appropriately chosen, the norm $\|\widetilde{\mathbf{m}}\|_\infty$ can be upper bounded by $q/2$. Finally, the client computes

$$\mathbf{c} \coloneqq [\widetilde{\mathbf{m}} + \mathbf{z}]_q\,,$$

and sends it to the server.

Offline Server-side Computation. Given a tuple of nonces $(\mathsf{nc}_0, \ldots, \mathsf{nc}_{B-1})$ and the FV-encrypted key $\mathcal{K}$, the server is able to construct a circuit for the homomorphic evaluation of E, denoted by $\mathsf{Eval}^{\mathsf{FV}}(\mathsf{E}, \{\mathsf{nc}_{\mathsf{ctr}}\}_{\mathsf{ctr}}, \cdot)$. The circuit constructed for column-wise (resp. row-wise) packing method returns 1 ciphertext (resp. n ciphertexts) which packs ℓ/n keystreams (resp. ℓ keystreams). With the FV-encrypted key $\mathcal{K}$, the server homomorphically computes $\mathcal{V} \coloneqq \mathsf{Eval}^{\mathsf{FV}}(\mathsf{E}, \{\mathsf{nc}_{\mathsf{ctr}}\}_{\mathsf{ctr}}, \mathcal{K})$. For ease of notation, we explain the remaining parts with column-wise packing method. Denoting the concatenation of ℓ/n keystreams by $(z_0, \ldots, z_{\ell-1}) \in \mathbb{Z}_q^\ell$, the resulting FV-ciphertext $\mathcal{V}$ can be represented as

$$\mathsf{Enc}^{\mathsf{FV}}\left(\mathsf{Ecd}_\ell^{\mathsf{FV}}(z_0, \ldots, z_{\ell-1})\right).$$

Finally, the server computes

$$\mathcal{Z} \coloneqq \mathsf{SlotToCoeff}^{\mathsf{FV}}(\mathcal{V}) = \mathsf{Enc}^{\mathsf{FV}}\left(\sum_{k=0}^{\ell-1} z_k X^{k \cdot N/\ell}\right).$$

Online Server-side Computation. Given a tuple of symmetric ciphertexts $\mathbf{c} = (\mathbf{c}_0, \ldots, \mathbf{c}_{\ell/n-1}) \in (\mathbb{Z}_q^n)^{\ell/n}$, the server scales up $\mathbf{c}$ into FV-ciphertext space to enable FV evaluation, namely

$$C \coloneqq \mathsf{VecToPoly}(\mathbf{c}),$$
$$\mathcal{C} \coloneqq (\Delta_{\mathsf{FV}} \cdot C, 0),$$

where VecToPoly is defined by

$$\begin{aligned} \mathsf{VecToPoly}: \qquad \mathbb{R}^\ell &\longrightarrow \mathbb{R}[X]/(\Phi_{2N}(X)) \\ (m_0, \ldots, m_{\ell-1}) &\longmapsto \sum_{k=0}^{\ell-1} m_k X^{k \cdot N/\ell}. \end{aligned}$$

Then, server computes $\mathcal{X} \coloneqq [\mathcal{C} - \mathcal{Z}]_Q$, where Q is the ciphertext modulus of $\mathcal{Z}$, and rescale it to the lowest level of CKKS.

KAIST

Now, the only remaining procedure is HalfBoot, which combines ModRaise, SubSum, CoeffToSlot^{CKKS}, and EvalMod sequentially. Denoting the scaled message by $(\widetilde{\mathbf{m}}_0, \ldots, \widetilde{\mathbf{m}}_{\ell/n-1}) := (\widetilde{m}_0, \ldots, \widetilde{m}_{\ell-1}) \in \mathbb{Z}^\ell$, the resulting ciphertext can be represented as

$$\mathcal{X} := \mathsf{Enc}^{\mathsf{CKKS}}\left(\sum_{k=0}^{\ell-1} \widetilde{m}_k X^{k\cdot N/\ell}\right).$$

Then, after ModRaise, we have

$$\mathcal{X}' := \mathsf{Enc}^{\mathsf{CKKS}}\left(\sum_{k=0}^{\ell-1} \widetilde{m}_k X^{k\cdot N/\ell} + Q_0 \cdot I(X)\right)$$

for some polynomial $I(X) = r_0 + \cdots + r_{N-1}X^{N-1} \in \mathcal{R}$. By evaluating SubSum, the polynomial $I(X)$ becomes sparsely packed

$$\widetilde{I}(X) = \frac{N}{\ell}\sum_{k=0}^{\ell-1} r_{k\cdot N/\ell} X^{k\cdot N/\ell}$$

and the message is scaled by N/ℓ, say $\widetilde{m}_k \leftarrow (N/\ell)\cdot\widetilde{m}_k$. Evaluating CoeffToSlot^{CKKS} gives two ciphertexts as follows.

$$\mathcal{Y}_0 = \mathsf{Enc}^{\mathsf{CKKS}}\left(\mathsf{Ecd}^{\mathsf{CKKS}}_{\ell/2}(\widetilde{m}_0 + Q_0\widetilde{r}_0, \ldots, \widetilde{m}_{\ell/2-1} + Q_0\widetilde{r}_{\ell/2-1})\right)$$
$$\mathcal{Y}_1 = \mathsf{Enc}^{\mathsf{CKKS}}\left(\mathsf{Ecd}^{\mathsf{CKKS}}_{\ell/2}(\widetilde{m}_{\ell/2} + Q_0\widetilde{r}_{\ell/2}, \ldots, \widetilde{m}_{\ell-1} + Q_0\widetilde{r}_{\ell-1})\right)$$

where $\widetilde{r}_k = (N/\ell)\cdot r_{k\cdot N/\ell}$ for $k = 0, 1, \ldots, \ell-1$. If $\ell \neq N$, then those two ciphertexts can be packed in a ciphertext. As EvalMod evaluates the modulo-Q_0 operation approximately, EvalMod operation results in what we want.

KAIST

3.1.3 Message Precision

As the CKKS scheme adds some noise for every arithmetic operation, it is important to analyze how close the output $\mathcal{M}$ of Algorithm 3 is to the original message. In this section, we bound the error occurred in the RtF framework. First, we bound the error in the middle state $\mathcal{X}$ in Algorithm 3.

Let $\mathbf{m} \in \mathbb{R}^\ell$ be an (ℓ/n)-concatenation of the client's message as an input to Algorithm 2 such that $\widetilde{\mathbf{m}} = \lfloor \delta \cdot \mathbf{m} \rceil$ and $\|\widetilde{\mathbf{m}}\|_\infty \leq \lfloor q/2 \rfloor$, and let $\mathcal{X}$ be the state in line 5 of Algorithm 3 before rescaling to zero level and HalfBoot in Algorithm 3. If $e_{\mathsf{eval}} \in \mathcal{R}$ is an error from homomorphic evaluation of E with FV such that $\|e_{\mathsf{eval}}\|_\infty < \Delta_{\mathsf{FV}}/2$ (i.e., the ciphertext is correctly FV-decryptable), then we have

$$\begin{aligned}\left\|\mathsf{VecToPoly}(\mathbf{m}) - \frac{\mathsf{Dec}^{\mathsf{CKKS}}(\mathcal{X})}{\delta\Delta_{\mathsf{FV}}}\right\|_\infty &\leq \frac{1}{2\delta} + \frac{\|e_{\mathsf{eval}}\|_\infty}{\Delta_{\mathsf{FV}}\delta} \\ &\leq \frac{1}{2\delta} + \frac{1}{2\delta} = \frac{1}{\delta}\end{aligned}$$

since $\|\mathbf{m} - \widetilde{\mathbf{m}}/\delta\|_\infty \leq \frac{1}{2\delta}$ and $[\widetilde{\mathbf{m}}]_t = \widetilde{\mathbf{m}}$. We remark that e_{eval} depends on the construction of E, which will be bounded appropriately for our new stream cipher and proposed parameters for HE.

The change of the message precision in HalfBoot varies according to which specific algorithm is used. We basically follow the work of Bossuat et al. [22] of CKKS bootstrapping, and we describe the message precision using those results.

In the bootstrapping procedure, the most significant step is to approximate modular reduction, which corresponds to EvalMod. As modular reduction itself is not well-matched with polynomial approximation, the sine function is commonly used as a stepping-stone to evaluate modular reduction in bootstrapping algorithms. As a result, there are two kinds of error to be considered in EvalMod: one from distance between modular reduction and sine function, and the other from polynomial approximation of the sine function.

The first one, from distance between modular reduction and the sine function, is mainly determined by the ratio of the bootstrapping scaling factor Δ' to the modulus Q_0. Bootstrapping algorithms use scaling factor Δ' larger than default scaling factor Δ_{CKKS} used for basic arithmetic, since approximating modular reduction induces much larger error. In this case, the distance between the modular reduction and the sine function is bounded by Taylor's theorem as follows.

$$\left| \frac{Q_0}{\Delta'} \left[\frac{\Delta'}{Q_0} x \right]_1 - \frac{Q_0}{2\pi\Delta'} \sin\left(\frac{2\pi\Delta' x}{Q_0} \right) \right| \leq \frac{Q_0}{2\pi\Delta'} \cdot \frac{1}{3!} \left(\frac{2\pi\Delta'}{Q_0} \right)^3 = \frac{2\pi^2}{3} \left(\frac{\Delta'}{Q_0} \right)^2 \tag{3.1}$$

provided that $|x| \leq 1$.

The other error from polynomial approximation of the sine function is determined by the polynomial interpolation algorithms. In this step, Bossuat et al. [22] adopted a specialized Chebyshev interpolation proposed by Han and Ki [63] for sparse keys, and combine it with their optimization method, which is called errorless polynomial evaluation. The error bound is calculated based on the distribution of Chebyshev nodes which is empirically achieved, and we recommend to see [63] for further discussion. Similarly to (3.1), this error bound also decreases when Δ'/Q_0 gets smaller. Thus, we present an experimental result of correlation between Δ'/Q_0 and the message precision in Table 3.1.

KAIST

Δ'/Q_0	2^{-6}	2^{-7}	2^{-8}	2^{-9}	2^{-10}	2^{-11}	2^{-12}
$-\log \varepsilon$	11.29	13.29	15.30	17.29	19.28	21.24	22.73

Table 3.1: This table presents experimental error of HalfBoot for various Δ'/Q_0. The value ε is the mean error occurred by HalfBoot. The experiment is done by using parameter `Par-128` in Table 3.10 except Δ'/Q_0.

In our transciphering framework, the value Δ'/Q_0 is approximately δ/q. The plaintext modulus q should be larger than the number of precision bits, which is the reason for ciphertext expansion in our framework. This expansion can be reduced when arcsin is evaluated after the sine function.

After HalfBoot, we obtain a refreshed CKKS ciphertext of pre-determined scale Δ_{CKKS} as a result of RtF framework. Although we can freely choose the final scale Δ_{CKKS}, the message precision of the RtF framework cannot exceed $\log \delta$ bits. Hence it is enough to choose $\Delta_{\mathsf{CKKS}} \gg \delta N$ to ensure maximum precision $\log \delta$ against scaling error.

3.2 HERA: Batching-friendly Ciphers

In the RtF transciphering framework, a stream cipher using modular arithmetic is required as a building block. There are only a few ciphers using modular arithmetic [1, 9, 11, 59], and even such algorithms are not suitable for our transciphering framework due to their high multiplicative depths. In order to make our transciphering framework efficiently work, we propose a new HE-friendly cipher HERA, operating on a modular space with low multiplicative depth.

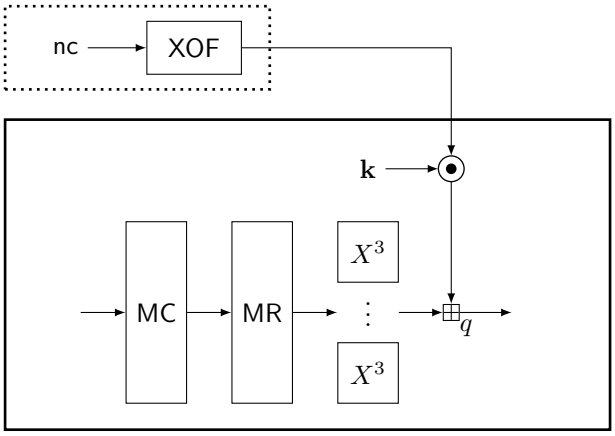


Figure 3.2: The round function of HERA. Operations in the box with dotted (resp. thick) lines are public (resp. secret). "MC" and "MR" represent MixColumns and MixRows, respectively.

Recent constructions for HE-friendly ciphers such as FLIP and Rasta use randomized linear layers in order to reduce the multiplicative depth without security degradation. However, this type of ciphers spend too many random bits to generate random matrices, slowing down the overall speed on both the client and the server sides. Instead of generating random matrices, we propose to randomize the key schedule algorithm by combining the secret key with a (public) random value for every round.

3.2.1 Specification

KAIST

A stream cipher HERA for λ-bit security takes as input a symmetric key $\mathbf{k} \in \mathbb{Z}_q^{16}$, a nonce $\mathsf{nc} \in \{0,1\}^\lambda$, and returns a keystream $\mathbf{k}_{\mathsf{nc}} \in \mathbb{Z}_q^{16}$, where the nonce is fed to the underlying extendable output function (XOF) that outputs an element in $(\mathbb{Z}_q^{16})^*$. In a nutshell, HERA is defined as follows.

$$\mathsf{HERA}[\mathbf{k}, \mathsf{nc}] = \mathsf{Fin}[\mathbf{k}, \mathsf{nc}, r] \circ \mathsf{RF}[\mathbf{k}, \mathsf{nc}, r-1] \circ \cdots \circ \mathsf{RF}[\mathbf{k}, \mathsf{nc}, 1] \circ \mathsf{ARK}[\mathbf{k}, \mathsf{nc}, 0]$$

where the i-th round function $\mathsf{RF}[\mathbf{k}, \mathsf{nc}, i]$ is defined as

$$\mathsf{RF}[\mathbf{k}, \mathsf{nc}, i] = \mathsf{ARK}[\mathbf{k}, \mathsf{nc}, i] \circ \mathsf{Cube} \circ \mathsf{MixRows} \circ \mathsf{MixColumns}$$

and the final round function Fin is defined as

$$\mathsf{Fin}[\mathbf{k}, \mathsf{nc}, r] = \mathsf{ARK}[\mathbf{k}, \mathsf{nc}, r] \circ \mathsf{MixRows} \circ \mathsf{MixColumns} \circ \mathsf{Cube} \circ \mathsf{MixRows} \circ \mathsf{MixColumns}$$

for $i = 1, 2, \ldots, r-1$ (see Figure 3.2).

Key Schedule. The round key schedule can be simply seen as component-wise product between a random value and the master key $\mathbf{k}$, where the uniformly random value in $\mathbb{Z}_q^\times$ is obtained from a certain extendable output function XOF with an input nc. Given a sequence of the outputs from XOF, say $\mathbf{rc} = (\mathbf{rc}_0, \ldots, \mathbf{rc}_r) \in (\mathbb{Z}_q^{16})^{r+1}$, ARK is defined as follows.

$$\mathsf{ARK}[\mathbf{k}, \mathsf{nc}, i](\mathbf{x}) = \mathbf{x} + \mathbf{k} \bullet \mathbf{rc}_i$$

for $i = 0, \ldots, r$, and $\mathbf{x} \in \mathbb{Z}_q^{16}$, where $\bullet$ (resp. $+$) denotes component-wise multiplication (resp. addition) modulo q. The extendable output function XOF might be instantiated with a sponge-type hash function

x_{00}	x_{01}	x_{02}	x_{03}
x_{10}	x_{11}	x_{12}	x_{13}
x_{20}	x_{21}	x_{22}	x_{23}
x_{30}	x_{31}	x_{32}	x_{33}

Figure 3.3: State of HERA. Each square stands for the component in $\mathbb{Z}_t$.

SHAKE256 [48].

Linear Layers. Each linear layer is the composition of MixColumns and MixRows. Similarly to AES, MixColumns multiplies a certain 4×4-matrix to each column of the state, where the state of HERA is also viewed as a 4×4-matrix over $\mathbb{Z}_q$ (see Figure 3.3). MixColumns and MixRows are defined as in Figure 3.4a and Figure 3.4b, respectively. The only difference of our construction from AES is that each entry of the matrix is an element of $\mathbb{Z}_q$.

$$\begin{bmatrix} y_{0c} \\ y_{1c} \\ y_{2c} \\ y_{3c} \end{bmatrix} = \begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix} \cdot \begin{bmatrix} x_{0c} \\ x_{1c} \\ x_{2c} \\ x_{3c} \end{bmatrix}$$

(a) MixColumns

$$\begin{bmatrix} y_{c0} \\ y_{c1} \\ y_{c2} \\ y_{c3} \end{bmatrix} = \begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix} \cdot \begin{bmatrix} x_{c0} \\ x_{c1} \\ x_{c2} \\ x_{c3} \end{bmatrix}$$

(b) MixRows

Figure 3.4: Definition of MixColumns and MixRows in HERA. For $c \in \{0, 1, 2, 3\}$, x_{ij} and y_{ij} are defined as in Figure 3.3.

KAIST

Nonlinear Layers. The nonlinear map Cube is the concatenation of 16 copies of the same S-box, where the S-box is defined by $x \mapsto x^3$ over $\mathbb{Z}_q$. So, for $\mathbf{x} = (x_0, \ldots, x_{15}) \in \mathbb{Z}_q^{16}$, we have

$$\mathsf{Cube}(\mathbf{x}) = (x_0^3, \ldots, x_{15}^3).$$

For the bijectivity of S-boxes, it is required that $\gcd(3, q-1) = 1$.

Encryption Mode. When a keystream of k blocks (in $(\mathbb{Z}_q^{16})^k$) is needed for some $k > 0$, the "inner-counter mode" can be used; for $\mathsf{ctr} = 0, 1, \ldots, k-1$, one computes

$$\mathbf{z}[\mathsf{ctr}] = \mathsf{HERA}\left[\mathbf{k}, \mathsf{nc} \| \mathsf{ctr}\right](\mathbf{ic}),$$

where $\mathbf{ic}$ denotes a constant $(1, 2, \ldots, 16) \in \mathbb{Z}_q^{16}$.

3.2.2 Design Rationale

Symmetric cipher designs for advanced protocols so far have been targeted at homomorphic encryption as well as various privacy preserving protocols such as multiparty computation (MPC) and zero knowledge proof (ZKP). In such protocols, multiplication is significantly more expensive than addition, so a new design principle has begun to attract attention in the literature: to use simple nonlinear layers at the cost of highly randomized linear layers (e.g., FLIP [85] and Rasta [44]). However, to the best of our knowledge, most symmetric ciphers following this new design principle operate only on binary spaces, rendering it difficult to apply them to our hybrid framework.

One might consider literally extending FLIP [85] or Rasta [44] to modular spaces. This straightforward approach will degrade the overall efficiency of the cipher. Furthermore, unlike MPC and ZKP, linear maps over homomorphically encrypted data may not be simply "free". In order to use the batching techniques for homomorphic evaluation, the random linear layers should be encoded into HE-plaintexts, and then applied to HE-ciphertexts. Since multiplication between (encoded) plaintexts and ciphertexts require $O(N \log N)$ time (besides many HE rotations), randomized linear layers might not be that practical except that a small number of rounds are sufficient to mitigate algebraic attacks. For this reason, we opted for fixed linear layers.

In Table 3.2, we compare different types of linear maps to the (nonlinear) Cube map in terms of evaluation time and noise consumption. This experiment is conducted with the HE-parameters $(N, \lceil \log Q \rceil) = (32768, 275)$ using row-wise packing, where the noise budget after the initialization is set to 239 bits. In this table, "Fixed matrix" and "Freshly-generated matrix" represent a non-sparse fixed matrix, and a set of distinct matrices freshly generated over different slots, respectively, where all the matrices are 16×16 square matrices and randomly generated. We see that a freshly-generated linear layer takes more time than Cube. A fixed linear layer is better than a freshly-generated one, but its time complexity is not negligible yet compared to Cube. On the other hand, our linear layer is even faster than (uniformly sampled) fixed linear layer due to its sparsity.

	Time (ms)	Consumed Noise (bits)
MixRows ∘ MixColumns	23.55	4
Fixed matrix	461.68	27
Freshly-generated matrix	4006.03	34.9
Cube	3479.07	86.4

KAIST

Table 3.2: Comparisons of different types of maps in terms of evaluation time and noise consumption.

The HERA cipher uses a sparse linear layer, whose design is motivated by the MixColumns layer in AES, enjoying a number of nice features; it is easy to analyze since its construction is based on an MDS (maximum distance separable) matrix and needs a small number of multiplications due to the sparsity of the matrix. We design a $\mathbb{Z}_q$-variant of the matrix and use it in the linear layers; it turns out to be an MDS matrix over $\mathbb{Z}_q$ when q is a prime number such that $q > 17$. Instead of using ShiftRows of AES, HERA uses an additional layer MixRows which is a "row version" of MixColumns to enhance the security against algebraic attacks; the composition of two linear functions generates all possible monomials, which makes algebraic attacks infeasible. Also, using MixRows mitigates linear cryptanalysis; the branch number of the linear layer is 8 (see Appendix A.1) so that HERA does not have a high-probability linear trail.

In the nonlinear layer, Cube takes the component-wise cube of the input. The cube map is studied from earlier multivariate cryptography [82], recently attracting renewed interest for the use in MPC/ZKP-friendly ciphers [1, 9]. The cube map has good linear/differential characteristics, whose inverse is of high degree, mitigating meet-in-the-middle algebraic attacks.

As multiplicative depth heavily impacts on noise growth of HE-ciphertexts, it is desirable to design HE-friendly ciphers using a small number of rounds. One of the most threatening attacks on ciphers with low algebraic degrees is the higher order differential attack. For a λ-bit secure (possibly non-binary) cipher, the algebraic degree of the cipher should be at least $\lambda - 1$. However, the attack is not available on randomized ciphers such as FLIP and Rasta.

To balance between efficiency and security, we propose a new direction: randomizing the key schedule. A randomized key schedule (RKS) is motivated by the tweakey framework [70]. In the tweakey

framework, a key schedule takes as input a public value (called a tweak) and a key, where an adversary is allowed to take control of tweaks. On the other hand, RKS is a key schedule which takes as input a randomized public value and a key together, where the random value comes from a certain pseudorandom function. So, in our design, an adversary is not able to freely choose the random value.

The design principle behind our RKS is simple: to use as small number of multiplications as possible. One might consider simply adding a fresh random value to the master key for every round. This type of key schedule might provide security against differential cryptanalysis, but it still might be vulnerable to algebraic attacks and linear cryptanalysis. It is important to enlarge the number of monomials in the first linear layer, while this candidate cannot obtain this effect since an adversary is able to use the linear change of variables (see Section 3.3.2). Based on this observation, we opt for component-wise multiplication. It offers better security on algebraic attacks and linear cryptanalysis. For a traditional block cipher using fixed keys, outer affine layers do not affect its overall security; when it comes to HERA, the first and the last affine layers, combined with the randomized key schedule, increases the number of monomials.

The input constant $\mathbf{ic} = (1, 2, \ldots, 16)$ consists of distinct numbers in $\mathbb{Z}_q^{16}$; it will make a larger number of monomials in the polynomial representation of the cipher (compared to using a too simple constant, say the all-zero vector), enhancing security against algebraic attacks.

3.3 Security Analysis of HERA

In this section, we provide the security analysis of HERA. Table 3.3 shows the number of rounds to prevent each of the attacks considered in this section according to the security level λ, where we assume that $q > 2^{16}$.

KAIST

Attack \ λ	80	128	192	256
Trivial Linearization	4	5	6	7
GCD Attack	1	1	1	7
Gröbner Basis Attack	4	5	6	7
Interpolation Attack	4	5	6	7
Linear Cryptanalysis	2	4	4	6

Table 3.3: Recommended number of rounds with respect to each attack.

Assumptions and the Scope of Analysis. We limit the number of encryptions under the same key up to the birthday bound with respect to λ, i.e., $2^{\lambda/2}$, since otherwise one would not be able to avoid a nonce collision (when nonces are chosen uniformly at random).

In this work, we will consider the standard "secret-key model", where an adversary arbitrarily chooses a nonce, and obtains the corresponding keystream without any information on the secret key. The related-key and the known-key models are beyond the scope of this paper.

Since HERA takes as input counters, an adversary is not able to control the differences of the inputs. Nonces can be adversarially chosen, while they are also fed to the extendable output function, which is modeled as a random oracle. So one cannot control the difference of the internal variables. For this reason, we believe that our construction is secure against any type of chosen-plaintext attack including (higher-order) differential, truncated differential, invariant subspace trail and cube attacks. A recent

generalization of an integral attack [19] requires only a small number of chosen plaintexts, while it is not applicable to HERA within the birthday bound.

The HERA cipher can be represented by a set of polynomials over $\mathbb{Z}_q$ in unknowns $k_0, \ldots, k_{15}$, where $k_i \in \mathbb{Z}_q$ denotes the i-th component of the secret key $\mathbf{k} \in \mathbb{Z}_q^{16}$. Since multiplication is more expensive than addition in HE schemes, most HE-friendly ciphers have been designed to have a low multiplicative depth. This property might possibly make such ciphers vulnerable to algebraic attacks. With this observation, our analysis will be focused on algebraic attacks.

3.3.1 On the Number of Monomials in HERA

The round function of HERA is defined by

$$\mathsf{RF} = \mathsf{ARK} \circ \mathsf{Cube} \circ \mathsf{MixRows} \circ \mathsf{MixColumns},$$

where the two linear maps MixColumns and MixRows can be represented by 16×16-matrices over $\mathbb{Z}_q$. Their product represents MixRows $\circ$ MixColumns as follows.

$$\mathsf{MixRows} \circ \mathsf{MixColumns} = \begin{pmatrix}
4 & 6 & 2 & 2 & 6 & 9 & 3 & 3 & 2 & 3 & 1 & 1 & 2 & 3 & 1 & 1 \\
2 & 4 & 6 & 2 & 3 & 6 & 9 & 3 & 1 & 2 & 3 & 1 & 1 & 2 & 3 & 1 \\
2 & 2 & 4 & 6 & 3 & 3 & 6 & 9 & 1 & 1 & 2 & 3 & 1 & 1 & 2 & 3 \\
6 & 2 & 2 & 4 & 9 & 3 & 3 & 6 & 3 & 1 & 1 & 2 & 3 & 1 & 1 & 2 \\
2 & 3 & 1 & 1 & 4 & 6 & 2 & 2 & 6 & 9 & 3 & 3 & 2 & 3 & 1 & 1 \\
1 & 2 & 3 & 1 & 2 & 4 & 6 & 2 & 3 & 6 & 9 & 3 & 1 & 2 & 3 & 1 \\
1 & 1 & 2 & 3 & 2 & 2 & 4 & 6 & 3 & 3 & 6 & 9 & 1 & 1 & 2 & 3 \\
3 & 1 & 1 & 2 & 6 & 2 & 2 & 4 & 9 & 3 & 3 & 6 & 3 & 1 & 1 & 2 \\
2 & 3 & 1 & 1 & 2 & 3 & 1 & 1 & 4 & 6 & 2 & 2 & 6 & 9 & 3 & 3 \\
1 & 2 & 3 & 1 & 1 & 2 & 3 & 1 & 2 & 4 & 6 & 2 & 3 & 6 & 9 & 3 \\
1 & 1 & 2 & 3 & 1 & 1 & 2 & 3 & 2 & 2 & 4 & 6 & 3 & 3 & 6 & 9 \\
3 & 1 & 1 & 2 & 3 & 1 & 1 & 2 & 6 & 2 & 2 & 4 & 9 & 3 & 3 & 6 \\
6 & 9 & 3 & 3 & 2 & 3 & 1 & 1 & 2 & 3 & 1 & 1 & 4 & 6 & 2 & 2 \\
3 & 6 & 9 & 3 & 1 & 2 & 3 & 1 & 1 & 2 & 3 & 1 & 2 & 4 & 6 & 2 \\
3 & 3 & 6 & 9 & 1 & 1 & 2 & 3 & 1 & 1 & 2 & 3 & 2 & 2 & 4 & 6 \\
9 & 3 & 3 & 6 & 3 & 1 & 1 & 2 & 3 & 1 & 1 & 2 & 6 & 2 & 2 & 4
\end{pmatrix}.$$

KAIST

We see that the matrix representation of MixRows $\circ$ MixColumns has no zero entry. It implies that MixRows $\circ$ MixColumns contains all the linear monomials in its polynomial representation, and hence RF contains all the cubic monomials. More precisely, if $a_i \neq 0$ for $i = 0, 1, \ldots, n-1$, then we have

$$\begin{aligned}
&(a_0x_0 + a_1x_1 + \cdots + a_{n-1}x_{n-1} + b)^3 \\
&= \sum_{i,j,k} a_ia_ja_kx_ix_jx_k + \sum_{i,j} 3a_ia_jbx_ix_j + \sum_i 3a_ib^2x_i + b^3 \\
&= \sum_{i \le j \le k} \alpha(i,j,k)a_ia_ja_kx_ix_jx_k + \sum_{i \le j} \beta(i,j)a_ia_jbx_ix_j + \sum_i 3a_ib^2x_i + b^3,
\end{aligned}$$

where

$$\alpha(i,j,k) = \begin{cases} 1 & \text{if } i=j=k; \\ 3 & \text{if either } i=j<k \text{ or } i<j=k; \\ 6 & \text{if } i<j<k, \end{cases}$$

and

$$\beta(i,j) = \begin{cases} 3 & \text{if either } i=j; \\ 6 & \text{if } i<j. \end{cases}$$

Since the plaintext modulus q is prime and $q > 2^{16}$, every monomial of degree 3 has a nonzero coefficient.

We can estimate the number of monomials in HERA with more rounds. Let $\mathbf{b} = (b_0, \ldots, b_{15})$ be the output of the first round function. The second round function will contain all the cubic monomials in $\mathbf{b}$. When we view the second round function as a polynomial in $b_0, \ldots, b_{15}$, some coefficients might become zero, while this happens only with probability of $1/q$. Heuristically (with the independence assumption), each monomial will remain at the second round with probability $1-(1/q)^{16}$. This heuristic is confirmed by our computation, showing all possible monomials at the end of the second round. We conjecture that this property will hold for more than two rounds.

3.3.2 Trivial Linearization

In Section 3.3.1, we explain the reason that all the monomials of degree $\leq 3^r$ are expected to appear after r rounds of HERA. Therefore, we can conclude that this attack requires $O(S)$ data and $O(S^\omega)$ time, where $2 \leq \omega \leq 3$ and

$$S = \sum_{i=0}^{3^r} \binom{16+i-1}{i}.$$

KAIST

We note that Guessing variables will not affect the security of HERA when $q > 2^{16}$.

On the First and the Last Affine Layers. A classic substitution-permutation block cipher does not begin with nor end with affine layers since they do not affect the overall security of the cipher. However, this is not the case for HERA. In the following, we give a trivial linearization attack for a variant of HERA where either the first or the last affine layer is missing.

If the first affine layer is missing, then the output of the first nonlinear layer will be of the form

$$(a_0(k_0+1)^3, a_1(k_1+2)^3, \ldots, a_{15}(k_{15}+16)^3)$$

for public coefficients $a_0, \ldots, a_{15}$, and unknowns (key variables) $k_0, \ldots, k_{15}$. An adversary can try trivial linearization with respect to the variables $(k'_0, \ldots, k'_{15})$ where $k'_i = (k_i + i + 1)^3$ for $i = 0, \ldots, 15$. The maximum number of all monomials is

$$S = \sum_{i=0}^{3^{r-1}} \binom{16+i-1}{i}$$

which is much lower than expected with the first affine layer.

On the other hand, if the last affine layer is missing, the input to the last nonlinear layer will be of the form

$$((c_0 - a_0k_0)^\alpha, (c_1 - a_1k_1)^\alpha, \ldots, (c_{15} - a_{15}k_{15})^\alpha)$$

where α is the inverse of 3 modulo $(q-1)$, $c = (c_0, \ldots, c_{15})$ is a keystream, and a_i is the i-th component of $\mathbf{rc}_r$. It can be converted to the form

$$(a_0^\alpha (b_0 - k_0)^\alpha, a_1^\alpha (b_1 - k_1)^\alpha, \ldots, a_{15}^\alpha (b_{15} - k_{15})^\alpha)$$

where $b_i = c_i a_i^{-\alpha}$. Setting new variables $k_{i,j} = k_i - j$ for $0 \leq j \leq t-1$, for r-round HERA^r, we can establish a meet-in-the-middle (MitM) equation

$$\mathsf{HERA}^{r-1}[k, \mathsf{nc}] = (-a_i^\alpha k_{i,b_i}^\alpha)_{0 \leq i \leq 15}.$$

When the number of equations is sufficiently large, the maximum number of monomials is given as

$$S = 16(q-1) + \sum_{i=0}^{3^{r-1}} \binom{16+i-1}{i}$$

which is also much lower than expected with the last affine layer.

3.3.3 GCD Attack

The GCD attack can be extended to a system of multivariate polynomial equations by guessing all the key variables except one. For r-round HERA, the complexity of the GCD attack is estimated as $O(q^{15} r^2 3^r)$. For a security parameter $\lambda \leq 240$, HERA will be secure against the GCD attack even with a single round as long as $q > 2^{16}$. If $\lambda = 256$, then the number of round should be at least 7.

KAIST

3.3.4 Gröbner Basis Attack

For the Gröbner basis attack, re-arranging equations may lead to a significant impact on the attack complexity. For example, one may set a system of equations using only plaintext-ciphertext pairs, or set an equation with new variables standing for internal states. The former will be a higher-degree system in a fewer variables, while the latter will be a lower-degree system in more variables.

From a set of nonce-plaintext-ciphertext triples $\{(\mathsf{nc}_i, \mathbf{m}_i, \mathbf{c}_i)\}$, an attacker will be able to establish an over-determined system of equation

$$f_1(k_0, \ldots, k_{15}) = f_2(k_0, \ldots, k_{15}) = \cdots = f_m(k_0, \ldots, k_{15}) = 0$$

where $k_i \in \mathbb{Z}_t$ is i-th component of the key variable. The degree of regularity of the system is computed as the degree of the first non-positive coefficient in

$$\left(1 - z^{3^r}\right)^{m-16} \left(\sum_{i=0}^{3^r-1} z^i\right)^{16}$$

where r is the number of rounds. Since the summation does not have any negative term, one easily see that the degree d_{reg} of regularity cannot be smaller than 3^r. So, in this case, the time complexity is lower bounded by

$$O\left(\binom{16+3^r}{3^r}^2\right).$$

Note that the hybrid approach has worse complexity when $r \leq 6$ and $q > 2^{16}$. Even for $r = 7$, there is

no significant impact on the security.

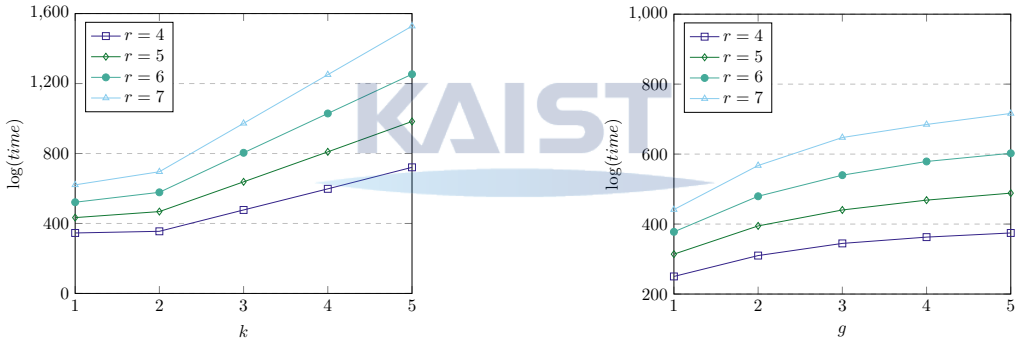
Instead of a system of equations of degree 3^r, one can establish a system of $16rk$ cubic equations in $16(r-1)k+16$ variables, where k is the block length of each query. Then, the complexity is

$$O\left(\binom{16(r-1)k+16+d_{reg}(r,k)}{d_{reg}(r,k)}^{\omega}\right).$$

We compute the degree $d_{reg}(r,k)$ of regularity assuming semi-regular sequences in Table 3.4, and give a plot of complexity according to k in Figure 3.5a. We see that the degree of regularity increases from $k=2$ as k grows. It seems too costly to attack HERA with this method.

r \ k	1	2	3	4	5
4	129	75	95	115	136
5	161	100	129	159	190
6	193	124	165	206	247
7	235	151	202	254	306

Table 3.4: $d_{reg}(r,k)$ for a semi-regular system of $16rk$ cubic equations in $16(r-1)k+16$ variables.



(a) Complexity as a function in k.

(b) Complexity as a function in g.

Figure 3.5: Complexity of the Gröbner basis attack with $\omega=2$ and $t=2^{16}+1$.

Suppose that one takes the hybrid approach. As the degree of regularity differs with a huge gap between $m=n$ and $m=n+1$, we arrange the degrees $d'_{reg}(r,g)$ of regularity when $k=1$ and the number g of guess is positive in Table 3.5, and give a plot of complexity depending g in Figure 3.5b.

r \ g	1	2	3	4	5
4	65	59	55	51	48
5	81	74	69	65	62
6	97	89	84	80	76
7	113	105	101	94	90

Table 3.5: $d'_{reg}(r,g)$ for a semi-regular system of $16r$ cubic equations in $16r$ variables with g guesses.

Provided that the complexity is

$$O\left(q^g \binom{16r - g + d'_{reg}(r,g)}{d'_{reg}(r,g)}^{\omega}\right),$$

we see that HERA provides at least 256-bit level of security for any set of parameters.

3.3.5 Interpolation Attack

For the r-round HERA cipher, let $\mathbf{rc} = (\mathbf{rc}_0, \ldots, \mathbf{rc}_r) \in (\mathbb{Z}_q^{16})^{r+1}$ be a sequence of the outputs from XOF. For $i = 0, \ldots, r$, $\mathbf{rc}_i$ is evaluated by a polynomial of degree 3^{r-i}. As we expect that the r-round HERA cipher has almost all monomials of degree $\leq 3^r$ in its polynomial representation, the number of monomials is lower bounded by

$$\sum_{j=0}^{r} \sum_{i=0}^{3^j} \binom{16+i-1}{i}.$$

One might try to recover the secret key using the interpolation attack on $r-1$ rounds. However, HERA uses the full key material for every round. It implies that the key recovery attack needs brute-force search for all the key space.

Meet-in-the-middle Approach. In the basic approach, one considers a polynomial equivalent to the output, while one might try to find a polynomial equation in the middle state. However, the inverse of the cube map is of degree $(2q-1)/3$, so the degree of the equation in the middle state will be too high to recover all its coefficients. So we conclude that the meet-in-the-middle approach is not applicable to HERA.

KAIST

3.3.6 Linear Cryptanalysis

One might consider two different approaches in the application of linear cryptanalysis on HERA according to how to take the input variables: the XOF output variables or the key variables. In the first case, unlike traditional linear cryptanalysis, the probability of any linear trail of HERA depends on the key since it is multiplied to the input. It seems infeasible to make a plausible linear trail without any information on the key material.

In the second case, the attack is reduced to solving an LWE-like problem as follows; given pairs $(\mathsf{nc}_i, \mathbf{y}_i)$ such that $\mathsf{HERA}(\mathbf{k}, \mathsf{nc}_i) = \mathbf{y}_i$, one can establish

$$\langle \mathbf{b}, \mathbf{y}_i \rangle = \langle \mathbf{a}, \mathbf{k} \rangle + e_i$$

for some vectors $\mathbf{a} \neq \mathbf{0}, \mathbf{b} \in \mathbb{Z}_q^n$ and error e_i sampled according to a certain distribution χ. It requires $1/\operatorname{ELP}^{\mathsf{E}}(\mathbf{a}, \mathbf{b})$ samples to distinguish χ from the uniform distribution [13]. The linear probability of r-round HERA is upper bounded by $(\mathrm{LP}^S)^{B_\ell \cdot \lfloor \frac{r}{2} \rfloor}$, where LP^S and B_ℓ denote the linear probability of the S-box and the (linear) branch number of the linear layer, respectively. Therefore, the data complexity for linear cryptanalysis is lower bounded approximately by

$$\frac{1}{(\mathrm{LP}^S)^{B_\ell \cdot \lfloor \frac{r}{2} \rfloor}}.$$

The linear probability LP^S is upper bounded as follows.

Lemma 3.3.1. For an odd prime t, let $S : \mathbb{Z}_q \to \mathbb{Z}_q$ be a permutation such that $S(x) = x^3$. Then, for any pair $(\alpha, \beta) \in \mathbb{Z}_q^2$ such that $\alpha \neq 0$, we have

$$\mathrm{LP}^S(\alpha, \beta) \leq \frac{4}{q}.$$

Proof. By the definition of LP, we have

$$\mathrm{LP}^S(\alpha, \beta) = \left|\mathbb{E}_m\Big[-\alpha m + \beta S(m)\Big]\right|^2 = \left|\frac{1}{q}\sum_{m=0}^{q-1} \exp\left\{\frac{2\pi i}{q}\left(-\alpha m + \beta m^3\right)\right\}\right|^2.$$

Carlitz and Uchiyama [28] proved that

$$\left|\sum_{x=0}^{q-1} \exp\left(\frac{2\pi i}{q} \cdot p(x)\right)\right| \leq (r-1)\sqrt{q}$$

for any polynomial $p(x)$ of degree r over $\mathbb{Z}_q$. Therefore, we have

$$\mathrm{LP}^S(\alpha, \beta) = \left|\frac{1}{q}\sum_{m=0}^{q-1} \exp\left\{\frac{2\pi i}{q}\Big(-\alpha m + \beta m^3\Big)\right\}\right|^2 \leq \frac{1}{q^2}\left(2\sqrt{q}\right)^2 = \frac{4}{q}.$$

□

We will show the branch number of the linear layer of HERA is 8 in Appendix A.1. Combined with Lemma 3.3.1, we can conclude that an r-round HERA cipher provides λ-bit security against linear cryptanalysis when

$$\left(\frac{q}{4}\right)^{8\cdot\lfloor\frac{r}{2}\rfloor} > 2^\lambda.$$

KAIST

3.3.7 Differential Cryptanalysis and Its Variants

Since HERA takes as input counters, an adversary is not able to control the differences of its inputs. Nonces can be adversarially chosen, while they are also fed to the extendable output function, which is modeled as a random oracle. So one cannot control the difference of the internal variables. For the reason, we believe that our construction will be secure against any type of chosen-plaintext attack including (higher-order) differential, truncated differential, invariant subspace trail and cube attacks.

Nonetheless, for an unsuspected differential-related attack, we present a computation of differential characteristic in the following. Resistance of a substitution-permutation cipher against differential cryptanalysis is typically estimated by the maximum probability of differential trails [20]. Let $S : \mathbb{Z}_q \to \mathbb{Z}_q$ be the nonlinear map (S-box) used in Cube. Given a pair $(\alpha, \beta) \in \mathbb{Z}_q^\times \times \mathbb{Z}_q$, the differential probability of S is defined by

$$\mathrm{DP}^S(\alpha, \beta) = \frac{1}{q} \cdot \left|\{x \in \mathbb{Z}_q | S(x+\alpha) - S(x) = \beta\}\right|.$$

So $\mathrm{DP}^S(\alpha, \beta)$ is determined by the number of solutions to $S(x+\alpha) - S(x) = \beta$, which is a quadratic equation in x since $S(x) = x^3$. Therefore, there are at most two solutions to this equation, which implies $\mathrm{DP}^S(\alpha, \beta) \leq \frac{2}{q}$.

Since the branch number of the linear layer of HERA is 8 (as shown in Appendix A.1), an r-round

HERA cipher provides λ-bit security against differential cryptanalysis when

$$\left(\frac{q}{2}\right)^{8\cdot\lfloor\frac{r}{2}\rfloor} > 2^{\lambda}.$$

3.4 Rubato: Noisy Ciphers for the RtF Framework

Designing a symmetric cipher can be seen as a trade-off between security and efficiency. A designer should identify important cost metrics of the targeted platform (e.g., x86, ARM, and HE), and focus on optimizing them within a given security level. When it comes to HE-friendly ciphers, one of the most important cost metrics is the time for evaluating the cipher while homomorphically encrypted, typically translated to multiplicative depth and complexity in the literature. To optimize such metrics, quadratic S-boxes [4], random linear layers [85], and nonlinear layers with high-degree inverses [44] have been used.

In this regard, the LWE encryption has promising properties as an HE-friendly cipher since it is based on a linear combination of key material, while noise prevents algebraic attacks. However, straightforward application of the LWE encryption has a disadvantage on the client-side; in the LWE encryption, $(\mathbf{a}, b = \langle \mathbf{a}, \mathbf{s} \rangle + e)$ is sampled from an LWE distribution, where $\mathbf{a}$ should be freshly generated by a pseudorandom function for every encryption. It makes the LWE encryption too costly on the client-side compared to conventional symmetric encryption.

In this section, we propose a new HE-friendly cipher, dubbed Rubato, as a cost-effective trade-off between the LWE encryption and conventional symmetric encryption in a transciphering framework for approximate homomorphic encryption. In particular, when the RtF transciphering framework is used, we can add noise only with a partial loss of precision. For a *low-degree* keyed function $\mathsf{E}_{\mathbf{k}} : \mathbb{Z}_q^k \times \mathbb{Z}_q^* \to \mathbb{Z}_q^\ell$, each sample is of the form

$$(\mathbf{a}, \mathsf{E}_{\mathbf{k}}(\mathbf{a}) + \mathbf{e})$$

where $\mathbf{e} \in \mathbb{Z}_q^\ell$ is sampled from a discrete Gaussian distribution, and $\mathbf{a} \in \mathbb{Z}_q^*$ is generated by an extendable output function (XOF) with a nonce. We remark that such a noisy cipher is not suitable for transciphering of exact data since the server might lose some information on the original message after transciphering. In Table 3.6, we compare Rubato to existing HE-friendly ciphers operating on modular domains assuming 128-bit security and the same modulus q.

Cipher	Masta	HERA	Pasta	LWE	Rubato
#(Key words)	16	16	64	1024	64
Multiplicative depth	7	10	5	0	2
#(Multiplications)	7	10	9.81	0	2.1
Random bits	400	150	250	25600	80
Source	[61]	Section 3.2	[43]	[93]	**This section**

Table 3.6: Comparison of HE-friendly ciphers operating on modular domains, where the modulus q is set to 25 bits. "#(Key words)" is the number of key words in $\mathbb{Z}_q$ and "#(Multiplications)" (resp. "Random bits") is the number of multiplications (resp. random bits) required to generate a single component of a ciphertext.

Since Rubato is a combination of a conventional symmetric cipher and the LWE encryption, we analyze its security in two ways: symmetric cryptanalysis and LWE cryptanalysis. We apply the symmetric cryptanalysis by guessing all the noise, while LWE cryptanalysis is considered by linearizing monomials to new variables. From extensive analysis, we recommend a set of parameters for various applications.

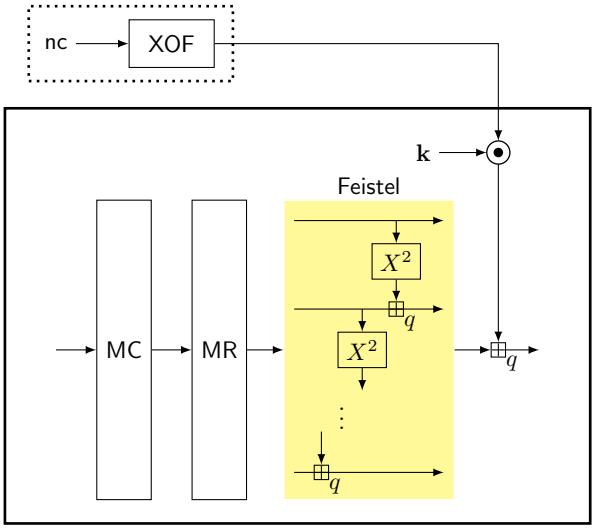


Figure 3.6: The round function of Rubato. Operations in the box with dotted (resp. thick) lines are public (resp. secret). "MC" and "MR" represent MixColumns and MixRows, respectively.

3.4.1 Specification

The Rubato cipher is designed to be flexible in block size so that it offers a more suitable choice of parameters for various applications. The block size n is the square of a positive integer v, which defines the size of matrices in linear layers. The stream cipher Rubato for λ-bit security takes as input a symmetric key $\mathbf{k} \in \mathbb{Z}_q^n$, a nonce $\mathsf{nc} \in \{0,1\}^\lambda$, and returns a keystream $\mathbf{k}_{\mathsf{nc}} \in \mathbb{Z}_q^\ell$ for some $\ell < n$, where the nonce is fed to the underlying extendable output function (XOF) that outputs an element in $(\mathbb{Z}_q^n)^*$. In a nutshell, Rubato is defined as follows.

$$\mathsf{Rubato}[\mathbf{k}, \mathsf{nc}] = \mathsf{AGN} \circ \mathsf{Fin}[\mathbf{k}, \mathsf{nc}, r] \circ \mathsf{RF}[\mathbf{k}, \mathsf{nc}, r-1] \circ \cdots \circ \mathsf{RF}[\mathbf{k}, \mathsf{nc}, 1] \circ \mathsf{ARK}[\mathbf{k}, \mathsf{nc}, 0]$$

where the i-th round function $\mathsf{RF}[\mathbf{k}, \mathsf{nc}, i]$ is defined as

$$\mathsf{RF}[\mathbf{k}, \mathsf{nc}, i] = \mathsf{ARK}[\mathbf{k}, \mathsf{nc}, i] \circ \mathsf{Feistel} \circ \mathsf{MixRows} \circ \mathsf{MixColumns}$$

and the final round function Fin is defined as

$$\mathsf{Fin}[\mathbf{k}, \mathsf{nc}, r] = \mathsf{Tr}_{n,\ell} \circ \mathsf{ARK}[\mathbf{k}, \mathsf{nc}, r] \circ \mathsf{MixRows} \circ \mathsf{MixColumns} \circ \mathsf{Feistel} \circ \mathsf{MixRows} \circ \mathsf{MixColumns}$$

for $i = 1, 2, \ldots, r-1$ (see Figure 3.6).

Key Schedule. The round key schedule can be simply seen as a component-wise product between random values and the master key $\mathbf{k}$, where the uniformly random values in $\mathbb{Z}_q^\times$ are obtained from a certain extendable output function XOF with an input nc. Given a sequence of the outputs from XOF, say $\mathbf{rc} = (\mathbf{rc}_0, \ldots, \mathbf{rc}_r) \in (\mathbb{Z}_q^n)^{r+1}$, ARK is defined as follows.

$$\mathsf{ARK}[\mathbf{k}, \mathsf{nc}, i](\mathbf{x}) = \mathbf{x} + \mathbf{k} \bullet \mathbf{rc}_i$$

for $i = 0, \ldots, r$, and $\mathbf{x} \in \mathbb{Z}_q^n$, where $\bullet$ (resp. $+$) denotes component-wise multiplication (resp. addition) modulo q. The extendable output function XOF might be instantiated with a sponge-type hash function SHAKE [48].

KAIST

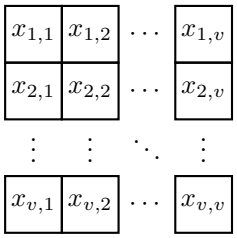


Figure 3.7: State of Rubato. Each square stands for the component in $\mathbb{Z}_q$.

Linear Layers. Each linear layer is the composition of MixColumns and MixRows. Similarly to HERA, MixColumns (resp. MixRows) multiplies a certain $v \times v$ MDS matrix $\mathbf{M}_v$ to each column (resp. row) of the state as in Figure 3.8a and Figure 3.8b, where the state of Rubato is also viewed as a $v \times v$-matrix over $\mathbb{Z}_q$ (see Figure 3.7). The MDS matrix $\mathbf{M}_v$ for $v = 4, 6, 8$ is defined as follows.

$$\begin{aligned}
\mathbf{y}_4 &= [2, 3, 1, 1] \\
\mathbf{y}_6 &= [4, 2, 4, 3, 1, 1] \\
\mathbf{y}_8 &= [5, 3, 4, 3, 6, 2, 1, 1] \\
\mathbf{M}_v &= \begin{bmatrix} \mathbf{y}_v \\ \mathrm{ROT}^1(\mathbf{y}_v) \\ \vdots \\ \mathrm{ROT}^{v-1}(\mathbf{y}_v) \end{bmatrix}
\end{aligned}$$

where $\mathrm{ROT}^i(\mathbf{y})$ is the rotation to the right of $\mathbf{y}$ by i components. Therefore, $\mathbf{M}_v$ is a circulant matrix derived from $\mathbf{y}_v$.

$$\begin{bmatrix} y_{1,c} \\ y_{2,c} \\ \vdots \\ y_{v,c} \end{bmatrix} = \mathbf{M}_v \cdot \begin{bmatrix} x_{1,c} \\ x_{2,c} \\ \vdots \\ x_{v,c} \end{bmatrix}$$

(a) MixColumns

$$\begin{bmatrix} y_{c,1} \\ y_{c,2} \\ \vdots \\ y_{c,v} \end{bmatrix} = \mathbf{M}_v \cdot \begin{bmatrix} x_{c,1} \\ x_{c,2} \\ \vdots \\ x_{c,v} \end{bmatrix}$$

(b) MixRows

Figure 3.8: Definition of MixColumns and MixRows. For $c \in \{1, 2, \ldots, v\}$, x_{ij} and y_{ij} are defined as in Figure 3.7.

Nonlinear Layers. The nonlinear map Feistel is a Feistel network in a row, which was proposed in [43]. For $\mathbf{x} = (x_1, \ldots, x_n) \in \mathbb{Z}_q^n$, we have

$$\mathsf{Feistel}(\mathbf{x}) = (x_1, x_2 + x_1^2, x_3 + x_2^2, \ldots, x_n + x_{n-1}^2).$$

It is naturally bijective and of degree 2.

Truncation. The truncation function $\mathsf{Tr}_{n,\ell} : \mathbb{Z}_q^n \to \mathbb{Z}_q^\ell$ is just a truncation of the last $n - \ell$ words. For $\mathbf{x} = (x_1, \ldots, x_n) \in \mathbb{Z}_q^n$, we have

$$\mathsf{Tr}_{n,\ell}(\mathbf{x}) = (x_1, \ldots, x_\ell).$$

KAIST

Although we know that the truncation function makes some part of the last ARK and MixRows meaningless, we write it in this way for the brevity. We recommend to instantiate $\mathsf{Tr}_{n,\ell} \circ \mathsf{ARK}[\mathbf{k}, \mathsf{nc}, r] \circ \mathsf{MixRows}$ as a whole in real implementation.

Adding Gaussian Noise. At the very last of the cipher, we add Gaussian noise to every component. From an one-dimensional discrete Gaussian distribution $D_{\alpha q}$ with zero mean and variance $(\alpha q)^2/2\pi$, we sample ℓ elements $e_1, \ldots, e_\ell \leftarrow D_{\alpha q}$ independently. For $\mathbf{x} = (x_1, \ldots, x_\ell) \in \mathbb{Z}_q^\ell$, we have

$$\mathsf{AGN}(\mathbf{x}) = (x_1 + e_1, \ldots, x_\ell + e_\ell).$$

Encryption Mode. When a keystream of k blocks (in $(\mathbb{Z}_q^\ell)^k$) is needed for some $k > 0$, the "inner-counter mode" can be used; for $\mathsf{ctr} = 0, 1, \ldots, k-1$, one computes

$$\mathbf{z}[\mathsf{ctr}] = \mathsf{Rubato}\,[\mathbf{k}, \mathsf{nc}\|\mathsf{ctr}]\,(\mathbf{ic}),$$

where $\mathbf{ic}$ denotes a constant $(1, 2, \ldots, n) \in \mathbb{Z}_q^n$. For a given message vector $\mathbf{m} \in (\mathbb{R}^\ell)^k$, encryption by Rubato is defined by

$$\mathbf{c} = \lfloor \Delta \cdot \mathbf{m} \rceil + \mathbf{z} \pmod q$$

where $\Delta \in \mathbb{R}$ is a scaling factor.

3.4.2 Design Rationale

KAIST

The main observation behind our design is that adding noise increases the algebraic degree of a cipher. Suppose that we are given LWE samples $\{(\mathbf{a}_i, b_i = \langle \mathbf{a}_i, \mathbf{s} \rangle + e_i)\}_i$. In the Arora-Ge attack [10], an attacker establishes an equation

$$\prod_{e=-t\alpha q}^{t\alpha q} (b_i - \langle \mathbf{a}_i, \mathbf{s} \rangle - e) = 0$$

in order to solve the LWE instance, where $t \in \mathbb{R}$ determines the adversarial success probability. In this way, the noisy linear equation becomes a polynomial equation of degree $2t\alpha q + 1$. If the linear equation $\langle \mathbf{a}, \mathbf{s} \rangle$ is replaced by a polynomial $F(\mathbf{a}, \mathbf{s})$ of degree d, the Arora-Ge equation becomes of degree $d(2t\alpha q + 1)$.

We choose the discrete Gaussian distribution for sampling noise since the cryptanalysis of LWE has been extensively studied under the discrete Gaussian assumption. In the main body of the stream cipher, we use building blocks from HERA (Section 3.2) and Pasta [43]. For linear layers and the key schedule, we follow the style of HERA. Although we are aware of generic ways of constructing an MDS matrix [60, 91], those approaches result in a matrix with large components. We keep the component of matrices $\mathbf{M}_v$ as small as possible for efficiency reason. When enlarging the block size n, we computationally find $v \times v$ MDS matrices since we cannot keep the original linear layer of HERA.

When we are designing HERA, we claimed that a nonlinear layer whose inverse is of a high degree mitigates algebraic MitM attacks. As there has not been any known quadratic function with the inverse of a high degree over $\mathbb{Z}_q$, a cubic S-box has been used in HERA, which leads to a larger multiplicative depth. After truncation was proposed for an alternative countermeasure for an algebraic MitM attack [42], Dobraunig et al. [43] proposed a Feistel structure for HE-friendly ciphers. Since the Feistel structure is vulnerable to algebraic MitM attacks, a cubic function for the last nonlinear layer and truncation are adopted to Pasta. As we thought that deploying both the cubic function and truncation is superfluous,

we conclude that truncation without the cubic function is sufficient for Rubato.

3.4.3 Parameter Selection

In this section, we recommend some sets of parameters and concrete instantiation of Rubato. Some sets of parameters are selected in Table 3.7. The notations in the table follow those in Section 3.4.1. We give three types of parameters: S, M, and L. These imply the size of blocks.

Parameter	λ	n	ℓ	$\lceil \log q \rceil$	αq	r
Par-80S	80	16	12	26	11.1	2
Par-80M	80	36	32	25	2.7	2
Par-80L	80	64	60	25	1.6	2
Par-128S	128	16	12	26	10.5	5
Par-128M	128	36	32	25	4.1	3
Par-128L	128	64	60	25	4.1	2

Table 3.7: Selected parameters of Rubato.

When choosing the modulus q, we consider the effect of noise on precision. For a discrete Gaussian distribution $D_{\alpha q}$, the size of noise is expectedly $\mathbb{E}_{e \leftarrow D_{\alpha q}}[\,|e|\,]$. Suppose we obtain p-bit average precision while using the RtF framework with some deterministic cipher (e.g., HERA). It means that, a given message x and the message after transciphering x', $|x - x'| < 1/2^p$. Then, the expected loss of precision bits is upper bounded by

$$p + \log_2 \left[\mathbb{E}_{e \leftarrow D_{\alpha q}}[1/2^p + e/\Delta]\right].$$

In our instantiation, we enlarge the modulus q to compensate this loss of precision.

The choice of the scaling factor Δ should vary along with the $\|\mathbf{m}\|_1$, where $\mathbf{m} \in \mathbb{R}^*$ is a message vector. In our experiment (see Section 3.6), we constrain $\|\mathbf{m}\|_1 \leq 1$ and choose $\Delta = q/16$ for the RtF framework. If someone manipulates a message $\|\mathbf{m}\|_1 \leq s$, it is appropriate to choose $\Delta = q/(16 \cdot s)$.

3.5 Security Analysis of Rubato

In this section, we provide the security analysis of Rubato. We summarize the analysis result in Table 3.8. We omit too costly attacks (i.e., time complexity larger than 2^{1000} for all the parameters) such as trivial linearization and interpolation attacks. In Appendix A.2, we give some additional plots on the security analysis.

Assumptions and the Scope of Analysis. In this work, we will consider the standard "secret-key model", where an adversary arbitrarily chooses a nonce and obtains the corresponding keystream without any information on the secret key. The related-key and the known-key models are beyond the scope of this paper.

Since Rubato takes as input counters, an adversary is not able to control the differences of the inputs. Nonces can be adversarially chosen, while they are also fed to the extendable output function, which is modeled as a random oracle. So one cannot control the difference of the internal variables. For this reason, we believe that our construction is secure against any type of chosen-plaintext attacks including (higher-order) differential, truncated differential, invariant subspace trail, and cube attacks. A recent generalization of an integral attack [19] requires only a small number of chosen plaintexts, while it is not applicable to Rubato within the birthday bound.

KAIST

Parameter	GCD	Gröbner	LC	Lattice	Arora-Ge
Par-80S	4393.6	80.04	155.9	760.48	80.04
Par-80M	878.6	84.55	249.9	↑	80.37
Par-80L	↑	82.73	349.8	↑	82.73
Par-128S	411.9	128.07	311.7	↑	128.1
Par-128M	880.7	128.1	249.9	↑	128.1
Par-128L	↑	169.6	349.8	↑	129.6

Table 3.8: The log of the complexity of the attacks on Rubato. The upward sign (↑) implies that the complexity is larger than 2^{1000}. The linear algebra constant ω is assumed to be 2.

3.5.1 Cryptanalysis Based on Symmetric Primitive Analysis

As the Gaussian noise is added at the end of the cipher, most of the symmetric key cryptanalysis is not directly applicable to Rubato. Nevertheless, by guessing all the noise, an attacker can try to analyze Rubato using symmetric key cryptanalysis. Since the noise is sampled from discrete Gaussian distribution $D_{\alpha q}$, it is always advantageous for an attacker to guess that the noise is zero when the data are sufficiently given. We denote the probability such that a sample from $D_{\alpha q}$ is zero by $\varepsilon_0 = \Pr[e \leftarrow D_{\alpha q} : e = 0]$.

3.5.1.1 On the Number of Monomials in Rubato

The round function of Rubato is defined by

$$\mathsf{RF}[\mathbf{k}, \mathsf{nc}, i] = \mathsf{ARK}[\mathbf{k}, \mathsf{nc}, i] \circ \mathsf{Feistel} \circ \mathsf{MixRows} \circ \mathsf{MixColumns},$$

KAIST

where the two linear maps MixColumns and MixRows can be represented by $n \times n$-matrices over $\mathbb{Z}_q$. Their product represents MixRows $\circ$ MixColumns as follows. Similar to HERA, we check that the matrix representation of MixRows $\circ$ MixColumns has no zero entry. It implies that MixRows $\circ$ MixColumns contains all the linear monomials in its polynomial representation, and hence RF contains all the quadratic monomials. More precisely, if $a_i \neq 0$ for $i = 1, \ldots, n$, then we have

$$\begin{aligned}(a_1x_1 + a_2x_2 + \cdots + a_nx_n + b)^2 &= \sum_{i,j} a_ia_jx_ix_j + \sum_i 2a_ibx_i + b^2 \\ &= \sum_{i \leq j} \alpha(i,j)a_ia_jx_ix_j + \sum_i 2a_ibx_i + b^2,\end{aligned}$$

where

$$\alpha(i,j) = \begin{cases} 1 & \text{if either } i = j; \\ 2 & \text{if } i < j. \end{cases}$$

Since the plaintext modulus q is prime and $q > 2^{20}$, every quadratic monomial has a nonzero coefficient.

We can estimate the number of monomials in Rubato with more rounds. Let $\mathbf{b} = (b_1, \ldots, b_n)$ be the output of the first round function. The second round function will contain all the quadratic monomials in $\mathbf{b}$. When we view the second round function as a polynomial in $b_1, \ldots, b_n$, some coefficients might become zero, while this happens only with a probability of $1/q$. Heuristically (with the independence assumption), each monomial will remain at the second round with probability $1 - (1/q)^n$. This heuristic is confirmed by our computation, showing all possible monomials at the end of the second round. We conjecture that this property will hold for more than two rounds.

3.5.1.2 Trivial Linearization

Trivial linearization is to solve a system of linear equations by replacing all monomials with new variables. When applied to the r-round Rubato cipher, the number of monomials appearing in this system is upper bounded by

$$S = \sum_{i=0}^{2^r} \binom{n+i-1}{i}.$$

Therefore, at most S equations will be enough to solve this system of equations. All the monomials of degree at most 2^r are expected to appear after r rounds of Rubato (as explained in detail in Section 3.5.1.1). Therefore, by guessing $e = 0$, we can conclude that this attack requires $O(S^\omega/\varepsilon_0^S)$ time, where $2 \leq \omega \leq 3$. Since the success probability is too small for $r \geq 1$, it will never be a dominant attack.

3.5.1.3 GCD Attack

The GCD attack seeks to compute the greatest common divisor (GCD) of univariate polynomials, and it can be useful for a cipher operating on a large field with its representation being a polynomial in a single variable. This attack can be extended to a system of multivariate polynomial equations by guessing all the key variables except one. For r-round Rubato, the complexity of the GCD attack is estimated as $O(q^{n-1}r^2 2^r)$ even if there is no noise. For a security parameter $\lambda \leq 256$, Rubato will be secure against the GCD attack even with a single round as long as $n \geq 16$.

3.5.1.4 Gröbner Basis Attack

KAIST

For the Gröbner basis attack, re-arranging equations may lead to a significant impact on the attack complexity. For example, one may set a system of equations using only plaintext-ciphertext pairs or set a system of equations with new variables standing for internal states. The former will be a higher-degree system in fewer variables, while the latter will be a lower-degree system in more variables.

From a set of nonce-plaintext-ciphertext triples $\{(\mathsf{nc}_i, \mathbf{m}_i, \mathbf{c}_i)\}_i$, an attacker will be able to establish an overdetermined system of equation

$$f_1(k_1, \ldots, k_n) = f_2(k_1, \ldots, k_n) = \cdots = f_m(k_1, \ldots, k_n) = 0$$

where $k_i \in \mathbb{Z}_q$ is the i-th component of the key variable. The degree of regularity of the system is computed as the degree of the first non-positive coefficient in

$$\left(1 - z^{2^r}\right)^{m-n} \left(\sum_{i=0}^{2^r-1} z^i\right)^n$$

where r is the number of rounds. The larger number of equations implies a smaller degree of regularity. Since the summation does not have any negative term, one easily sees that the degree d_{reg} of regularity cannot be smaller than 2^r. We conservatively lower bound the time complexity when there is no noise by

$$O\left(\binom{n+2^r}{2^r}^\omega\right)$$

regardless of the number of equations. Since at least n equations are required for the unique root, we

can conclude that this attack requires n data and

$$O\left(\binom{n+2^r}{2^r}^{\omega}\varepsilon_0^{-n}\right)$$

time. We note that the hybrid approach always has worse complexity.

Instead of a system of equations of degree 2^r, one can establish a system of $((r-1)n+\ell)k$ quadratic equations in $n(r-1)k+n$ variables, where k is the block length of each query. To get the unique root, it requires that $k \geq n/\ell$. Then, the complexity is

$$O\left(\binom{n(r-1)k+n+d_{reg}(r,k)}{d_{reg}(r,k)}^{\omega}\varepsilon_0^{-\ell k}\right)$$

where the degree $d_{reg}(r,k)$ of regularity is computed under the semi-regular assumption.

Although we explain that the truncation can prevent MitM attack, MitM attack is not a "never-applicable" attack for Rubato. Suppose $\mathbf{y} = (y_1, \ldots, y_\ell)$ be a keystream. By creating new variables $x_{\ell+1}, \ldots, x_n$, an attacker can make n MitM equations in $2n-\ell$ variables $k_1, \ldots, k_n, x_{\ell+1}, \ldots, x_n$. Denoting the first $\lfloor r/2 \rfloor$-round function by F and the last $\lceil r/2 \rceil$-round function except for $\mathsf{Tr}_{n,\ell}$ and AGN by G,

$$\begin{aligned}
&\mathsf{Rubato}[\mathbf{k}, \mathsf{nc}_1] = (y_1, \ldots, y_\ell) \\
&G \circ F[\mathbf{k}, \mathsf{nc}_1] = (y_1 - e_1, \ldots, y_\ell - e_\ell, x_{\ell+1}, \ldots, x_n) \\
&F[\mathbf{k}, \mathsf{nc}_1] = G^{-1}(y_1 - e_1, \ldots, y_\ell - e_\ell, x_{\ell+1}, \ldots, x_n) \qquad (3.2)
\end{aligned}$$

where e_i's are guessed noise. Equation 3.2 is of degree $2^{\lceil r/2 \rceil}$ so that the lower bound of the degree of regularity is also $2^{\lceil r/2 \rceil}$. Similarly as above, to get the unique root, the queried block length k should satisfy $n + k(n-\ell) \leq nk$. Then, the time complexity is lower bounded by

$$O\left(\binom{n+(n-\ell)k+2^{\lceil r/2 \rceil}}{2^{\lceil r/2 \rceil}}^{\omega}\varepsilon_0^{-\ell k}\right).$$

3.5.1.5 Interpolation Attack

For the r-round Rubato cipher, let $\mathbf{rc} = (\mathbf{rc}_0, \ldots, \mathbf{rc}_r) \in (\mathbb{Z}_q^n)^{r+1}$ be a sequence of the outputs from XOF. For $i = 0, \ldots, r$, $\mathbf{rc}_i$ is evaluated by a polynomial of degree 2^{r-i}. As we expect that the r-round Rubato cipher has almost all monomials of degree $\leq 2^r$ in its polynomial representation, the number of monomials is lower bounded by

$$\sum_{j=0}^{r}\sum_{i=0}^{2^j}\binom{n+i-1}{i}.$$

Similarly as the trivial linearization, the success probability is too small for $r \geq 1$, it will never be a dominant attack.

3.5.1.6 Linear Cryptanalysis

Although it seems that the linear cryptanalysis cannot be applied to Rubato directly because of the noise, we give a security bound for linear cryptanalysis assuming no noise. There are two applications of linear cryptanalysis on Rubato according to how to take the input variables: the XOF output variables or the key variables. In the first case, unlike traditional linear cryptanalysis, the probability of any linear trail

of Rubato depends on the key since it is multiplied by the input. It seems infeasible to make a plausible linear trail without any information on the key material.

In the second case, the attack is reduced to solving an LWE-like problem as follows; given pairs $(\mathsf{nc}_i, \mathbf{y}_i)$ such that $\mathsf{Rubato}(\mathbf{k}, \mathsf{nc}_i) = \mathbf{y}_i$, one can establish

$$\langle \mathbf{b}, \mathbf{y}_i \rangle = \langle \mathbf{a}, \mathbf{k} \rangle + e_i$$

for some vectors $\mathbf{a} \neq \mathbf{0}, \mathbf{b} \in \mathbb{Z}_q^n$ and error e_i sampled according to a certain distribution χ. An attacker requires $1/\operatorname{ELP}^{\mathsf{E}}(\mathbf{a}, \mathbf{b})$ samples to distinguish χ from the uniform distribution [13].

Lemma 3.5.1. For any $\mathbf{a} = (a_1, \ldots, a_n) \neq \mathbf{0}, \mathbf{b} = (b_1, \ldots, b_n) \in \mathbb{Z}_q^n$ such that $\mathsf{hw}(b_2, b_3, \ldots, b_n) = h$, the linear probability of Feistel is

$$\operatorname{LP}^{\mathsf{Feistel}}(\mathbf{a}, \mathbf{b}) \leq \frac{1}{q^h}.$$

Proof. By the definition, we have

$$\begin{aligned}
&\operatorname{LP}^{\mathsf{Feistel}}(\mathbf{a}, \mathbf{b}) \\
&= \left| \mathbb{E}_{\mathbf{m}} \left[\exp\left\{ \frac{2\pi i}{q} \left(-\langle \mathbf{a}, \mathbf{m} \rangle + \langle \mathbf{b}, \mathsf{Feistel}(\mathbf{m}) \rangle \right) \right\} \right] \right|^2 \\
&= \left| \mathbb{E}_{\mathbf{m}} \left[\exp\left\{ \frac{2\pi i}{q} \left(\sum_{k=1}^{n-1} (-a_k m_k + b_k m_k + b_{k+1} m_k^2) + (-a_n + b_n) m_n \right) \right\} \right] \right|^2 \\
&= \left| \mathbb{E}_{m_n} \left[\exp\left\{ \frac{2\pi i}{q} \left((-a_n + b_n) m_n \right) \right\} \right] \right|^2 \times \prod_{i=1}^{n-1} \left| \mathbb{E}_{m_i} \left[\exp\left\{ \frac{2\pi i}{q} \left((-a_i + b_i) m_i + b_{i+1} m_i^2 \right) \right\} \right] \right|^2.
\end{aligned}$$

Carlitz and Uchiyama [28] proved that

$$\left| \sum_{x=0}^{q-1} \exp\left(\frac{2\pi i}{q} \cdot p(x) \right) \right| \leq (r-1)\sqrt{q}$$

for any polynomial $p(x)$ of degree r over $\mathbb{Z}_q$. Therefore, we have

$$\left| \mathbb{E}_{m_i} \left[\exp\left\{ \frac{2\pi i}{q} \left((-a_i + b_i) m_i + b_{i+1} m_i^2 \right) \right\} \right] \right|^2 \leq \left| \frac{1}{q} \cdot \sqrt{q} \right|^2 \leq \frac{1}{q}$$

and it implies that

$$\operatorname{LP}^{\mathsf{Feistel}}(\mathbf{a}, \mathbf{b}) \leq \frac{1}{q^h}.$$

□

Since the branch number of the linear layer of Rubato is $2v$ (as shown in Appendix A.1), we can conclude that an r-round Rubato cipher provides λ-bit security against linear cryptanalysis when $q^{(2v-2)\cdot\lfloor \frac{r}{2} \rfloor} > 2^\lambda$.

3.5.1.7 Differential Cryptanalysis and Its Variants

Since Rubato takes counters as input, an adversary is not able to control the differences of its inputs. Nonces can be adversarially chosen, while they are also fed to the extendable output function, which is modeled as a random oracle. So one cannot control the difference of the internal variables. For

this reason, we believe that our construction will be secure against any type of chosen-plaintext attack including (higher-order) differential, truncated differential, invariant subspace trail, and cube attacks.

Nonetheless, to prevent an unsuspected differential-related attack, we present a computation of a differential characteristic in the following. Given a pair $\mathbf{a}, \mathbf{b} \in \mathbb{Z}_q^n$, the differential probability of Feistel is defined by

$$\mathrm{DP}^{\mathsf{Feistel}}(\mathbf{a}, \mathbf{b}) = \frac{1}{q^n} \cdot \left| \{\mathbf{x} \in \mathbb{Z}_q^n : \mathsf{Feistel}(\mathbf{x} + \mathbf{a}) - \mathsf{Feistel}(\mathbf{x}) = \mathbf{b}\} \right|.$$

So $\mathrm{DP}^{\mathsf{Feistel}}(\mathbf{a}, \mathbf{b})$ is determined by the number of solutions to $\mathsf{Feistel}(\mathbf{x} + \mathbf{a}) - \mathsf{Feistel}(\mathbf{x}) = \mathbf{b}$.

Lemma 3.5.2. For any $\mathbf{a} = (a_1, \ldots, a_n) \neq \mathbf{0}, \mathbf{b} = (b_1, \ldots, b_n) \in \mathbb{Z}_q^n$ such that $\mathsf{hw}(a_1, a_2, \ldots, a_{n-1}) = h$, the differential probability of Feistel is

$$\mathrm{DP}^{\mathsf{Feistel}}(\mathbf{a}, \mathbf{b}) \leq \frac{1}{q^h}.$$

Proof. Our goal is to find the maximum number of solutions to the equation

$$\mathsf{Feistel}(\mathbf{x} + \mathbf{a}) - \mathsf{Feistel}(\mathbf{x}) = (a_1, 2a_1x_1 + a_1^2 + a_2, \ldots, 2a_nx_{n-1} + a_{n-1}^2 + a_n) = \mathbf{b}.$$

For $i \leq n-1$, the equation $2a_ix_i + a_i^2 + a_{i+1} = b_i$ has a unique solution $x_i = (b_i - a_i^2 - a_{i+1}) \cdot (2a_i)^{-1}$ if $a_i \neq 0$ and the equation has maximally q solutions if $a_i = 0$. For $i = n$, the variable x_n is free so that the maximal number of solution is q. It implies that

$$\mathrm{DP}^{\mathsf{Feistel}}(\mathbf{a}, \mathbf{b}) \leq \frac{1}{q^h}.$$

KAIST

□

Since the branch number of the linear layer of Rubato is $2v$ (as shown in Appendix A.1), we can conclude that an r-round Rubato cipher provides λ-bit security against differential cryptanalysis when $q^{(2v-2)\cdot\lfloor \frac{r}{2} \rfloor} > 2^\lambda$.

3.5.2 Cryptanalysis Based on LWE Analysis

As Rubato is not an LWE instance, algorithms solving LWE are not directly applied to Rubato. However, if someone considers a single component of a keystream block of Rubato as

$$\left((a_{\mathbf{u}})_{\mathbf{u}}, \sum_{\mathbf{u}} a_{\mathbf{u}} \mathbf{k}^{\mathbf{u}} + e \right)$$

where $\mathbf{u} = (u_1, \ldots, u_n) \in \mathbb{Z}_{\geq 0}^n$, and $\mathbf{k}^{\mathbf{u}} = \prod_i k_i^{u_i}$ implies a monomial with degree $\mathbf{u}$, it becomes an LWE instance with the linearized variables whose dimension is

$$S_{n,r} = \sum_{i=1}^{2^r} \binom{n+i-1}{i}$$

where $2^r < q$.

In this section, we will denote notations in a linearized way. For example, we denote Rubato samples by $(\mathbf{A}, \mathbf{c} = \mathbf{As} + \mathbf{e})$ where $\mathbf{s}$ stands for the vector $(\mathbf{k}^{\mathbf{u}})_{\mathbf{u}}$ and $\mathbf{A}$ stands for a set of $(a_{\mathbf{u}})_{\mathbf{u}}$ in a certain monomial order.

3.5.2.1 Exhaustive Search

The most naive approach for solving LWE is the exhaustive search. Given m samples $(\mathbf{A}, \mathbf{c})$, an attacker guesses noise $\mathbf{e} = (e_1, \ldots, e_m)$ and finds $\mathbf{s}$ satisfying $\mathbf{As} = \mathbf{c} - \mathbf{e}$ where $\mathbf{A}$ is required to have a left inverse. To attack Rubato, the attacker needs to guess at least $(2t\alpha q+1)^{S_{n,r}}$ times for success probability $\Pr[e \leftarrow D_{\alpha q} : |e_i| \leq t\alpha q \text{ for all } i]$ where the expected time complexity is upper bounded by $\varepsilon_0^{S_{n,r}}$. Since the success probability is too small for $r \geq 1$, it will never be a dominant attack.

There is a meet-in-the-middle (MitM) approach mentioned in [12], which is a time-memory trade-off of the exhaustive search. For the same reason, the MitM approach cannot be a dominant attack.

3.5.2.2 Lattice Attacks

Reduction to lattice problems is one way to solve LWE. To solve the lattice problems, an attacker needs a short enough basis of the given lattice. This short basis is obtained by using a lattice reduction algorithm such as the BKZ algorithm [95, 32].

Core-SVP Hardness of BKZ Algorithms. The BKZ algorithm is a lattice reduction algorithm that uses an (approximate-)SVP oracle of small dimension β. This algorithm repeatedly calls the SVP oracle as a subroutine to find the shortest vectors in the projected lattice of dimension β. We call an output from the BKZ-β algorithm "BKZ$_\beta$-reduced basis". There are sieving algorithms and enumeration algorithms for instantiations of the SVP oracle.

Unfortunately, it is hard to predict the number of calls to the SVP oracle in the BKZ algorithm, which is related to exact complexity analysis. So, we analyze the security of Rubato only considering a single call, which is called *core-SVP hardness* [8]. We arrange the expected time complexity of SVP algorithms in terms of BKZ block size β and root-Hermite factor δ in Table 3.9. In Table 3.9, there are two types of models. Above the dashed horizontal line, two models are extrapolated results (in seconds) of the experimental running time of the BKZ algorithm. Below the dashed line, the complexity of the three models is for a single call of the oracle. When estimating the security of Rubato against lattice attacks, we neglect the $o(\beta)$ terms.

For estimating the quality of a BKZ$_\beta$-reduced basis, Chen [31] gave a limit

$$\lim_{N\to\infty} \delta = \left(v_\beta^{-\frac{1}{\beta}}\right)^{\frac{1}{\beta-1}} \approx \left(\frac{\beta}{2\pi e}(\pi\beta)^{\frac{1}{\beta}}\right)^{\frac{1}{2(\beta-1)}} \tag{3.3}$$

of the root-Hermite factor δ assuming the Gaussian heuristic and the geometric series assumption. Chen also gave experimental proof that this limit is a reasonable choice when N is finite. As another estimate of δ for a BKZ$_\beta$-reduced basis, the *lattice rule of thumb* [90], which says $\delta = \beta^{\frac{1}{2\beta}}$, is often used in the literature. When we need to compute an appropriate choice of β from required δ, we choose Chen's limit.

Primal Attack. Primal attack is the strategy of solving the search-LWE problem via solving the bounded distance decoding (BDD) problem. Given m samples $(\mathbf{A}, \mathbf{c} = \mathbf{As} + \mathbf{e})$ following $L_{\mathbf{s},\chi}$, one can see that $\mathbf{c}$ is near the lattice $L(\mathbf{A})$. Finding the nearest lattice point from $\mathbf{c}$ is equivalent to finding the secret vector $\mathbf{s}$ when $\mathbf{A}$ is (left) invertible. If $\mathbf{A}$ is not invertible, it is sufficient to gather a few more samples.

In order to solve the derived BDD problem, there are two approaches: enumeration [77, 52] and reduction to unique-SVP (uSVP) [3, 7]. Since the enumeration method is treated as a subroutine in the BKZ algorithm, we do not take it into account as a direct solver of the BDD problem.

Model	Log of Complexity
Lindner and Peikert [77]	$\frac{1.8}{\log \delta} - 110$
Albrecht et al. [6]	$\frac{0.009}{\log^2 \delta} - 27$
Enumeration [5]	$\frac{\beta \log \beta}{8} - 0.654\beta + 25.84$
Classical Sieve [16]	$0.292\beta + o(\beta)$
Quantum Sieve [74]	$0.265\beta + o(\beta)$

Table 3.9: The expected time complexity of the BKZ algorithm in terms of BKZ block size β and root-Hermite factor δ.

The second approach, the reduction to uSVP, was firstly proposed by Albrecht et al. [3]. The main idea of the approach is to solve SVP of the larger lattice $L = L(\mathbf{B})$ of basis

$$\mathbf{B} = \begin{pmatrix} \mathbf{A} & \mathbf{c} \\ \mathbf{0} & u \end{pmatrix}$$

where $u = \mathrm{dist}(\mathbf{c}, L(\mathbf{A}))$. This lattice contains an unusually small vector $(\mathbf{e}, -u)$, which implies the gap $\lambda_2(L)/\lambda_1(L)$ is large. Assuming Gaussian heuristic and linear independence of $\mathbf{A}$, Göpfert [57] showed that an attacker can create the $\lambda_2(L)/\lambda_1(L)$-gap greater than

$$\frac{\min\left\{q, q^{1-N/m}\sqrt{\frac{m}{2\pi e}}\right\}}{\sqrt{m} \cdot \frac{\alpha q}{\sqrt{2\pi}}}.$$

As a lattice reduction satisfying $\lambda_2(L)/\lambda_1(L) > \tau\delta^m$ for some constant $\tau \leq 1$ is sufficient to solve a uSVP instance [51], this approach requires log root-Hermite factor

$$\log \delta = \frac{\log^2(\tau\alpha\sqrt{e})}{4N \log q}$$

if $\min\left\{q, q^{1-N/m}\sqrt{\frac{m}{2\pi e}}\right\} = q^{1-N/m}\sqrt{\frac{m}{2\pi e}}$. Although experimental evidence suggests $\tau \leq 0.4$ [51], we set $\tau = 1$ for the conservative choice of parameters.

Alkim et al. gave an alternative success condition of the attack [8]. Denoting $d = m + N + 1$ and $\sigma = \alpha q/\sqrt{2\pi}$, the requirement is that

$$\sigma\sqrt{\beta} \leq \delta^{2\beta - d} q^{m/d}$$

where δ is computed by Equation 3.3. We take both into account along with the parameter $N = S_{n,r}$.

Dual Attack. The dual attack, also called the short integer solution (SIS) strategy, is an attack finding small vector $\mathbf{w} \in \mathbb{Z}_q^m$ such that $\mathbf{w}^\intercal \mathbf{A} \equiv 0 \pmod q$. Given m samples $(\mathbf{A}, \mathbf{c})$ from $L_{\mathbf{s},\chi}$, finding a short vector satisfying $\mathbf{w}^\intercal \mathbf{A} \equiv \mathbf{0} \pmod q$ leads to

$$\mathbf{w}^\intercal \mathbf{c} = \mathbf{w}^\intercal(\mathbf{A}\mathbf{s} + \mathbf{e}) = \mathbf{w}^\intercal \mathbf{e}$$

where the last term is small. The short vector $\mathbf{w}$ should satisfy

$$\|\mathbf{w}\|_2 = \frac{1}{\alpha}\sqrt{\frac{\ln(1/\epsilon)}{\pi}}$$

in order to distinguish $L_{\mathbf{s},\chi}$ from random with advantage ϵ [77]. By the definition of the root-Hermite factor δ, the attack requires that

$$\log \delta = \frac{\log^2 \left(\frac{1}{\alpha} \sqrt{\frac{\ln(1/\epsilon)}{\pi}} \right)}{4N \log q}$$

for the LWE instance parametrized by N, α, and q. When evaluating the security of Rubato, we set $N = S_{n,r}$ and $\epsilon \approx 1/23$.

3.5.2.3 BKW Attack

The original BKW algorithm was proposed for solving the learning parity with noise (LPN) problem by Blum, Kalai, and Wasserman [21]. Regev pointed out that the BKW algorithm can be used for solving LWE [93], and Albrecht et al. gave the formal analysis of the BKW algorithm for LWE [6]. We briefly explain the BKW attack on LWE, and we refer to [6] for more details.

The BKW attack is a lattice-version of Gaussian elimination parametrized by a and b. Suppose there is an LWE distribution $L_{\mathbf{s},\chi}$ where $\chi = D_{\alpha q}$ is parametrized by the dimension N and the modulus q. Given enough samples $(\mathbf{A}, \mathbf{c})$ from $L_{\mathbf{s},\chi}$, the BKW attack first reduces $\mathbf{A}$ to a kind of block diagonal matrix. The width of the block is b and there will be an $a = \lceil N/b \rceil$ blocks.

As samples from $L_{\mathbf{s},\chi}$ intrinsically include noise, too many additions or subtractions between samples result in a useless equation. Instead, the attacker gathers $(q^b - 1)/2$ samples having all nonzero values up to sign in the first block. Denote this table of samples by T^1. If $L_{\mathbf{s},\chi}$ outputs a sample $(\mathbf{a}_i, c_i)$ which has the same first block in T^1, the attacker makes $(\mathbf{a}_i, c_i)$ to have all-zero first block by adding/subtracting the sample in T^1. Similarly, the attacker can gather $(q^b - 1)/2$ samples having all-zero first block and all nonzero values up to sign in the second block. Denote this table of samples by T^2. By repeating the same process, the attacker can construct a matrix of the form

$$\begin{pmatrix}
T^1 & & & & & & & & & & \\
0 & \dots & 0 & T^2 & & & & & & & \\
0 & \dots & 0 & 0 & \dots & 0 & T^3 & & & & \\
\vdots & \ddots & \vdots & \vdots & \ddots & \vdots & & & & & \\
0 & \dots & 0 & 0 & \dots & 0 & 0 & \dots & 0 & T^{a-1} & \\
0 & \dots & 0 & 0 & \dots & 0 & 0 & \dots & 0 & 0 & T^a \\
0 & \dots & 0 & 0 & \dots & 0 & 0 & \dots & 0 & \dots & 0 & M
\end{pmatrix}.$$

After solving a lattice problem with respect to M, the attacker can perform back substitution for the remaining parts.

The time complexity of this attack to solve search-LWE is

$$\left(\frac{q^b - 1}{2} \right) \cdot \left(\frac{a(a-1)}{s} \cdot (N+1) \right) + \left\lceil \frac{q^b}{2} \right\rceil \cdot \left(\left\lceil \frac{N}{d} \right\rceil + 1 \right) \cdot d \cdot a + \mathrm{poly}(N)$$

where $d = N - \lfloor N/b \rfloor$. The parameter a should satisfy $a \leq \log(\alpha^{-2})$ in order to distinguish $L_{\mathbf{s},\chi}$ from random [90]. We compute the concrete complexity by using $a = \log(\alpha^{-2})$ without the polynomial terms. As $N = S_{n,r}$ for Rubato, the complexity of the attack is at least $2^{S_{n,r} \log q / (-2 \log \alpha)}$.

3.5.2.4 Arora-Ge Attack

Arora and Ge proposed an algebraic algorithm to solve the search-LWE problem [10]. The main idea of this attack is that, given LWE samples $\{(\mathbf{a}_i, b_i)\}_i$, the errors fall into some interval $[-t\alpha q, t\alpha q]$ for some large enough t so that the equations

$$\prod_{e=-t\alpha q}^{t\alpha q} (b_i - \langle \mathbf{a}_i, \mathbf{s} \rangle - e) = 0$$

holds. Although the complexity of this attack for LWE is well-organized in [2], we independently describe the lower bound of complexity as the equations are different from LWE.

When guessing noise, an attacker may control the range of guesses to minimize the cost of attacks since the noise is not uniformly distributed. We denote the probability such that a sample from discrete Gaussian $D_{\alpha q}$ lies in the interval $[-t\alpha q, t\alpha q]$ by

$$\varepsilon_t = \Pr\left[e \leftarrow D_{\alpha q} : |e| \leq t\alpha q\right].$$

Since this probability determines the complexity of many attacks, we find the minimum of the complexity among $\{t : -3\sqrt{2\pi} \leq t \leq 3\sqrt{2\pi}\}$ which is equivalent to the range of 6 times the standard deviation.

Let $\mathsf{E}_i(\cdot)$ denote the i-th component of the Rubato cipher without noise. Then, by using Arora-Ge attack, an attacker can make a set of equations as follows. Given m nonce-plaintext-ciphertext triples $\{(\mathsf{nc}_j, \mathbf{m}_j, \mathbf{c}_j)\}_j$,

$$\left\{ \prod_{e_{i,j}=-t\alpha q}^{t\alpha q} (\mathbf{c}_{j,i} - \mathsf{E}_i(\mathsf{nc}_j, \mathbf{m}_j) - e_{i,j}) = 0 \right\}_{\substack{1 \leq i \leq \ell \\ 1 \leq j \leq m}} \tag{3.4}$$

KAIST

where $\mathbf{c}_{j,i}$ is the i-th component of $\mathbf{c}_j$ and $e_{i,j}$ are independently sampled from discrete Gaussian distribution. These equations are for the key variable $\mathbf{k}$ of total degree $2^r(2t\alpha q + 1)$.

Now, we give a lower bound of the complexity of solving Equation 3.4 by using Gröbner basis attack. As discussed in Section 3.5.1.4, we can conservatively assume the degree of regularity of Equation 3.4 is $2^r(2t\alpha q+1)$ regardless of the number of nonce-plaintext-ciphertext triples. We have the time complexity at least

$$O\left(\binom{n + 2^r(2t\alpha q + 1)}{2^r(2t\alpha q + 1)}^{\omega} \varepsilon_t^{-c}\right)$$

where c should be larger than or equal to n to get the unique root. This complexity formula also lower bounds the trivial linearization approach to solving Equation 3.4.

Meet-in-the-middle Approach. Similar to most of the algebraic attacks, one can try to use the MitM approach for Arora-Ge attack. From Equation 3.2, the attacker can build the Arora-Ge equations as follows.

$$\left\{ \prod_{(e_{i,j})_i \in C_j} \left(F_i[\mathbf{k}, \mathsf{nc}_j] - (G^{-1})_i(y_1 - e_{1,j}, \ldots, y_\ell - e_{\ell,j}, x_{\ell+1}, \ldots, x_n)\right) = 0 \right\}_{1 \leq j \leq m}$$

where

$$C_j \coloneqq \left\{(e_1, \ldots, e_\ell) \in \mathbb{Z}^\ell : -t\alpha q \leq e_i \leq t\alpha q \text{ for all } i\right\},$$

F_i and $(G^{-1})_i$ are the i-th components of F and G^{-1} respectively. These equations are of degree $2^{\lceil r/2 \rceil}(2t\alpha q + 1)^\ell$ so that the lower bound of the degree of regularity also is $2^{\lceil r/2 \rceil}(2t\alpha q + 1)^\ell$. Similarly

as above, to get the unique root, the queried block length k should satisfy $n + k(n - \ell) \leq nk$. Then, the time complexity is lower bounded by

$$O\left(\binom{n + (n-\ell)k + 2^{\lceil r/2 \rceil}(2t\alpha q + 1)^{\ell}}{2^{\lceil r/2 \rceil}(2t\alpha q + 1)^{\ell}}^{\omega} \varepsilon_t^{-\ell k}\right).$$

We give some plots of the complexity of the Arora-Ge attack according to the choice of t in Appendix A.2.

3.6 Performance Evaluation of the RtF Framework

In this section, we evaluate the performance of the RtF framework combined with HERA and Rubato distinguishing two different parts: the client-side and the server-side as separated in Figure 3.1. On the client-side, the latency includes time for generating pseudorandom numbers (needed to generate a single keystream), keystream generation from HERA, message scaling, rounding and vector addition over $\mathbb{Z}_q$. The extendable output function is instantiated with SHAKE256 in `XKCP`.

The server-side offline latency includes time for the randomized key schedule, homomorphic evaluation of the keystreams from the cipher, and $\mathsf{SlotToCoeff}^{\mathsf{FV}}$. The cipher is homomorphically evaluated by using row-wise packing. The online latency includes scaling up to FV-ciphertext space, the homomorphic subtraction, rescaling to the lowest level, and HalfBoot. We measure the latency until the first HE-ciphertext comes out, while the throughput is measured until all the n HE-ciphertexts come out. We note that our evaluation does not take into account key encryption since the encrypted key will be used over multiple sessions once it is computed. For the same reason, the initialization process of the HE schemes is not considered.

KAIST

Environment. The source codes of server-side computation are developed in Golang version 1.16.4 with `Lattigo` library[2] which implements RNS (residue number system) variants of the FV and CKKS schemes. The source codes of client-side computation are developed in C++17, using GNU C++ 7.5.0 compiler with AVX2 instruction set. For the instantiation of the XOF, we use AES in counter-mode as well as SHAKE256 in `openssl` library[3] and `XKCP` library[4], respectively. Our experiments are done in AMD Ryzen 7 2700X @ 3.70 GHz single-threaded with 64 GB memory.

3.6.1 Parameter Selection

The sets of parameters used in our implementation are given in Table 3.10, where

- λ is the security parameter;
- p is the number of precision bits of the RtF framework;
- L' is the ciphertext level at the end of the framework;
- q is the plaintext modulus;
- r is the number of rounds of the symmetric ciphers;
- N is the degree of the polynomial modulus in the HE schemes;

[2] https://github.com/ldsec/lattigo
[3] https://github.com/openssl/openssl
[4] https://github.com/XKCP/XKCP

- ℓ is the number of slots in the FV scheme in the RtF framework;
- QP is the largest ciphertext modulus of the HE schemes including special primes.

For the CKKS scheme, the message space is $\mathbb{C}^{\ell/2}$.

In Table 3.10, we recommend secure parameters of HERA when combined with the RtF framework. For the parameters related to bootstrapping, we follow the choice of bootstrapping parameters in [22]. Specifically,

- the hamming weight h of the secret key is 192;
- the range K of the sine evaluation is 25;
- the number R of the double angle formula is 2;
- the degree d_{sin} of the sine evaluation is 63;
- the degree d_{arcsin} of the inverse sine evaluation is 7, if necessary.

We also experiment the effect of the inverse sine evaluation [75]. The parameter names ending with `a` stands for the evaluation of the inverse sine function. The parameter sets ending with `s` stands for a small number of slots. It uses 16 slots in order to evaluate HERA. Parameter Q_0/Δ' is the ratio of the first ciphertext modulus Q_0 to the bootstrapping scaling factor Δ' which is introduced in Section 3.1.3. We use 128-bit secure HE parameters for all parameter sets. If an application requires more depth with 80-bit security, then a few dozens of level can be appended without raising N and degradation of security.

KAIST

Set	λ	SKE		HE					
		$\lceil \log q \rceil$	r	$\log N$	$\log \ell$	$\lceil \log QP \rceil$	Δ_{CKKS}	Q_0/Δ'	arcsin
`Par-80`	80	28	4	16	16	1533	2^{40}	512	✗
`Par-80a`	80	25	4	16	16	1533	2^{45}	16	✓
`Par-80s`	80	28	4	16	4	1533	2^{40}	512	✗
`Par-80as`	80	25	4	16	4	1533	2^{45}	16	✓
`Par-128`	128	28	5	16	16	1533	2^{40}	512	✗
`Par-128a`	128	25	5	16	16	1533	2^{45}	16	✓
`Par-128s`	128	28	5	16	4	1533	2^{40}	512	✗
`Par-128as`	128	25	5	16	4	1533	2^{45}	16	✓

Table 3.10: Selected sets of parameters used in our implementation. The rest of the bootstrapping parameters is set to be $(h, K, R, d_{\text{sin}}, d_{\text{arcsin}}) = (192, 25, 2, 63, 0/7)$.

3.6.2 Benchmark of HERA

We summarize implementation results of HERA in Table 3.11. This table includes ciphertext expansion ratio (CER), time-relevant measurements, precision, and homomorphic capacity. One can see that the parameters with arcsin (`Par-a`) offer smaller CER while the remaining levels are less than other parameters. On the other hand, the parameters with small slots (`Par-s`) take less time for evaluation since the complexity of evaluating $\mathsf{SlotToCoeff}^{\mathsf{FV}}$ and $\mathsf{CoeffToSlot}^{\mathsf{CKKS}}$ is affected by the number of slots.

Set	CER	Client-side		Server-side			p	$\log Q_{L'}$
		Lat. (ms)	Thrp. (MB/s)	Lat. Off (s)	Lat. On (s)	Thrp. (KB/s)		
Par-80	1.54	1.520	22.86	98.56	16.84	5.066	17.22	500
Par-80a	1.24	1.443	26.62	91.09	20.68	5.412	19.13	375
Par-80s	1.53	1.520	22.95	71.89	13.23	0.0019	17.29	500
Par-80as	1.23	1.443	26.77	68.31	14.14	0.0020	19.25	375
Par-128	1.54	1.599	21.73	128.7	19.00	4.738	17.22	500
Par-128a	1.24	1.520	25.26	120.7	20.88	5.077	19.13	375
Par-128s	1.54	1.599	21.72	89.62	13.34	0.0018	17.21	500
Par-128as	1.23	1.520	25.26	84.02	14.31	0.0019	19.35	375

Table 3.11: Performance of the RtF transciphering framework with HERA.

3.6.3 Benchmark of Rubato

We summarize our implementation results of Rubato in Table 3.12 and Table 3.13. For the HE parameters, we use the RtF parameter Par-128a in Table 3.10, which uses the arcsin function. In Table 3.12, the client-side latency and throughput for each instantiation of the XOF are given. Table 3.13 includes ciphertext expansion ratio (CER), time-related measurements, and precision. One can see that a larger parameter implies higher throughput at the cost of higher latency on both sides. As Rubato needs a substantial amount of random bits, the client-side performance is significantly influenced by the choice of XOF. On the server-side, we note that Rubato only affects on the offline latency while the online latency is affected by the efficiency of CKKS bootstrapping.

KAIST

Set	AES		SHAKE256	
	Latency (cycle)	Throughput (C/B)	Latency (cycle)	Throughput (C/B)
Par-80S	2154	72.63	5906	199.1
Par-80M	3644	49.36	11465	143.5
Par-80L	4957	32.97	16679	110.9
Par-128S	3076	103.6	10446	351.8
Par-128M	4381	55.10	14292	179.7
Par-128L	5323	35.70	16920	113.5

Table 3.12: Client-side performance of the RtF transciphering framework with Rubato.

Set	CER	Latency		Throughput (KB/s)	Precision (bits)
		Offline (s)	Online (s)		
Par-80S	1.31	21.48	19.75	6.676	18.8
Par-80M	1.25	37.44	19.71	7.032	19.0
Par-80L	1.25	85.65	19.79	6.520	19.1
Par-128S	1.31	50.78	20.28	6.083	18.8
Par-128M	1.26	68.47	19.88	6.666	18.9
Par-128L	1.26	86.34	20.09	6.712	18.9

Table 3.13: Server-side performance of the RtF transciphering framework with Rubato.

3.6.4 Performance Comparison

We compare the result of HERA, Rubato, LWEs-to-RLWE conversions [79], and CKKS itself. The comparison is summarized in Table 3.14. The source codes of LWEs-to-RLWE conversion are taken from the `OpenPegasus` library[5]. As `OpenPegasus` library does not include symmetric LWE encryption, we implement (seeded) symmetric LWE encryption with AVX2-optimized SHAKE256. We use `Lattigo` library for CKKS encryption. Since the `OpenPegasus` library supports only selected sets of parameters in terms of the number of slots and the ciphertext modulus (at the point of submission), we implemented LWEs-to-RLWE for $N = 2^{16}$ and $\ell = 2^{10}$ which does not provide exactly the same functionality as ours with full available slots.

In this table, the security parameter λ is set to 128. For the fairness of comparison, the remaining levels after transciphering are all set to be 7. For all experiments, we sample the domain of each component of the message vector from uniform distribution over $(-1, 1)$. When computing the ciphertext expansion ratio, we use the formula $\log q/(p+1)$, which excludes the effect of sending a public nonce. Multiple use of different nonces can be dealt with a counter so that the effect of a nonce on the ratio is asymptotically zero.

HERA outperforms the LWEs-to-RLWE conversion with respect to CER, ciphertext size and client-side performance, achieving the main purpose of the transciphering framework. Compared to the CKKS-only environment, HERA with the RtF framework has better CER and client-side performance, while the CKKS-only environment requires no additional computation. When Rubato and HERA are compared in the RtF framework, client-side and server-side throughput is improved by 22.9% and 32.2%, respectively, at the cost of only 1.6% larger ciphertext expansion.

KAIST

Scheme	N	ℓ	Ctxt. Exp.		Client		Server		p
			Ctxt. (KB)	Ratio	Lat. (μs)	Thrp. (MB/s)	Lat. (s)	Thrp. (KB/s)	
RtF-HERA	2^{16}	2^{16}	0.055	1.24	1.520	25.26	141.58	5.077	19.1
RtF-Rubato	2^{16}	2^{16}	0.183	1.26	4.585	31.04	106.4	6.712	18.9
LWE [79]	$2^{16}(2^{10})$	2^{10}	0.007	4.84	21.91	0.051	65.88	0.010	9.3
CKKS	2^{14}	2^{14}	468	23.25	9656	2.035	none		19.1

Table 3.14: Comparison of the RtF transciphering framework with Rubato to previous measures supporting homomorphic encryption of approximate numbers. All the experiments are done with 128-bit security. Parameter N in parentheses implies the dimension of LWE. The parameter p stands for the bits of precision.

[5] https://github.com/Alibaba-Gemini-Lab/OpenPEGASUS

Chapter 4. Efficient Transciphering of Binary Numbers

In BGV/FV-style HE schemes, a user who wants to encrypt binary data will use a plaintext space $\mathbb{Z}_2[X]/(\Phi_M(X))$ for a polynomial $\Phi_M(X)$. The batching technique divides the plaintext space into ℓ slots for some ℓ dividing N, where $\deg \Phi_M = N$. In this way, the user is allowed to pack ℓ bits of data in a single ciphertext. The value of ℓ depends on the polynomial $\Phi_M(X)$, but typically $\ell \ll N$.

The plaintext is an encoding of elements in the splitting field of $\Phi_M(X)$ over $\mathbb{Z}_2$, which is equivalent to the e-bit string where $e = N/\ell$. Consequently, a single ciphertext can encrypt N bits of data. However, when we use the whole splitting field for the data storage, then we cannot compute AND gates homomorphically. The multiplication in HE is translated to the multiplication on $\mathbb{F}_{2^e}$.

As a solution to this problem, we propose a new transciphering framework, dubbed BtE (Binary-to-Extension-field). It utilizes the whole inner-slot storage by computing the cipher over an extension field $\mathbb{F}_{2^e}$. This offers better HE performance since the number of multiplications per keystream decreases. At the end of the transciphering, we place a *digit extraction* function which homomorphically extracts every single bit in inner-slot storage. We depict the digit extraction function in Figure 4.1. After the digit extraction function, the resulting homomorphic ciphertext can be efficiently computed by AND gates.

4.1 BtE Transciphering Framework

KAIST

In this section, we propose a new transciphering framework BtE over an extension field. The BtE framework is no other than normal transciphering framework over an extension field, except the digit extraction. The digit extraction function distributes coefficient bits into binary elements which occupy one whole slot. We note that the idea of digit extraction has already been introduced in the literature [54, 62, 30, 97]. After extracting digits, the server can efficiently evaluate the binary AND gate. In this section, we look into the digit extraction function and specify the whole framework.

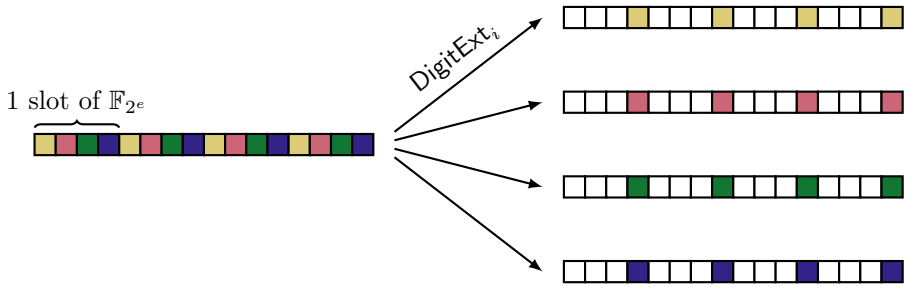


Figure 4.1: Pictorial description of DigitExt. Each 16-cell bar stands for a homomorphic ciphertext, 4-cell bar stands for a single slot of $\mathbb{F}_{2^e}$, and each cell stands for the digit (bit for binary). Each digit is extracted and moved to the LSB of the slot.

4.1.1 Digit Extraction

Assume that field $\mathbb{F}_{2^e}$ has a power basis $\{1, \alpha, \alpha^2, \ldots, \alpha^{e-1}\}$ over $\mathbb{F}_2$. Then, an element x can be represented by

$$x = \sum_{i=0}^{e-1} a_i \alpha^i \in \mathbb{F}_{2^e}$$

where $a_i \in \mathbb{F}_2$. Now we can define digit extraction τ_i as follows.

$$\tau_i : \sum_{i=0}^{e-1} a_i \alpha^i \mapsto a_i \in \mathbb{F}_2 \subset \mathbb{F}_{2^e}$$

One can see that τ_i is a linear operation so that it can be efficiently evaluated homomorphically. For homomorphic ciphertexts in $\mathcal{C}$, the digit extraction function $\mathsf{DigitExt} : \mathcal{C} \to \mathcal{C}^e$ can be defined by the sequential concatenation of $\mathsf{DigitExt}_i$ where

$$\mathsf{DigitExt}_i : \mathsf{Enc}([x_0, \ldots, x_{\ell-1}]) \mapsto \mathsf{Enc}([\tau_i(x_0), \ldots, \tau_i(x_{\ell-1})]).$$

Such function τ_i is in fact a matrix multiplication over $\mathbb{F}_2$. By the isomorphism $\iota : \mathbb{F}_{2^e} \to \mathbb{F}_2^e$ using the power basis, τ_i can be redefined by

$$\tau_i(x) = \mathbf{M}_i \cdot \iota(x)$$

where the matrix $\mathbf{M}_i$ is defined by

KAIST

$$(\mathbf{M}_i)_{jk} = \begin{cases} 1 & \text{if } j = 0 \text{ and } k = i \\ 0 & \text{otherwise } (0 \le j, k \le e-1). \end{cases}$$

When computing it homomorphically, one cannot use matrix multiplication over $\mathbb{F}_2$. Instead, evaluating corresponding linearized polynomial has the same effect as the matrix multiplication.

Given $x(\alpha) \in \mathbb{F}_{2^e}$ and a matrix $\mathbf{M} \in \mathbb{F}_2^{e \times e}$, there exists a tuple $\mathbf{b}^\top = (b_0, \ldots, b_{e-1}) \in (\mathbb{F}_{2^e})^e$ such that

$$\iota^{-1}(\mathbf{M} \cdot \iota(x)) = \sum_{i=0}^{e-1} b_i x^{2^i}.$$

This implies that

$$\begin{aligned} \iota^{-1}(M \cdot \iota(x)) &= \sum_{i=0}^{e-1} b_i x(\alpha^{2^i}) \\ &= \mathbf{b}^\top \mathbf{D} \cdot \iota(x) \end{aligned} \tag{4.1}$$

where the $e \times e$-matrix $\mathbf{D} \in \mathbb{F}_{2^e}^{e \times e}$ is defined by

$$(\mathbf{D})_{ij} = \alpha^{j \cdot 2^i}$$

for $0 \le i, j \le e-1$. Here one can regard that ι^{-1} as a inner product with $\mathbf{c} = (1, \alpha, \alpha^2, \ldots, \alpha^{e-1})$. By extending the domain of ι^{-1} from $\mathbb{F}_2^e$ to $\mathbb{F}_{2^e}^e$, Equation 4.1 can be restated by

$$\mathbf{c}^\top \mathbf{M} \cdot \iota(x) = \mathbf{b}^\top \mathbf{D} \cdot \iota(x)$$

Since Equation 4.1 satisfies for all $x \in \mathbb{F}_{2^e}$, we can say that

$$\mathbf{b}^\top = \mathbf{c}^\top \mathbf{M}\mathbf{D}^{-1}$$

where $\mathbf{D}$ is well-known to be invertible.

4.1.2 Specification of the Framework

Given a symmetric cipher E over $\mathbb{F}_{2^e}$, BtE transciphering framework evaluates the digit extraction functions after the basic transciphering framework over $\mathbb{F}_{2^e}$. We remind client-side computation briefly, and will specify the server-side one.

Client-side Computation. Before the symmetric encryption, the client is required to send HE-ciphertext $\mathcal{K}$ of symmetric key $\mathbf{k}$ and nonce $\mathsf{nc} \in \{0,1\}^\lambda$ once to the server. One may use column-wise packing or row-wise packing to establish $\mathcal{K}$.

For a given message $\mathbf{m} \in \{0,1\}^{ne}$, the client encodes $\mathbf{m}$ into $\widetilde{\mathbf{m}} \in \mathbb{F}_{2^e}^n$ where encoding function can varies[1]. Then, using symmetric cipher E, the client computes

$$\mathbf{c} = \widetilde{\mathbf{m}} + \mathsf{E}_{\mathbf{k}}(\mathsf{nc})$$

and send $\mathbf{c}$ to the server.

Server-side Computation. Let B be the number of symmetric ciphertexts transciphered at a time. The number B can be $\lfloor \ell/n \rfloor$ for the column-wise packing or ℓ for the row-wise packing where ℓ is the number of slots. Given a tuple of nonces $(\mathsf{nc}_0, \ldots, \mathsf{nc}_{B-1})$ and the HE-encrypted key $\mathcal{K}$, the server is able to construct a circuit for the homomorphic evaluation of E, denoted by $\mathsf{Eval}(\mathsf{E}, \{\mathsf{nc}_{\mathsf{ctr}}\}_{\mathsf{ctr}}, \cdot)$. The server can restore HE-ciphertext of keystreams $\mathsf{E}_{\mathbf{k}}(\mathsf{nc}_i)$ by evaluating

$$\mathcal{V} \coloneqq \mathsf{Eval}(\mathsf{E}, \{\mathsf{nc}_{\mathsf{ctr}}\}_{\mathsf{ctr}}, \mathcal{K}).$$

The server encodes received symmetric ciphertexts $(\mathbf{c}_0, \ldots, \mathbf{c}_{B-1})$ to C with proper packing method. Then, $\mathcal{V} + C$ is an HE-encryption of the original messages $(\mathbf{m}_0, \ldots, \mathbf{m}_{B-1})$. After the digit extraction, the server has a set of ciphertexts

$$\mathsf{DigitExt}(\mathcal{V} + C) = \{\mathsf{DigitExt}_i(\mathcal{V} + C)\}_{i=0}^{e-1}.$$

4.2 eHERA: Ciphers over Extension Fields

As the BtE framework requires a cipher over a binary extension field, we propose a new variant of HERA, dubbed eHERA (which means "extended HERA"), operating on a binary extension field $\mathbb{F}_{2^e}$. We note that previous ciphers over extension fields [1, 9, 11, 59] are not suitable for the BtE framework since they use too large fields such as $e \approx 128$. Comprehensive cryptanalysis of eHERA is given in Section 4.3. The overall structure of eHERA is similar to HERA, while we need to carefully estimate the number of monomials in its algebraic representation since a power map might result in a sparse polynomial in a field of characteristic 2.

[1] The output bits of the transciphering will be scattered into different HE-ciphertexts. If there is any wanted shape of packing, a corresponding permutation of bit positions can be appended in the encoding function.

KAIST

4.2.1 Design Change of eHERA from HERA

When dealing with fields of characteristic 2, it requires additional considerations with respect to the security and the efficiency. In algebraic attacks, many monomials disappear regularly (i.e., not depending on the choice of nonce) so that the number of monomials in eHERA cannot be similarly deduced as in HERA. For example, for $a_i \in \mathbb{F}$ where $\mathrm{char}(\mathbb{F}) = 2$,

$$\begin{aligned}\left(\sum_{i=1}^{n} a_i\right)^3 &= \sum_{i=1}^{n} a_i^3 + \sum_{i<j} 3a_i^2 a_j + \sum_{i<j<k} 6a_i a_j a_k \\ &= \sum_{i=1}^{n} a_i^3 + \sum_{i<j} a_i^2 a_j.\end{aligned}$$

It implies that not all monomials of degree t appear after the first round when $t = 3$.

From an efficiency perspective, exponentiation by power-of-two is not a costly evaluation regardless of what the exact exponent is. By evaluating one (iterative) Frobenius automorphism homomorphically ($f(X) \mapsto f(X^{2^\theta})$), the degree of the cipher over $\mathbb{F}_{2^e}$ can be easily raised without excessive cost. Considering that HE-friendly ciphers are usually weak against algebraic attack, this point is favourable to design a cipher.

Overall, we raise the degree t of S-box to $2^\theta + 1$. The parameter θ is chosen in order to count the number of monomials well, which is elaborated in Section 4.3.1.

4.2.2 Specification

KAIST

Each component in the stream cipher eHERA is quite similar to HERA. However, the difference of domain makes slight changes from the original one. A stream cipher eHERA for λ-bit security takes as input a symmetric key $\mathbf{k} \in \mathbb{F}_{2^e}^{16}$, a nonce $\mathsf{nc} \in \{0,1\}^\lambda$, and returns a keystream $\mathbf{k}_{\mathsf{nc}} \in \mathbb{F}_{2^e}^{16}$, where the nonce is fed to the underlying extendable output function (XOF) that outputs an element in $(\mathbb{F}_{2^e}^{16})^*$. In a nutshell, eHERA is defined as follows.

$$\mathsf{eHERA}[\mathbf{k}, \mathsf{nc}] = \mathsf{Fin}[\mathbf{k}, \mathsf{nc}, r] \circ \mathsf{RF}[\mathbf{k}, \mathsf{nc}, r-1] \circ \cdots \circ \mathsf{RF}[\mathbf{k}, \mathsf{nc}, 1] \circ \mathsf{ARK}[\mathbf{k}, \mathsf{nc}, 0]$$

where the i-th round function $\mathsf{RF}[\mathbf{k}, \mathsf{nc}, i]$ is defined as

$$\mathsf{RF}[\mathbf{k}, \mathsf{nc}, i] = \mathsf{ARK}[\mathbf{k}, \mathsf{nc}, i] \circ \mathsf{Exp}_t \circ \mathsf{LinearLayer}$$

and the final round function Fin is defined as

$$\mathsf{Fin}[\mathbf{k}, \mathsf{nc}, r] = \mathsf{ARK}[\mathbf{k}, \mathsf{nc}, r] \circ \mathsf{LinearLayer} \circ \mathsf{Exp}_t \circ \mathsf{LinearLayer}$$

for $i = 1, 2, \ldots, r-1$ (see Figure 4.2).

Key Schedule. The round key schedule can be simply seen as component-wise product between a random value and the master key $\mathbf{k}$, where the uniformly random value in $\mathbb{F}_{2^e}^\times$ is obtained from a certain extendable output function XOF with an input nc. Given a sequence of the outputs from XOF, say $\mathbf{rc} = (\mathbf{rc}_0, \ldots, \mathbf{rc}_r) \in (\mathbb{F}_{2^e}^{16})^{r+1}$, ARK is defined as follows.

$$\mathsf{ARK}[\mathbf{k}, \mathsf{nc}, i](\mathbf{x}) = \mathbf{x} + \mathbf{k} \bullet \mathbf{rc}_i$$

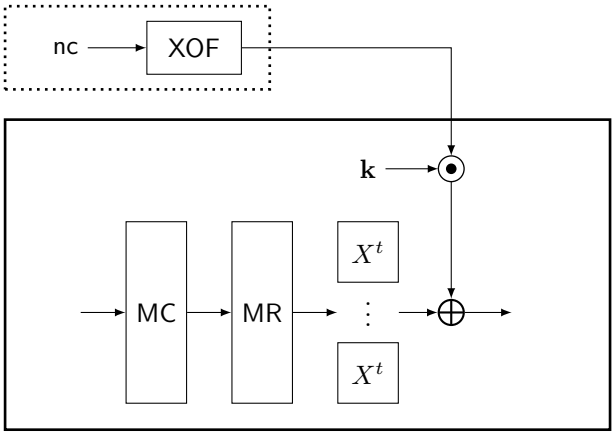


Figure 4.2: The round function of eHERA. Operations in the box with dotted (resp. thick) lines are public (resp. secret). "MC" and "MR" represent MixColumns and MixRows, respectively.

for $i = 0, \ldots, r$, and $\mathbf{x} \in \mathbb{F}_{2^e}^{16}$, where $\bullet$ (resp. $+$) denotes component-wise multiplication (resp. addition) over $\mathbb{F}_{2^e}$.

Linear Layers. Linear layer of HERA is composed of MixColumns and MixRows. Similarly to HERA, linear layer of eHERA is also composed of MixColumns and MixRows whereas the concrete numbers in the matrices are adjusted to the extension fields. MixColumns and MixRows are defined as in Figure 4.3a and Figure 4.3b, respectively. The notations in matrices, for example, '11' stands for $X + 1 \in \mathbb{F}_2[X]/(f(X)) \simeq \mathbb{F}_{2^e}$ for some specific field representation.

$$\begin{bmatrix} y_{0c} \\ y_{1c} \\ y_{2c} \\ y_{3c} \end{bmatrix} = \begin{bmatrix} 10 & 11 & 01 & 01 \\ 01 & 10 & 11 & 01 \\ 01 & 01 & 10 & 11 \\ 11 & 01 & 01 & 10 \end{bmatrix} \cdot \begin{bmatrix} x_{0c} \\ x_{1c} \\ x_{2c} \\ x_{3c} \end{bmatrix} \qquad \begin{bmatrix} y_{c0} \\ y_{c1} \\ y_{c2} \\ y_{c3} \end{bmatrix} = \begin{bmatrix} 10 & 11 & 01 & 01 \\ 01 & 10 & 11 & 01 \\ 01 & 01 & 10 & 11 \\ 11 & 01 & 01 & 10 \end{bmatrix} \cdot \begin{bmatrix} x_{c0} \\ x_{c1} \\ x_{c2} \\ x_{c3} \end{bmatrix}$$

(a) MixColumns (b) MixRows

Figure 4.3: Definition of MixColumns and MixRows in eHERA. For $c \in \{0, 1, 2, 3\}$, x_{ij} and y_{ij} are defined as in Figure 3.3.

Nonlinear Layers. Suppose positive integer t is given such that $t = 2^\theta + 1$ for some positive integer θ and $\gcd(t, 2^e - 1) = 1$. The nonlinear map Exp_t is the concatenation of 16 copies of the same S-box, where the S-box is defined by $x \mapsto x^t$ over $\mathbb{F}_{2^e}$. So, for $\mathbf{x} = (x_0, \ldots, x_{15}) \in \mathbb{F}_{2^e}^{16}$, we have

$$\mathsf{Exp}_t(\mathbf{x}) = (x_0^t, \ldots, x_{15}^t).$$

Encryption Mode. When a keystream of k blocks (in $(\mathbb{F}_{2^e}^{16})^k$) is needed for some $k > 0$, the "inner-counter mode" can be used; for $\mathsf{ctr} = 0, 1, \ldots, k-1$, one computes

$$\mathbf{z}[\mathsf{ctr}] = \mathsf{HERA}\left[\mathbf{k}, \mathsf{nc} \| \mathsf{ctr}\right](\mathbf{ic}),$$

where $\mathbf{ic}$ denotes a constant $(1, 2, \ldots, 16) \in \mathbb{F}_{2^e}^{16}$.

KAIST

4.2.3 Parameter Selection

Designing cryptographic primitive is a tense negotiation between security and efficiency. Choosing parameters is usually the last step of the negotiation. In that sense, in symmetric cryptography, the security analysis of the round function comes first and the choice of parameters – such as the number of rounds – follows from the analysis. However, the BtE framework requires the specific parameters of eHERA so that we fix the parameters of eHERA and analyze it. We make a checklist for the choice of parameters.

Suppose we have a (HE) plaintext space $\mathbb{Z}_2[X]/(\Phi_M(X))$ with $\phi(M) = N$. Then there is a factorization

$$\Phi_M(X) = F_1(X) \cdot F_2(X) \cdot \cdots \cdot F_\ell(X)$$

such that all the $F_i(X)$ is irreducible over $\mathbb{Z}_2$ of degree d satisfying $N = d\ell$. If we want to evaluate eHERA over this plaintext space, $\mathbb{F}_{2^e}$ should be a subfield of (or isomorphic to) $\mathbb{F}_{2^d}$, which implies that $e \mid d$. When we select the degree t (equivalently θ) of S-box, for the invertibility of S-box, e and t should satisfy $\gcd(t, 2^e - 1) = 1$. In the linear cryptanalysis, we use the linear probability result for $\gcd(\theta, e) = 1$ where e is odd and $\theta < (e-1)/2$. Overall, we select some parameters in Table 4.1. We use $X^{21} + X^2 + 1$ for the irreducible polynomial defining $\mathbb{F}_{2^e}$

Cipher	Symmetric key parameters						HE parameters				
	λ (bits)	n	e	t	Key Size (bits)	r	M	λ' (bits)	Slot	Ctxt. Mod. (bits)	Remaining Budget (bits)
eHERA-4	80	16	21	17	336	4	42799	364	2016	270	49
eHERA-6	128	16	21	17	336	6	42799	320	2016	324	25
eHERA-7	192	16	21	17	336	7	42799	296	2016	348	8
eHERA-8	256	16	21	17	336	8	42799	247	2016	392	12

Table 4.1: Some selected parameters of eHERA. The ciphertext moduli are tightly chosen, which means that further computation after transciphering requires larger moduli. "Ctxt. Mod." stands for the log size of ciphertext modulus.

4.3 Security Analysis of eHERA

In this section, we provide the security analysis of eHERA. Table 4.2 shows the number of rounds to prevent each of the attacks considered in this section according to the security level λ for $n = 16$, $e = 21$ and $t = 17$.

Attack \ λ	80	128	192	256
Trivial Linearization	4	6	7	8
GCD Attack	1	1	1	1
Gröbner Basis Attack	2	2	3	3
Interpolation Attack	4	6	7	8
Linear Cryptanalysis	2	2	4	4

Table 4.2: Recommended number of rounds with respect to each attack.

Assumptions and the Scope of Analysis. We limit the number of encryptions under the same key up to the birthday bound with respect to λ, i.e., $2^{\lambda/2}$, since otherwise one would not be able to avoid a

nonce collision (when nonces are chosen uniformly at random).

In this work, we will consider the standard "secret-key model", where an adversary arbitrarily chooses a nonce, and obtains the corresponding keystream without any information on the secret key. The related-key and the known-key models are beyond the scope of this paper.

Since eHERA takes as input counters, an adversary is not able to control the differences of the inputs. Nonces can be adversarially chosen, while they are also fed to the extendable output function, which is modeled as a random oracle. So one cannot control the difference of the internal variables. For this reason, we believe that our construction is secure against any type of chosen-plaintext attack including (higher-order) differential, truncated differential, invariant subspace trail and cube attacks.

4.3.1 On the Number of Monomials in eHERA

The eHERA cipher can be represented as a set of polynomials over $\mathbb{F}_{2^e}$ in unknowns $k_0, k_1, \ldots, k_{n-1}$ where $k_i \in \mathbb{F}_{2^e}$ denotes the i-th component of the secret key. Since the number of monomials appearing in the polynomials plays a key role in algebraic analysis, we first specify a lower bound of the number of monomials appearing in the eHERA cipher for the security analysis.

Linear Layer. The round function of eHERA is defined by

$$\mathsf{RF} = \mathsf{ARK} \circ \mathsf{Exp}_t \circ \mathsf{MixRows} \circ \mathsf{MixColumns},$$

where the two linear maps MixColumns and MixRows can be represented by 16×16-matrices over $\mathbb{F}_{2^e}$. Their product represents MixRows $\circ$ MixColumns as follows.

$$\mathsf{MixRows} \circ \mathsf{MixColumns} = \begin{pmatrix}
4 & 6 & 2 & 2 & 6 & 5 & 3 & 3 & 2 & 3 & 1 & 1 & 2 & 3 & 1 & 1 \\
2 & 4 & 6 & 2 & 3 & 6 & 5 & 3 & 1 & 2 & 3 & 1 & 1 & 2 & 3 & 1 \\
2 & 2 & 4 & 6 & 3 & 3 & 6 & 5 & 1 & 1 & 2 & 3 & 1 & 1 & 2 & 3 \\
6 & 2 & 2 & 4 & 5 & 3 & 3 & 6 & 3 & 1 & 1 & 2 & 3 & 1 & 1 & 2 \\
2 & 3 & 1 & 1 & 4 & 6 & 2 & 2 & 6 & 5 & 3 & 3 & 2 & 3 & 1 & 1 \\
1 & 2 & 3 & 1 & 2 & 4 & 6 & 2 & 3 & 6 & 5 & 3 & 1 & 2 & 3 & 1 \\
1 & 1 & 2 & 3 & 2 & 2 & 4 & 6 & 3 & 3 & 6 & 5 & 1 & 1 & 2 & 3 \\
3 & 1 & 1 & 2 & 6 & 2 & 2 & 4 & 5 & 3 & 3 & 6 & 3 & 1 & 1 & 2 \\
2 & 3 & 1 & 1 & 2 & 3 & 1 & 1 & 4 & 6 & 2 & 2 & 6 & 5 & 3 & 3 \\
1 & 2 & 3 & 1 & 1 & 2 & 3 & 1 & 2 & 4 & 6 & 2 & 3 & 6 & 5 & 3 \\
1 & 1 & 2 & 3 & 1 & 1 & 2 & 3 & 2 & 2 & 4 & 6 & 3 & 3 & 6 & 5 \\
3 & 1 & 1 & 2 & 3 & 1 & 1 & 2 & 6 & 2 & 2 & 4 & 5 & 3 & 3 & 6 \\
6 & 5 & 3 & 3 & 2 & 3 & 1 & 1 & 2 & 3 & 1 & 1 & 4 & 6 & 2 & 2 \\
3 & 6 & 5 & 3 & 1 & 2 & 3 & 1 & 1 & 2 & 3 & 1 & 2 & 4 & 6 & 2 \\
3 & 3 & 6 & 5 & 1 & 1 & 2 & 3 & 1 & 1 & 2 & 3 & 2 & 2 & 4 & 6 \\
5 & 3 & 3 & 6 & 3 & 1 & 1 & 2 & 3 & 1 & 1 & 2 & 6 & 2 & 2 & 4
\end{pmatrix}$$

We see that the matrix representation of MixRows $\circ$ MixColumns has no zero entry. It implies that MixRows $\circ$ MixColumns contains all the linear monomials in its polynomial representation.

S-box. Next, subsequent nonlinear layer makes monomials of higher degrees. For $t = 2^\theta + 1$, the S-box output for an input $x_0 + x_1 + \cdots + x_{n-1}$ is

$$(x_0 + x_1 + \cdots + x_{n-1})^t = (x_0 + x_1 + \cdots + x_{n-1})(x_0^{2^\theta} + x_1^{2^\theta} + \cdots + x_{n-1}^{2^\theta})$$

since $\mathbb{F}_{2^e}$ is of characteristic 2. From this observation, we can see that monomials of highest degree in $(x_0 + x_1 + \cdots + x_{n-1})^t$ are $x_i^{2^\theta} x_j$ for $0 \le i, j \le n-1$. Since the input of each S-box of the first round is $\sum_{i=0}^{n-1} a_i k_i + b$ for some nonzero $a_i \in \mathbb{F}_{2^e}$'s and $b \in \mathbb{F}_{2^e}$, the output of the S-box has n^2 monomials of degree t.

Now, suppose no monomial of degree t is vanished on the linear layer of the second round. After the nonlinear layer of the second layer, we obtain monomials of the highest degree of the form $x_0^{2^{2\theta}} x_1^{2^\theta} x_2^{2^\theta} x_3$ where $x_0, x_1, x_2, x_3 \in \{k_0, k_1, \ldots, k_{15}\}$. The number of such monomials is at least

$$\left(\!\!\binom{n}{1}\!\!\right) \cdot \left(\!\!\binom{n}{2}\!\!\right) \cdot \left(\!\!\binom{n}{1}\!\!\right) = \binom{n}{1} \cdot \binom{n+1}{2} \cdot \binom{n}{1}$$

where $\left(\!\binom{n}{k}\!\right) = \binom{n+k-1}{k}$ is a combination with repetition, assuming that $2^\theta > 2$. The assumption $2^\theta > 2$ is required to ensure that $2^{2\theta} > 2 \cdot 2^\theta + 1$, i.e., the same monomial can be made only by permutations between x_i's of the same exponent. Repeating this process, we can represent the highest degree monomials after the r-th round as

$$X_r = \prod_{i=0}^{2^r - 1} x_i^{2^{(r-a_i)\theta}}$$

where $x_i \in \{k_0, \ldots, k_{n-1}\}$ and $a_i = \min_{0 \le a \le r} \left\{ a : \sum_{j=0}^{a} \binom{r}{j} \ge i + 1 \right\}$ for $i = 0, \ldots, 2^r - 1$, provided that no highest degree monomials are not vanished during linear layers. Then the number of monomials of degree 2^r is at least

$$S = \prod_{i=0}^{r} \left(\!\!\binom{n}{b_i}\!\!\right) = \prod_{i=0}^{r} \binom{n + b_i - 1}{b_i} \tag{4.2}$$

where $b_i = \binom{r}{i}$, assuming $2^\theta > \binom{r}{\lfloor r/2 \rfloor}$. Note that this argument works correctly until $r\theta < e$ since $x^{2^e} \equiv x$ for all $x \in \mathbb{F}_{2^e}$. When $e = 21$ and $t = 17$, the conditions $2^\theta > \binom{r}{\lfloor r/2 \rfloor}$ and $r\theta < e$ holds only if $r \le 5$. Hence we can use the equation (4.2) until $r \le 5$, and more detailed analysis is required for $r > 5$.

For $r = 6$, we have monomials of the form $X_r = \prod_{i=1}^{2^r} x_i^{2^{(r-a_i)\theta}}$, which has one variable of exponent $2^{6\theta}$, $\binom{6}{1}$ variables of exponent $2^{5\theta}$, $\binom{6}{2}$ variables of exponent $2^{4\theta}$, and so on. We have to deal with variables of exponent $2^{6\theta}$, $2^{5\theta}$, and $2^{3\theta}$ carefully because $2^{6\theta} > 2^e$, $2^{5\theta+1} = 2^e$, and there are more than 2^θ variables of exponent $2^{3\theta}$. Firstly, $x^{2^{6\theta}} = x^{2^{24}} = x^{2^3}$. Degree 2^3 cannot be made from variables of lower exponents because $2^3 < 2^\theta$ and there are only one variable of exponent 1. Thus we can choose the variable x_0 of order $2^{6\theta}$ freely from $\{k_0, \ldots, k_{n-1}\}$. Next, $(x^{2^{5\theta}})^2 = x^{2^{5\theta+1}} = x^{2^e} = x$. To prevent such situation, choose 6 variables of exponent $2^{5\theta}$ to be distinct. Then there are $\binom{n}{6}$ cases to choose variables of exponent $2^{5\theta}$. Lastly, $(x^{2^{3\theta}})^{16} = x^{2^{3\theta} \cdot 2^4} = x^{2^{4\theta}}$. We exclude such event during counting the number of possible choices for variables of exponent $2^{3\theta}$. The number of excluded cases is $n\left(\!\!\binom{n}{20-2^\theta}\!\!\right)$ since there are n possible choices of the variable appearing more than or equal to 2^θ times and $\left(\!\!\binom{n}{20-2^\theta}\!\!\right)$ possible choices of the remaining variables. Hence, the number of monomials after 6 rounds is at least

$$S = n \times \binom{n}{6} \times \left(\!\!\binom{n}{15}\!\!\right) \times \left\{ \left(\!\!\binom{n}{20}\!\!\right) - n \left(\!\!\binom{n}{4}\!\!\right) \right\} \times \left(\!\!\binom{n}{15}\!\!\right) \times \left(\!\!\binom{n}{6}\!\!\right) \times n \tag{4.3}$$

which is about $2^{122.71}$ for $n = 16$.

Rounds	4	5	6	7
$\log S$	47.57	79.12	122.71	150.12

Table 4.3: Values of $\log S$ according to the number of rounds when $n = 16$, $e = 21$ and $t = 17$. S is a lower bound of the number of monomials.

For $r = 7$, we have monomials of the form $X_r = \prod_{i=1}^{2^r} x_i^{2^{(r-a_i)\theta}}$ containing one variable of exponent $2^{7\theta}$, $\binom{7}{1}$ variables of exponent $2^{6\theta}$, $\binom{7}{2}$ variables of exponent $2^{5\theta}$, and so on. Here, we must be careful about the most of the exponents. Firstly, $x^{2^{7\theta}} = x^{2^{28}} = x^{2^7}$. Degree 2^7 cannot be made from variables of lower exponents because $2^7 < 2^{2\theta}$ and there are only 7 variables of exponent $2^\theta = 2^4$. Similar argument holds for variables of exponent $2^{6\theta}$, and we can choose distinct 7 variables of exponent $2^{6\theta}$. However, we cannot choose distinct 21 variables of exponent $2^{5\theta}$ as in the case $r = 6$ if $n = 16$, so just fix variables of exponent $2^{5\theta}$ and ignore them. For variables of exponent $2^{4\theta}$, $2^{3\theta}$, and $2^{2\theta}$, exclude the cases when exponent $2^{j\theta}$ is carried up to $2^{(j+1)\theta}$ as in the case $r = 6$. Then the number of monomials after 7 rounds is at least

$$S = n \times \binom{n}{7} \times \left\{ \left(\!\!\binom{n}{32}\!\!\right) - n \left(\!\!\binom{n}{16}\!\!\right) \right\}^2 \times \left\{ \left(\!\!\binom{n}{21}\!\!\right) - n \left(\!\!\binom{n}{5}\!\!\right) \right\} \times \left(\!\!\binom{n}{7}\!\!\right) \times n \tag{4.4}$$

which is about $2^{150.12}$ for $n = 16$.

Values of $\log S$ are given in Table 4.3 for some number of rounds when $n = 16$, $e = 21$ and $t = 17$.

Inverse S-box. One might construct a polynomial in the decryption direction. In this case, we have to consider the number of monomials after the inverse S-box. When $e = 21$ and $t = 17$, the inverse S-box is given by $x \mapsto x^{740171}$:

$$(x^{17})^{740171} = x^{6 \cdot 2^{21} - 5} = (x^{2^{21}})^6 \cdot x^{-5} = x.$$

From $740171 = 2^{19} + 2^{17} + 2^{16} + 2^{14} + 2^{11} + 2^9 + 2^8 + 2^6 + 2^3 + 2 + 1$, the output of the inverse S-box for an input $x_0 + x_1 + \cdots + x_{n-1}$ is a product of 11 polynomials of form $x_0^{2^j} + x_1^{2^j} + \ldots x_{n-1}^{2^j}$ as follows.

$$\left(\sum_{i=0}^{n-1} x_i\right)^{740171} = \left(\sum_{i=0}^{n-1} x_i^{2^{19}}\right) \times \left(\sum_{i=0}^{n-1} x_i^{2^{17}}\right) \times \left(\sum_{i=0}^{n-1} x_i^{2^{16}}\right) \times \cdots \times \left(\sum_{i=0}^{n-1} x_i\right)$$

Using the same argument as above, we can conclude that the number of monomials of the highest degree after one round in inverse direction is n^{11}, which is 2^{44} when $n = 16$.

In case of two rounds, we have monomials of the form

$$X_0^{2^{19}} X_1^{2^{17}} X_2^{2^{16}} X_3^{2^{14}} X_4^{2^{11}} X_5^{2^9} X_6^{2^8} X_7^{2^6} X_8^{2^3} X_9^2 X_{10}$$

where each X_i is a monomial appearing after one round in inverse direction. Here, we ignore coefficients of the monomials. Fix $X_1, X_2, \ldots, X_7$ for simplicity, and let

$$\begin{aligned}
X_0 &= x_0^{2^{19}} x_1^{2^{17}} x_2^{2^{16}} x_3^{2^{14}} x_4^{2^{11}} x_5^{2^9} x_6^{2^8} x_7^{2^6} x_8^{2^3} x_9^2 x_{10}, \\
X_8 &= y_0^{2^{19}} y_1^{2^{17}} y_2^{2^{16}} y_3^{2^{14}} y_4^{2^{11}} y_5^{2^9} y_6^{2^8} y_7^{2^6} y_8^{2^3} y_9^2 y_{10}, \\
X_9 &= z_0^{2^{19}} z_1^{2^{17}} z_2^{2^{16}} z_3^{2^{14}} z_4^{2^{11}} z_5^{2^9} z_6^{2^8} z_7^{2^6} z_8^{2^3} z_9^2 z_{10}, \\
X_{10} &= w_0^{2^{19}} w_1^{2^{17}} w_2^{2^{16}} w_3^{2^{14}} w_4^{2^{11}} w_5^{2^9} w_6^{2^8} w_7^{2^6} w_8^{2^3} w_9^2 w_{10}
\end{aligned}$$

r \ g	0	1	2	3	4	5	6	7
4	95.14	113.43	131.55	149.48	167.19	184.64	201.79	218.59
5	158.24	174.26	189.99	205.39	220.42	235.03	249.14	262.67
6	245.42	257.27	268.61	279.36	289.43	298.72	307.08	314.31
7	300.24	308.97	317.06	324.41	330.93	336.45	340.79	343.65

Table 4.4: The log of the time complexity according to the the number of rounds r and the number of guessed variables g.

where $x_i, y_i, z_i, w_i \in \{k_0, k_1, \ldots, k_{n-1}\}$ for all $i = 0, 1, \ldots, 10$. Then we have

$$\begin{aligned} X_0^{2^{19}} &= x_9^{2^{20}} x_{10}^{2^{19}} x_0^{2^{17}} x_1^{2^{15}} x_2^{2^{14}} x_3^{2^{12}} x_4^{2^9} x_5^{2^7} x_6^{2^6} x_7^{2^4} x_8^2, \\ X_8^{2^2} &= y_1^{2^{19}} y_2^{2^{18}} y_3^{2^{16}} y_4^{2^{13}} y_5^{2^{11}} y_6^{2^{10}} y_7^{2^8} y_8^{2^5} y_9^{2^3} y_{10}^{2^2} y_0, \\ X_9^2 &= z_0^{2^{20}} z_1^{2^{18}} z_2^{2^{17}} z_3^{2^{15}} z_4^{2^{12}} z_5^{2^{10}} z_6^{2^9} z_7^{2^7} z_8^{2^4} z_9^{2^2} z_{10}^2, \\ X_{10} &= w_0^{2^{19}} w_1^{2^{17}} w_2^{2^{16}} w_3^{2^{14}} w_4^{2^{11}} w_5^{2^9} w_6^{2^8} w_7^{2^6} w_8^{2^3} w_9^2 w_{10}. \end{aligned}$$

For variables of the same exponent in the expression of $X_0^{2^{19}} X_8^{2^2} X_9^2 X_{10}$, we consider only the case when they are distinct. For example, $x_9^{2^{20}}$ and $y_0^{2^{20}}$ have the same exponent 2^{20}, so choose distinct x_9 and y_0 to make their product do not have exponent 2^{21}. Among exponents $2^0, 2^1, \ldots, 2^{20}$ appearing in the expression of $X_0^{2^{19}} X_8^{2^2} X_9^2 X_{10}$, there are 2 exponents having only one variable, 15 exponents having two variables, and 4 exponents having three variables. Then the number of such monomials is at least

$$S' = n^2 \times \binom{n}{2}^{15} \times \binom{n}{3}^4 = \frac{n^{21}(n-1)^{19}(n-2)^4}{2^{19} \cdot 3^4} \tag{4.5}$$

which is about $2^{148.12}$ for $n = 16$.

4.3.2 Trivial Linearization

Trivial linearization is to make a system of polynomial equations *linear* by replacing all monomials by new variables, and solve it. Representing the cipher by a system of polynomial equations, we denote S the number of monomials appearing in the system. After making the system linear by replacing all monomials by new variables, we can solve the system using $O(S)$ data and $O(S^\omega)$ time where $2 \leq \omega \leq 3$. A lower bound of S is given in Table 4.3.

One might use guess and determine strategy before trivial linearization to reduce the number of monomials. Let S_g be a lower bound of the number of monomials with guessing g variables, then S_g can be computed using the equations (4.2), (4.3), and (4.4) with n replaced by $n - g$. In this case, the overall time complexity becomes $O(2^{eg} S_g^\omega)$. The log of the time complexity according to the number of rounds and the number of guessed variables are given in Table 4.4.

Another possible approach is using the meet-in-the-middle attack to reduce the number of monomials in the equations. In Section 4.3.1, we have shown that the number of monomials S' in the polynomial of inverse one round is at least 2^{44}, and that of inverse two rounds is at least $2^{148.12}$. Hence it is infeasible to apply the meet-in-the-middle attack to a polynomial of inverse more than one round (even inverting one round is enough for 80-bit security). If the meet-in-the-middle attack is combined with guess and determine strategy, the number of monomials S'_g in the resulting polynomial can be computed by the

equation (4.5) with n replaced by $n-g$. In this case, the overall time complexity becomes $O(2^{eg}(S'_g)^\omega)$, and it is worse than that of plain meet-in-the-middle attack, $O((S')^\omega)$.

It is also possible to represent the encryption function as a polynomial over $\mathbb{F}_2$, not $\mathbb{F}_{2^e}$. In this case, the S-box $x \mapsto x^t$ for $t = 2^\theta + 1$ is of degree 2 in terms of Boolean inputs. We have checked the number of monomials of degree at most 2 after one round using `SageMath` [94]; each word (in $\mathbb{F}_{2^e}$) has all the monomials except 1041 ones, and the entire output contains all the monomials. Hence we expect that all the monomials of degree at most 2^r appear in the output of r-round eHERA, so that the number of monomials grows quickly when the encryption function is viewed as a polynomial over $\mathbb{F}_2$.

4.3.3 Interpolation Attack

To apply interpolation to the r-round eHERA cipher, the interpolating polynomial is considered to take the round constants $\mathbf{rc} = (\mathbf{rc}_0, \ldots, \mathbf{rc}_r) \in (\mathbb{F}_{2^e}^{16})^{r+1}$ generated from XOF as inputs. The number of monomials in the polynomial of the round constants is larger than or equal to that of the key variables (which is given in Section 4.3.1). Assuming that the data limit up to the birthday bound, the recommended number of rounds with respect to this attack is lower than or equal to that for the trivial linearization attack.

When the interpolation attack is used for key-recovery attack, an attacker constructs a polynomial on $r-1$ rounds and guesses the last round key. This approach works when the last round key is generated by a part of the master key. Since eHERA uses the full key material for every round key, such key-recovery attack cannot be applied to eHERA.

Another possible approach is using the meet-in-the-middle attack. As in the trivial linearization, marginal one round is enough to prevent the meet-in-the-middle approach and it is not even necessary for 80-bit security.

KAIST

4.3.4 GCD Attack

To apply GCD attack to a system of multivariate polynomial equations likes eHERA, guessing all the key variables except one is required to make the system univariate. Then guessing requires 2^{15e} repetition of computing GCD, so that eHERA is secure against GCD attack for a security parameter at least 315.

4.3.5 Gröbner Basis Attack.

For the Gröbner basis attack, re-arranging equations may lead to a significant impact on the attack complexity. For example, one may set a system of equations using only plaintext-ciphertext pairs, or set an equation with new variables standing for internal states. The former will be a higher-degree system in a fewer variables, while the latter will be a lower-degree system in more variables. It is also possible to set some equations in the system in the inverse direction, while it is not effective due to the high degree of the inverse S-box.

Equations using Plaintext-Ciphertext Pairs. From a set of nonce-plaintext-ciphertext triples $\{(\mathsf{nc}_i, \mathbf{m}_i, \mathbf{c}_i)\}$, an attacker will be able to establish an over-determined system of equation

$$f_1(k_0, \ldots, k_{15}) = f_2(k_0, \ldots, k_{15}) = \cdots = f_m(k_0, \ldots, k_{15}) = 0$$

r \ g	0	1	2	3	4	5	6	7
1	60.2	79.2	98.0	116.7	135.3	153.7	172.0	190.1
2	174.4	186.9	199.2	211.4	223.3	235.0	246.5	257.7
3	304.0	308.4	312.7	316.8	320.7	324.3	327.7	330.8
4	434.7	431.0	427.1	423.0	418.7	414.2	409.4	404.4
5	565.5	553.6	541.6	529.3	516.8	504.1	491.2	477.9

Table 4.5: Lower bound of log of the time complexity of the Gröbner basis attack for a semi-regular system of the key variables according the the number of rounds r and g guesses.

where $k_i \in \mathbb{F}_{2^e}$ is i-th component of the key variable. The degree of regularity of the system is computed as the degree of the first non-positive coefficient in

$$\left(1 - z^{d_r}\right)^{m-16} \left(\sum_{i=0}^{d_r-1} z^i\right)^{16}$$

where d_r is the degree of eHERA of r rounds. Since the summation does not have any negative term, one easily see that the degree d_{reg} of regularity cannot be smaller than d_r. So, in this case, the time complexity is lower bounded by

$$O\left(\binom{16+d_r}{d_r}^2\right).$$

We have $d_r = t^r$ only until $t^r < 2^e - 1$, which is equivalent to $r \leq 5$. For $r > 5$, the degree is at least d_5. If one uses the hybrid approach with g guesses of the key variables, then the time complexity is lower bounded by

$$O\left(2^{eg}\binom{16-g+d_r}{d_r}^2\right).$$

Table 4.5 shows lower bounds of log of the time complexity according to the number r of rounds and the number g of guessed variables.

Equations using Internal States. Let $m = 16rk$ and $n = 16(r-1)k + 16$ where k is the block length of each query. By using the internal states as new variables, one can establish a system of m equations of degree t in n variables. Then, the complexity is

$$O\left(\binom{16(r-1)k + 16 + d_{reg}(r,k)}{d_{reg}(r,k)}^{\omega}\right) \tag{4.6}$$

where $d_{reg}(r,k)$ is the degree of regularity. When $k = 1$, the system is regular as $m = n$ so that we can compute $d_{reg}(r,k)$ using the Macaulay bound. For $k > 1$, assuming semi-regularity, we can compute $d_{reg}(r,k)$ by finding the degree of the first non-positive coefficient on the Hilbert series. We represent $d_{reg}(r,k)$ and the corresponding time complexity in Table 4.6.

Suppose that one takes the hybrid approach. As the degree of regularity differs with a huge gap between $m = n$ and $m = n + 1$, we arrange the degrees $d'_{reg}(r,g)$ of regularity in Table 4.7a when $k = 1$ and the number g of guesses is positive, and compute the time complexity in Table 4.7b using the equation

$$O\left(2^{eg}\binom{16r - g + d'_{reg}(r,g)}{d'_{reg}(r,g)}^{\omega}\right). \tag{4.7}$$

r \ k	1	2	3	4	5
1	257	83	71	65	61
2	513	262	309	362	417
3	769	459	583	713	844
4	1025	666	874	1089	1306

(a) $d_{reg}(r, k)$

r \ k	1	2	3	4	5
1	169.2	119.9	113.4	109.8	107.2
2	343.8	377.5	485.0	594.4	704.6
3	518.8	644.4	876.3	1110	1344
4	693.9	915.8	1276	1638	2000

(b) Log of the time complexity

Table 4.6: $d_{reg}(r, k)$ for a semi-regular system of $16rk$ equations of degree t in $16(r-1)k+16$ variables and log of the time complexity of the Gröbner basis attack for the system with $w = 2$.

r \ g	1	2	3	4	5
1	129	109	95	84	74
2	257	229	209	193	179
3	385	351	327	307	289
4	513	474	446	422	402

(a) $d'_{reg}(r, g)$

r \ g	1	2	3	4	5
1	153.4	161.5	171.4	182.3	193.6
2	297.4	302.5	310.2	319.1	328.7
3	441.7	444.8	450.9	458.3	466.4
4	586.3	587.6	592.3	598.2	605.5

(b) Log of the time complexity

Table 4.7: $d'_{reg}(r, g)$ for a semi-regular system of $16r$ equations of degree t in $16r$ variables with g guesses and log of the time complexity of the Gröbner basis attack for the system with $w = 2$.

KAIST

Equations in the Inverse Direction. When constructing a system of equations with respect to the key variables, an attacker may use the meet-in-the-middle approach by inverting some rounds. In this case, each equation has degree at least $t' = 740171$, which is the degree of the inverse S-box (see Section 4.3.1), and the time complexity is lower bounded by

$$O\left(2^{eg}\binom{16-g+t'}{t'}^2\right)$$

when g variables are guessed. It is larger than 2^{256} for all $0 \le g \le 16$.

Now consider a system of m equations in n variables where $m = 16rk$ and $n = 16(r-1)k+16$ as the above case, except that some of the equations are established in the inverse direction. Suppose a equations among the m equations in the system are of degree t'. When $k = 1$, the degree of regularity becomes $1 + (m-a)(t-1) + a(t'-1)$ from the Macaulay bound. When $k > 1$, the degree of regularity can be computed by the degree of the first non-positive coefficient in the Hilbert series

$$\frac{1}{(1-z)^n}(1-z^t)^{m-a}(1-z^{t'})^a.$$

If $m - a \le n$ then we can represent the above series as

$$(1-z^{t'})^{m-n}\left(\sum_{i=0}^{t-1} z^i\right)^{m-a}\left(\sum_{j=0}^{t'-1} z^j\right)^{n-(m-a)},$$

so that the degree of regularity is at least t'. Otherwise we can represent the above series as

$$(1-z^t)^{m-a-n}(1-z^{t'})^{m-a}\left(\sum_{i=0}^{t-1} z^i\right)^n.$$

The degree of the first non-negative coefficient of the above polynomial is at least that of

$$(1-z^t)^{m-a-n}\left(\sum_{i=0}^{t-1} z_i\right)^n.$$

We find that all the degrees are minimized when $a = 0$ so that inverting some equations does not work effectively. It is also true for the hybrid approach.

4.3.6 Linear Cryptanalysis

Unlike differential cryptanalysis, it might be possible to apply linear cryptanalysis to eHERA as it is known-plaintext attack. There are two possible choices for the input variables of linear cryptanalysis: the XOF output and the key variables. The former one seems infeasible to use since the probability of linear trails of eHERA depends on the key value, which is multiplied to the XOF output.

In the second case, the attack is reduced to solving an LPN-like problem as follows; given pairs $(\mathsf{nc}_i, \mathbf{y}_i)$ such that $\mathsf{eHERA}(\mathbf{k}, \mathsf{nc}_i) = \mathbf{y}_i$, one can establish

$$\langle \mathbf{b}, \mathbf{y}_i \rangle = \langle \mathbf{a}, \mathbf{k} \rangle + e_i$$

for some vectors $\mathbf{a} \neq 0$, $\mathbf{b} \in \mathbb{F}_{2^e}$ and error e_i sampled according to a certain distribution χ. The linear probability of r-round eHERA is upper bounded by $(\mathrm{LP}^S)^{B_\ell \cdot \lfloor \frac{r}{2} \rfloor}$, where LP^S and B_ℓ denote the linear probability of the S-box in Exp_t and the (linear) branch number of the linear layer, respectively. Therefore, the data complexity for linear cryptanalysis is lower bounded approximately by $1/(\mathrm{LP}^S)^{B_\ell \cdot \lfloor \frac{r}{2} \rfloor}$. We choose eHERA parameters to make its S-box be an almost bent function [26], so that LP^S is upper bounded by 2^{-e+1}. The branch number of the linear layer of eHERA is the same as that of HERA, which is 8 (see Appendix A.1 for the proof), we can conclude that r-round eHERA provides λ-bit security against linear cryptanalysis when

$$(2^{e-1})^{8 \cdot \lfloor \frac{r}{2} \rfloor} > 2^\lambda.$$

4.3.7 Differential Cryptanalysis

To apply differential cryptanalysis, an attack should be able to control differences of inputs. It is possible to choose nonces adversarially, but it is impossible to control output of the XOF fed with the chosen nonces as the XOF is modeled as a random oracle. Hence we believe that eHERA will be secure against any type of chosen-plaintext attack such as (higher-order) differential, truncated differential, invariant subspace trail and cube attacks.

Nonetheless, considering an unsuspected differential-related attack we show that eHERA has small enough differential characteristic in the following. Resistance of a substitution-permutation cipher against differential cryptanalysis is typically estimated by the maximum probability of differential trails [20]. Just like the linear cryptanalysis, the S-box of eHERA is almost perfectly nonlinear so that its differential probability DP^S is upper bounded by 2^{-e+1} [26]. Since the branch number of the linear layer of eHERA is 8 (see Appendix A.1), we can conclude that r-round eHERA provides λ-bit security against differential

cryptanalysis when

$$(2^{e-1})^{8\cdot\lfloor\frac{r}{2}\rfloor} > 2^{\lambda}.$$

This inequality is exactly the same as the inequality for linear cryptanalysis.

4.4 Performance Evaluation of the BtE Framework

Environment. In this section, we evaluate the performance of the BtE framework combined with the eHERA cipher in terms of encryption speed. The source codes developed in C++17, using GNU C++ 7.5.0 compiler with AVX2 instruction set. The source codes of server-side computation are developed with `HElib` library [65] which implements the BGV scheme. XOF is instantiated with BLAKE3 in `BLAKE3` library [88]. Our experiments are done in AMD Ryzen 7 2700X @ 3.70 GHz single-threaded with 64 GB memory.

4.4.1 Benchmark

We measure the performance of the BtE framework combined with eHERA, distinguishing two different parts: the client-side and the server-side. On the client-side, the latency includes time for generating pseudorandom numbers (needed to generate a single keystream in $\mathbb{F}_{2^e}^{16}$), keystream generation from eHERA, and vector addition over $\mathbb{F}_{2^e}$. The extendable output function is instantiated with BLAKE3.

Cipher	Client		Server	
	Latency (Cycles)	Throughput (C/B)	Latency (s)	Throughput (B/s)
eHERA-4	4635	110.4	48.79	265.0
eHERA-6	6310	150.2	76.74	225.0
eHERA-7	7152	170.3	88.61	217.5
eHERA-8	7942	189.1	115.0	178.4

Table 4.8: The client-side and the server-side performance of the BtE framework equipped with eHERA.

The server-side latency includes time for the randomized key schedule, homomorphic evaluation of the keystreams from eHERA, the homomorphic subtraction, and digit extraction. We measure the latency until the first (digit-extracted) HE-ciphertext comes out, while the throughput is measured until all the 16 HE-ciphertexts come out. We note that our evaluation does not take into account key encryption since the encrypted key will be used over multiple sessions once it is computed. For the same reason, the initialization process of the HE schemes is not considered. We summarize our implementation results in Table 4.8.

4.4.2 Comparison to Other Binary Ciphers

We compare the BtE framework with eHERA to other HE-friendly ciphers over $\mathbb{Z}_2$ in Table 4.9. Other ciphers are implemented in `HHE` (hybrid homomorphic encryption) library[2] which is a contribution of [43]. We use the plain (client-side) and `HElib` implementations of the ciphers. All the plain implementations in the `HHE` library is implemented by C++ without SIMD instructions.

[2] https://github.com/IAIK/hybrid-HE-framework

In Table 4.9, we implement 3 versions of eHERA for the fair comparison: SHAKE128, BLAKE3, AVX2. As Rasta, Agrasta, and Dasta implemented in [43] use SHAKE128, we give the performance of eHERA-6 using SHAKE128 without SIMD instructions. BLAKE3 implies eHERA-6 using BLAKE3 without SIMD instructions. Finally, AVX2 implies eHERA-6 using BLAKE3 with AVX2 instructions. For the server-side performance, we just evaluate BLAKE3-version since XOF does not make a significant difference on the performance.

Cipher	Client		Server		Notes
	Latency (Cycles)	Throughput (C/B)	Latency (s)	Throughput (B/s)	
LowMC	3.713×10^8	2.303×10^7	1242	0.026	
Rasta-5-r	2.648×10^8	4.035×10^6	294.1	0.223	SHAKE128
Rasta-5-c	-	-	41.43	1.059	
Rasta-6-r	6.740×10^7	1.536×10^6	215.4	0.204	SHAKE128
Rasta-6-c	-	-	55.66	0.788	
Dasta-5-r	2.450×10^6	3.734×10^4	328.2	0.200	SHAKE128
Dasta-5-c	-	-	46.10	1.426	
Dasta-6-r	1.113×10^6	2.536×10^4	243.8	0.180	SHAKE128
Dasta-6-c	-	-	60.83	0.722	
Agrasta-r	4.041×10^6	2.506×10^5	48.66	0.332	SHAKE128
Agrasta-c	-	-	20.48	0.792	
Kreyvium	6.583×10^4	5.266×10^5	483.9	0.0003	
Kreyvium-12	6.543×10^4	1.138×10^4	486.4	0.012	
Kreyvium-13	7.375×10^4	4720	553.4	0.028	
FiLIP-1216	4.914×10^7	6.143×10^6	1371	0.006	
FiLIP-1280	1.331×10^7	1.664×10^6	2178	0.004	
eHERA-6	2.419×10^4	576.0	-	-	SHAKE128
eHERA-6	2.062×10^4	491.0	76.74	225.0	BLAKE3
eHERA-6	6310	150.2	-	-	AVX2

Table 4.9: Performance comparison between HE-friendly ciphers over $\mathbb{Z}_2$. The choice of parameters can be found in Table 4.10.

Figure 4.4 shows their client-side and server-side performance in terms of throughput and latency. Throughput and latency in the client-side are measured in terms of cycles per byte (C/B) and the number of cycles, respectively, all in the logarithmic scale. In case of server-side, throughput and latency are measured in terms of bytes per second (B/s) and time (seconds), respectively, all in the logarithmic scale.

We organize and explain about the parameter selection of Table 4.9. We once more remark that we use the parameters in `HHE` library [43]. Unlike [43], we present packed implementations[3] of Rasta, Agrasta and Dasta. We marked as suffixes '-r' and '-c', which means row-wise packing and column-wise packing, respectively.

After the proposition of Kreyvium, all the HE-friendly ciphers have been proposed as stream ciphers. Nonetheless, since Rasta, Agrasta and Dasta have similar design to block ciphers, we call the bit-length of output (which is same as the key size) blocksize. In the column of blocksize, the asterisked items are the bit-length of output keystream which is benchmarked. For this reason, the latency of FiLIP for a single bit will be roughly 64 times less than in Table 4.9.

For the number of slots, there are two numbers per single item. The first ones are nominal number of slots; it is just the maximal number of slots for this parameter. The second ones in the parentheses

[3] They are also implemented in `HHE` library.

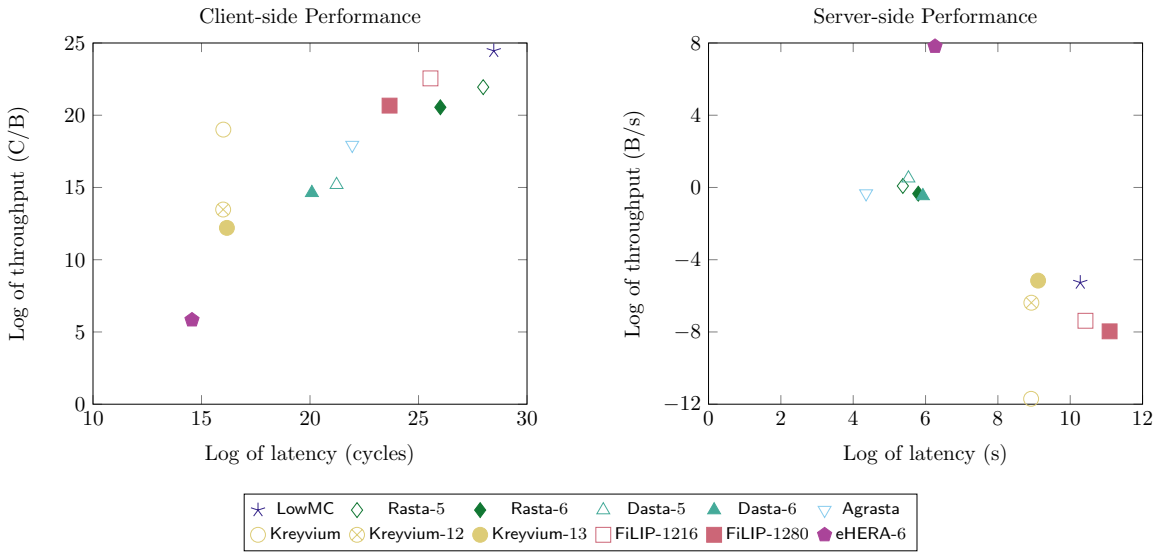


Figure 4.4: Client-side and server-side performances of HE-friendly ciphers.

are the effective number of slots; we use those number of slots in real computation. For the case of LowMC, homomorphic computation with (row-wise) batching will never lose efficiency than now so that the throughput can be multiplied by the nominal number of slots. However, for the case of Rasta, Dasta, Agrasta, and FiLIP, utilizing the full slots is quite costly because of their random linear layers.

Cipher	Symmetric key Parameters			HE Parameters				
	Blocksize (* Output)	Key Size	Round	M	λ'	Slot	Ctxt. Mod.	Remaining Budget
LowMC	256	128	14	23377	110	480(1)	456	13
Rasta-5-r	525	525	5	11441	110	224(1)	216	27
Rasta-5-c	525	525	5	15709	113	682(525)	275	5
Rasta-6-r	351	351	6	11441	110	224(1)	216	5
Rasta-6-c	351	351	6	15709	100	682(351)	324	4
Dasta-5-r	525	525	5	10752	110	224(1)	216	27
Dasta-5-c	525	525	5	15709	113	682(525)	275	5
Dasta-6-r	351	351	6	10752	110	224(1)	216	16
Dasta-6-c	351	351	6	11441	100	682(351)	324	4
Agrasta-r	129	129	4	10261	117	330(1)	171	17
Agrasta-c	129	129	4	15709	115	682(129)	270	51
Kreyvium	1*	128	-	14351	107	504(1)	270	18
Kreyvium-12	46*	128	-	14351	107	504(1)	270	18
Kreyvium-13	125*	128	-	15709	113	682(1)	275	2
FiLIP-1216	64*	16384	-	5461	113	378(1)	108	19
FiLIP-1280	64*	4096	-	8435	119	240(1)	112	2

Table 4.10: Parameters used in the comparison. These parameters are originally used in [43].

KAIST

Chapter 5. Conclusion

Over a decade, the field of homomorphic encryption have been rapidly matured in both theory and practice. Nevertheless, there remains a gap between real-world applications and the current HE technology. In this dissertation, we presented some efficient solutions to close the gap: transciphering frameworks and HE-friendly ciphers. We showed that some applications of HE, such as secure outsourcing computation and secure data collection, become realistic when combining with symmetric cryptography.

In Chapter 3, we introduced a transciphering framework for real numbers, dubbed RtF transciphering framework, which is a combination of CKKS and FV. To realize its concrete practicality, we proposed HE-friendly ciphers HERA and Rubato. We designed HERA not for theoretical efficiency but for practical efficiency by investigating cost measures other than multiplicative depth. Rubato has been built upon a novel design strategy: noise reduces multiplicative depth.

In Chapter 4, we presented a framework for efficient transciphering of binary numbers, dubbed BtE transciphering framework, using extension field structure. To compare concrete efficiency with previous HE-friendly ciphers, we proposed an extension field variant eHERA of HERA. We found out that the BtE transciphering framework combined with eHERA requires way less costs compared to previous ciphers.

Future Work. We list up some future works in this section.

- The RtF framework requires HalfBoot at the last part. As this operation runs online and costs a lot, removing HalfBoot from the RtF framework will make it significantly more efficient.
- The $\mathsf{SlotToCoeff}^{\mathsf{FV}}$ operation can be computed in a various way according to computing resources. For an extreme optimization, one can investigate the optimal way and required resources to compute $\mathsf{SlotToCoeff}^{\mathsf{FV}}$.
- As all of our ciphers contain the generation of random bits, the client-side time complexity is comparatively worse than conventional symmetric ciphers. If one can reduce the required randomness (less random bits or weak randomness) without the increase of multiplicative depth, we can expect a cipher which is efficient on the both sides.
- We presented a noisy cipher Rubato, which is a combination of symmetric cipher and LWE encryption. We think that the LWE encryption is just one of examples, and expect a hybrid construction from any symmetric and public key cryptography. For example, if the rounding part in LWR (learning with rounding) can be efficiently computed homomorphically, a hybrid construction from symmetric cipher and LWR can be one of the choices.

KAIST

Bibliography

[1] Martin Albrecht et al. "MiMC: Efficient Encryption and Cryptographic Hashing with Minimal Multiplicative Complexity". In: *Advances in Cryptology – ASIACRYPT 2016*. Ed. by Jung Hee Cheon and Tsuyoshi Takagi. Vol. 10031. Springer, 2016, pp. 191–219.

[2] Martin Albrecht et al. "On the Complexity of the Arora-Ge Algorithm against LWE". In: *SCC 2012 – Third international conference on Symbolic Computation and Cryptography*. July 2012, pp. 93–99.

[3] Martin R. Albrecht, Robert Fitzpatrick, and Florian Göpfert. "On the Efficacy of Solving LWE by Reduction to Unique-SVP". In: *Information Security and Cryptology – ICISC 2013*. Ed. by Hyang-Sook Lee and Dong-Guk Han. Vol. 8565. Springer, 2014, pp. 293–310.

[4] Martin R. Albrecht et al. "Ciphers for MPC and FHE". In: *Advances in Cryptology – EUROCRYPT 2015*. Ed. by Elisabeth Oswald and Marc Fischlin. Vol. 9056. Springer, 2015, pp. 430–454.

[5] Martin R. Albrecht et al. "Lattice Reduction with Approximate Enumeration Oracles". In: *Advances in Cryptology – CRYPTO 2021*. Ed. by Tal Malkin and Chris Peikert. Vol. 12826. Springer, 2021, pp. 732–759.

[6] Martin R. Albrecht et al. "On the Complexity of the BKW Algorithm on LWE". In: *Designs, Codes and Cryptography* 74.2 (Feb. 2015), pp. 325–354.

[7] Martin R. Albrecht et al. "Revisiting the Expected Cost of Solving uSVP and Applications to LWE". In: *Advances in Cryptology – ASIACRYPT 2017*. Ed. by Tsuyoshi Takagi and Thomas Peyrin. Vol. 10624. Springer, 2017, pp. 297–322.

[8] Erdem Alkim et al. "Post-Quantum Key Exchange: A New Hope". In: *Proceedings of the 25th USENIX Conference on Security Symposium*. SEC'16. Austin, TX, USA: USENIX Association, 2016, pp. 327–343.

[9] Abdelrahaman Aly et al. "Design of Symmetric-Key Primitives for Advanced Cryptographic Protocols". In: *IACR Transactions on Symmetric Cryptology* 2020.3 (Sept. 2020).

[10] Sanjeev Arora and Rong Ge. "New Algorithms for Learning in Presence of Errors". In: *Automata, Languages and Programming*. Ed. by Luca Aceto, Monika Henzinger, and Jiří Sgall. Vol. 6755. Springer, 2011, pp. 403–415.

[11] Tomer Ashur and Siemen Dhooghe. *MARVELlous: a STARK-Friendly Family of Cryptographic Primitives*. IACR Cryptology ePrint Archive, Report 2018/1098. https://eprint.iacr.org/2018/1098. 2018.

[12] Shi Bai and Steven D. Galbraith. "Lattice Decoding Attacks on Binary LWE". In: *Information Security and Privacy – ACISP 2014*. Ed. by Willy Susilo and Yi Mu. Vol. 8544. Springer, 2014, pp. 322–337.

[13] Thomas Baignères, Jacques Stern, and Serge Vaudenay. "Linear Cryptanalysis of Non Binary Ciphers". In: *Selected Areas in Cryptography – SAC 2007*. Ed. by Carlisle Adams, Ali Miri, and Michael Wiener. Vol. 4876. Springer, 2007, pp. 184–211.

KAIST

[14] Achiya Bar-On et al. “Cryptanalysis of SP Networks with Partial Non-Linear Layers”. In: *Advances in Cryptology – EUROCRYPT 2015*. Ed. by Elisabeth Oswald and Marc Fischlin. Vol. 9056. Springer, 2015, pp. 315–342.

[15] Magali Bardet, Jean-Charles Faugère, and Bruno Salvy. “Asymptotic Behaviour of the Index of Regularity of Semi-Regular Quadratic Polynomial Systems”. In: *MEGA 2005 - 8th International Symposium on Effective Methods in Algebraic Geometry*. 2005.

[16] Anja Becker et al. “New Directions in Nearest Neighbor Searching with Applications to Lattice Sieving”. In: *Proceedings of the twenty-seventh annual ACM-SIAM symposium on Discrete algorithms*. SIAM. 2016, pp. 10–24.

[17] Luk Bettale, Jean-Charles Faugere, and Ludovic Perret. “Hybrid Approach for Solving Multivariate Systems over Finite Fields”. In: *Journal of Mathematical Cryptology* 3.3 (2009), pp. 177–197.

[18] Luk Bettale, Jean-Charles Faugère, and Ludovic Perret. “Solving Polynomial Systems over Finite Fields: Improved Analysis of the Hybrid Approach”. In: *Proceedings of the 37th International Symposium on Symbolic and Algebraic Computation*. ISSAC ’12. New York, NY, USA: Association for Computing Machinery, 2012.

[19] Tim Beyne et al. “Out of Oddity – New Cryptanalytic Techniques Against Symmetric Primitives Optimized for Integrity Proof Systems”. In: *Advances in Cryptology – CRYPTO 2020*. Ed. by Daniele Micciancio and Thomas Ristenpart. Vol. 12172. Springer, 2020, pp. 299–328.

[20] Eli Biham and Adi Shamir. “Differential Cryptanalysis of DES-like Cryptosystems”. In: *Advances in Cryptology – CRYPTO ’90*. Ed. by Alfred J. Menezes and Scott A. Vanstone. Vol. 537. Springer, 1991, pp. 2–21.

[21] Avrim Blum, Adam Kalai, and Hal Wasserman. “Noise-Tolerant Learning, the Parity Problem, and the Statistical Query Model”. In: *J. ACM* 50.4 (July 2003), pp. 506–519.

[22] Jean-Philippe Bossuat et al. “Efficient Bootstrapping for Approximate Homomorphic Encryption with Non-sparse Keys”. In: *Advances in Cryptology – EUROCRYPT 2021*. Ed. by Anne Canteaut and François-Xavier Standaert. Vol. 12696. Springer, 2021, pp. 587–617.

[23] Christina Boura et al. “Simulating Homomorphic Evaluation of Deep Learning Predictions”. In: *Cyber Security Cryptography and Machine Learning*. Ed. by Shlomi Dolev et al. Vol. 11527. Springer, 2019, pp. 212–230.

[24] Zvika Brakerski. “Fully Homomorphic Encryption without Modulus Switching from Classical GapSVP”. In: *Advances in Cryptology – CRYPTO 2012*. Ed. by Reihaneh Safavi-Naini and Ran Canetti. Vol. 7417. Springer, 2012, pp. 868–886.

[25] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. “(Leveled) Fully Homomorphic Encryption without Bootstrapping”. In: *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*. Cambridge, Massachusetts: ACM, 2012, pp. 309–325. ISBN: 9781450311151.

[26] Anne Canteaut, Pascale Charpin, and Hans Dobbertin. “A New Characterization of Almost Bent Functions”. In: *Fast Software Encryption – FSE ’99*. Ed. by Lars Knudsen. Springer, 1999, pp. 186–200.

[27] Anne Canteaut et al. “Stream ciphers: A Practical Solution for Efficient Homomorphic-Ciphertext Compression”. In: *Journal of Cryptology* 31.3 (2018), pp. 885–916.

KAIST

[28] Leonard Carlitz and Saburo Uchiyama. “Bounds for Exponential Sums”. In: *Duke mathematical Journal* 24.1 (1957), pp. 37–41.

[29] Melissa Chase et al. “Post-Quantum Zero-Knowledge and Signatures from Symmetric-Key Primitives”. In: *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*. 2017, pp. 1825–1842.

[30] Hao Chen and Kyoohyung Han. “Homomorphic Lower Digits Removal and Improved FHE Bootstrapping”. In: *Advances in Cryptology – EUROCRYPT 2018*. Ed. by Jesper Buus Nielsen and Vincent Rijmen. Springer, 2018, pp. 315–337.

[31] Yuanmi Chen. “Réduction de Réseau et Sécurité Concrète du Chiffrement Complètement Homomorphe”. Thèse de doctorat dirigée par Nguyen, Phong-Quang Informatique Paris 7 2013. PhD thesis. 2013, 1 vol. (133 p.)

[32] Yuanmi Chen and Phong Q. Nguyen. “BKZ 2.0: Better Lattice Security Estimates”. In: *Advances in Cryptology – ASIACRYPT 2011*. Ed. by Dong Hoon Lee and Xiaoyun Wang. Vol. 7073. Springer, 2011, pp. 1–20.

[33] Jung Hee Cheon, Miran Kim, and Kristin Lauter. “Homomorphic Computation of Edit Distance”. In: *Financial Cryptography and Data Security*. Ed. by Michael Brenner et al. Vol. 8976. Springer, 2015, pp. 194–212.

[34] Jung Hee Cheon et al. “Bootstrapping for Approximate Homomorphic Encryption”. In: *Advances in Cryptology – EUROCRYPT 2018*. Ed. by Jesper Buus Nielsen and Vincent Rijmen. Vol. 10820. Springer, 2018, pp. 360–384.

[35] Jung Hee Cheon et al. “Homomorphic Encryption for Arithmetic of Approximate Numbers”. In: *Advances in Cryptology – ASIACRYPT 2017*. Ed. by Tsuyoshi Takagi and Thomas Peyrin. Vol. 10624. Springer, 2017, pp. 409–437.

[36] Jung Hee Cheon et al. “Privacy-Preserving Computations of Predictive Medical Models with Minimax Approximation and Non-Adjacent Form”. In: *Financial Cryptography and Data Security*. Ed. by Michael Brenner et al. Vol. 10323. Springer, 2017, pp. 53–74.

[37] Ilaria Chillotti et al. “TFHE: Fast Fully Homomorphic Encryption Over the Torus”. In: *Journal of Cryptology* 33.1 (2020), pp. 34–91.

[38] Joan Daemen. “Cipher and Hash Function Design Strategies Based on Linear and Differential Cryptanalysis”. In: *Doctoral Dissertation, KU Leuven* (1995).

[39] Christophe De Cannière and Bart Preneel. “Trivium”. In: *New Stream Cipher Designs: The eSTREAM Finalists*. Ed. by Matthew Robshaw and Olivier Billet. Vol. 4986. Springer, 2008, pp. 244–266.

[40] Itai Dinur et al. “Optimized Interpolation Attacks on LowMC”. In: *Advances in Cryptology – ASIACRYPT 2015*. Ed. by Tetsu Iwata and Jung Hee Cheon. Vol. 9453. Springer, 2015, pp. 535–560.

[41] Christoph Dobraunig, Maria Eichlseder, and Florian Mendel. “Higher-Order Cryptanalysis of LowMC”. In: *Information Security and Cryptology – ICISC 2015*. Ed. by Soonhak Kwon and Aaram Yun. Vol. 9558. Springer, 2016, pp. 87–101.

KAIST

[42] Christoph Dobraunig et al. “Ciminion: Symmetric Encryption Based on Toffoli-Gates over Large Finite Fields”. In: *Advances in Cryptology – EUROCRYPT 2021*. Ed. by Anne Canteaut and François-Xavier Standaert. Vol. 12697. Springer, 2021, pp. 3–34.

[43] Christoph Dobraunig et al. *Pasta: A Case for Hybrid Homomorphic Encryption*. Cryptology ePrint Archive, Report 2021/731. https://ia.cr/2021/731. 2021.

[44] Christoph Dobraunig et al. “Rasta: A Cipher with Low ANDdepth and Few ANDs per Bit”. In: *Advances in Cryptology – CRYPTO 2018*. Ed. by Hovav Shacham and Alexandra Boldyreva. Vol. 10991. Springer, 2018, pp. 662–692.

[45] Yarkın Doröz et al. “Toward Practical Homomorphic Evaluation of Block Ciphers Using Prince”. In: *Financial Cryptography and Data Security*. Ed. by Rainer Böhme et al. Vol. 8438. Springer, 2014, pp. 208–220.

[46] Léo Ducas and Daniele Micciancio. “FHEW: Bootstrapping Homomorphic Encryption in Less Than a Second”. In: *Advances in Cryptology – EUROCRYPT 2015*. Ed. by Elisabeth Oswald and Marc Fischlin. Vol. 9056. Springer, 2015, pp. 617–640.

[47] Sébastien Duval, Virginie Lallemand, and Yann Rotella. “Cryptanalysis of the FLIP Family of Stream Ciphers”. In: *Advances in Cryptology – CRYPTO 2016*. Ed. by Matthew Robshaw and Jonathan Katz. Vol. 9814. Springer, 2016, pp. 457–475.

[48] Morris J Dworkin. *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*. Tech. rep. National Institute of Standards and Technology, 2015.

[49] Junfeng Fan and Frederik Vercauteren. *Somewhat Practical Fully Homomorphic Encryption*. IACR Cryptology ePrint Archive, Report 2012/144. https://eprint.iacr.org/2012/144. 2012.

[50] Ralf Fröberg. “An Inequality for Hilbert Series of Graded Algebras.” In: *MATHEMATICA SCANDINAVICA* 56 (Dec. 1985).

[51] Nicolas Gama and Phong Q. Nguyen. “Predicting Lattice Reduction”. In: *Advances in Cryptology – EUROCRYPT 2008*. Ed. by Nigel Smart. Springer, 2008, pp. 31–51.

[52] Nicolas Gama, Phong Q. Nguyen, and Oded Regev. “Lattice Enumeration Using Extreme Pruning”. In: *Advances in Cryptology – EUROCRYPT 2010*. Ed. by Henri Gilbert. Springer, 2010, pp. 257–278.

[53] Craig Gentry. “Fully Homomorphic Encryption Using Ideal Lattices”. In: *Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing – STOC 2009*. ACM, 2009, pp. 169–178.

[54] Craig Gentry, Shai Halevi, and Nigel P. Smart. “Better Bootstrapping in Fully Homomorphic Encryption”. In: *Public Key Cryptography – PKC 2012*. Ed. by Marc Fischlin, Johannes Buchmann, and Mark Manulis. Vol. 7293. Springer, 2012, pp. 1–16.

[55] Craig Gentry, Shai Halevi, and Nigel P. Smart. “Homomorphic Evaluation of the AES Circuit”. In: *Advances in Cryptology – CRYPTO 2012*. Ed. by Reihaneh Safavi-Naini and Ran Canetti. Vol. 7417. Springer, 2012, pp. 850–867.

[56] Craig Gentry, Amit Sahai, and Brent Waters. “Homomorphic Encryption from Learning with Errors: Conceptually-Simpler, Asymptotically-Faster, Attribute-Based”. In: *Advances in Cryptology – CRYPTO 2013*. Ed. by Ran Canetti and Juan A. Garay. Vol. 8042. Springer, 2013, pp. 75–92.

KAIST

[57] Florian Göpfert. “Securely Instantiating Cryptographic Schemes Based on the Learning with Errors Assumption”. PhD thesis. Darmstadt: Technische Universität, 2016.

[58] Lorenzo Grassi, Christian Rechberger, and Markus Schofnegger. “Proving Resistance Against Infinitely Long Subspace Trails: How to Choose the Linear Layer”. In: *IACR Transactions on Symmetric Cryptology* 2021.2 (June 2021), pp. 314–352.

[59] Lorenzo Grassi et al. “MPC-Friendly Symmetric Key Primitives”. In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security – CCS 2016*. Vienna, Austria: ACM, 2016, pp. 430–443. ISBN: 9781450341394.

[60] Jian Guo, Thomas Peyrin, and Axel Poschmann. “The PHOTON Family of Lightweight Hash Functions”. In: *Advances in Cryptology – CRYPTO 2011*. Ed. by Phillip Rogaway. Springer, 2011, pp. 222–239.

[61] J. Ha et al. “Masta: An HE-Friendly Cipher Using Modular Arithmetic”. In: *IEEE Access* 8 (2020), pp. 194741–194751.

[62] Shai Halevi and Victor Shoup. “Bootstrapping for HElib”. In: *Advances in Cryptology – EUROCRYPT 2015*. Ed. by Elisabeth Oswald and Marc Fischlin. Vol. 9056. Springer, 2015, pp. 641–670.

[63] Kyoohyung Han and Dohyeong Ki. “Better Bootstrapping for Approximate Homomorphic Encryption”. In: *Topics in Cryptology – CT-RSA 2020*. Ed. by Stanislaw Jarecki. Vol. 12006. Springer, 2020, pp. 364–390.

[64] Phil Hebborn and Gregor Leander. “Dasta – Alternative Linear Layer for Rasta”. In: *IACR Transactions on Symmetric Cryptology* 2020.3 (Sept. 2020), pp. 46–86.

[65] *HElib (release 2.1.0)*. https://github.com/homenc/HElib. July 2021.

[66] Clément Hoffmann, Pierrick Méaux, and Thomas Ricosset. “Transciphering, Using FiLIP and TFHE for an Efficient Delegation of Computation”. In: *Progress in Cryptology – INDOCRYPT 2020*. Ed. by Karthikeyan Bhargavan, Elisabeth Oswald, and Manoj Prabhakaran. Springer, 2020, pp. 39–61.

[67] Seokhie Hong et al. “Provable Security against Differential and Linear Cryptanalysis for the SPN Structure”. In: *Fast Software Encryption – FSE 2000*. Ed. by Gerhard Goos et al. Vol. 1978. Springer, 2001.

[68] Kai Hu et al. *Massive Superpoly Recovery with Nested Monomial Predictions*. Cryptology ePrint Archive, Report 2021/1225. https://ia.cr/2021/1225, to appear in Asiacrypt 2021. 2021.

[69] Thomas Jakobsen and Lars R. Knudsen. “The Interpolation Attack on Block Ciphers”. In: *Fast Software Encryption – FSE ’97*. Ed. by Eli Biham. Vol. 1267. Springer, 1997, pp. 28–40.

[70] Jérémy Jean, Ivica Nikolić, and Thomas Peyrin. “Tweaks and Keys for Block Ciphers: The TWEAKEY Framework”. In: *Advances in Cryptology – ASIACRYPT 2014*. Ed. by Palash Sarkar and Tetsu Iwata. Vol. 8874. Springer, 2014, pp. 274–288.

[71] Chiraag Juvekar, Vinod Vaikuntanathan, and Anantha Chandrakasan. “GAZELLE: A Low Latency Framework for Secure Neural Network Inference”. In: *Proceedings of the 27th USENIX Conference on Security Symposium*. Baltimore, MD, USA: USENIX Association, 2018, pp. 1651–1668. ISBN: 9781931971461.

KAIST

[72] Nathan Keller and Asaf Rosemarin. “Mind the Middle Layer: The HADES Design Strategy Revisited”. In: *Advances in Cryptology – EUROCRYPT 2021*. Ed. by Anne Canteaut and François-Xavier Standaert. Vol. 12697. Springer, 2021, pp. 35–63.

[73] Abhishek Kesarwani et al. “New Cube Distinguishers on NFSR-based Stream Ciphers”. In: *Designs, Codes and Cryptography* 88.1 (Jan. 2020), pp. 173–199.

[74] Thijs Laarhoven. “Search Problems in Cryptography: from Fingerprinting to Lattice Sieving”. English. Proefschrift. PhD thesis. Mathematics and Computer Science, Feb. 2016.

[75] Joon-Woo Lee et al. “High-Precision Bootstrapping of RNS-CKKS Homomorphic Encryption Using Optimal Minimax Polynomial Approximation and Inverse Sine Function”. In: *Advances in Cryptology – EUROCRYPT 2021*. Ed. by Anne Canteaut and François-Xavier Standaert. Vol. 12696. Springer, 2021, pp. 618–647.

[76] Tancrède Lepoint and Michael Naehrig. “A Comparison of the Homomorphic Encryption Schemes FV and YASHE”. In: *Progress in Cryptology – AFRICACRYPT 2014*. Ed. by David Pointcheval and Damien Vergnaud. Vol. 8469. Springer, 2014, pp. 318–335.

[77] Richard Lindner and Chris Peikert. “Better Key Sizes (and Attacks) for LWE-Based Encryption”. In: *Topics in Cryptology – CT-RSA 2011*. Ed. by Aggelos Kiayias. Vol. 6558. Springer, 2011, pp. 319–339.

[78] Fukang Liu et al. *Algebraic Attacks on Rasta and Dasta Using Low-Degree Equations*. Cryptology ePrint Archive, Report 2021/474. https://ia.cr/2021/474, to appear in Asiacrypt 2021. 2021.

[79] W. Lu et al. “PEGASUS: Bridging Polynomial and Non-polynomial Evaluations in Homomorphic Encryption”. In: *2021 2021 IEEE Symposium on Security and Privacy (S&P)*. IEEE Computer Society, May 2021, pp. 1057–1073.

[80] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. “On Ideal Lattices and Learning with Errors over Rings”. In: *Advances in Cryptology – EUROCRYPT 2010*. Ed. by Henri Gilbert. Vol. 6110. Springer, 2010, pp. 1–23.

[81] Mitsuru Matsui. “Linear Cryptanalysis Method for DES Cipher”. In: *Advances in Cryptology — EUROCRYPT ’93*. Ed. by Tor Helleseth. Vol. 765. Springer, 1994, pp. 386–397.

[82] Tsutomu Matsumoto and Hideki Imai. “Public Quadratic Polynomial-Tuples for Efficient Signature-Verification and Message-Encryption”. In: *Advances in Cryptology – EUROCRYPT ’88*. Ed. by D. Barstow et al. Vol. 330. Springer, 1988, pp. 419–453.

[83] Alexander Maximov and Alex Biryukov. “Two Trivial Attacks on Trivium”. In: *Selected Areas in Cryptography – SAC 2007*. Ed. by Carlisle Adams, Ali Miri, and Michael Wiener. Springer, 2007, pp. 36–55.

[84] Pierrick Méaux et al. “Improved Filter Permutators for Efficient FHE: Better Instances and Implementations”. In: *Progress in Cryptology – INDOCRYPT 2019*. Ed. by Feng Hao, Sushmita Ruj, and Sourav Sen Gupta. Vol. 11898. Springer, 2019, pp. 68–91.

[85] Pierrick Méaux et al. “Towards Stream Ciphers for Efficient FHE with Low-Noise Ciphertexts”. In: *Advances in Cryptology – EUROCRYPT 2016*. Ed. by Marc Fischlin and Jean-Sébastien Coron. Vol. 9665. Springer, 2016, pp. 311–343.

[86] Christian Mouchet et al. “Multiparty Homomorphic Encryption from Ring-Learning-with-Errors”. In: *Proc. Priv. Enhancing Technol.* 2021.4 (2021), pp. 291–311.

KAIST

[87] Michael Naehrig, Kristin Lauter, and Vinod Vaikuntanathan. "Can Homomorphic Encryption be Practical?" In: *Proceedings of the 3rd ACM Workshop on Cloud Computing Security Workshop*. Chicago, Illinois, USA: ACM, 2011, pp. 113–124. ISBN: 9781450310048.

[88] Jack O'Connor et al. *BLAKE3 (version 1.0.0)*. https://github.com/BLAKE3-team/BLAKE3. July 2021.

[89] S. Park et al. "HE-Friendly Algorithm for Privacy-Preserving SVM Training". In: *IEEE Access* 8 (2020), pp. 57414–57425.

[90] Rachel Player. "Parameter Selection in Lattice-based Cryptography". English. PhD thesis. Royal Holloway, University of London, 2018.

[91] "Poseidon: A New Hash Function for Zero-Knowledge Proof Systems". In: *30th USENIX Security Symposium (USENIX Security 21)*. Vancouver, B.C.: USENIX Association, Aug. 2021.

[92] Christian Rechberger, Hadi Soleimany, and Tyge Tiessen. "Cryptanalysis of Low-Data Instances of Full LowMCv2". In: *IACR Transactions on Symmetric Cryptology* 2018.3 (Sept. 2018), pp. 163–181.

[93] Oded Regev. "On Lattices, Learning with Errors, Random Linear Codes, and Cryptography". In: *J. ACM* 56.6 (Sept. 2009). ISSN: 0004-5411.

[94] The Sage Developers. *SageMath, the Sage Mathematics Software System (Version 9.4)*. https://www.sagemath.org. 2021.

[95] C. P. Schnorr and M. Euchner. "Lattice Basis Reduction: Improved Practical Algorithms and Solving Subset Sum Problems". In: *Mathematical Programming* 66.1 (Aug. 1994), pp. 181–199.

[96] Claus-Peter Schnorr. "Lattice Reduction by Random Sampling and Birthday Methods". In: *Proceedings of the 20th Annual Symposium on Theoretical Aspects of Computer Science*. STACS '03. Springer-Verlag, 2003, pp. 145–156. ISBN: 3540006230.

[97] Benjamin Hong Meng Tan et al. "Efficient Private Comparison Queries over Encrypted Databases using Fully Homomorphic Encryption with Finite Fields". In: *IEEE Transactions on Dependable and Secure Computing* (2020), pp. 1–1.

[98] Yosuke Todo et al. "Cube Attacks on Non-Blackbox Polynomials Based on Division Property". In: *IEEE Transactions on Computers* 67.12 (2018), pp. 1720–1736.

[99] Qingju Wang et al. "Improved Division Property Based Cube Attacks Exploiting Algebraic Properties of Superpoly". In: *Advances in Cryptology – CRYPTO 2018*. Ed. by Hovav Shacham and Alexandra Boldyreva. Vol. 10991. Springer, 2018, pp. 275–305.

[100] Yuhei Watanabe, Takanori Isobe, and Masakatu Morii. "Cryptanalysis of reduced Kreyvium". In: *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences* 101.9 (2018), pp. 1548–1556.

KAIST

Chapter A. Additional Information on Security Analysis

A.1 Branch Number of the Linear Layer in Rubato

In this section, we show that the branch number of linear layer in Rubato is $2v$, which is a generalization of linear layers in HERA and eHERA. The result of Rubato directly implies the result of HERA and eHERA. Linear branch number B_ℓ and differential branch number B_d for a given a $v \times v$-matrix $\mathbf{M}$ are defined by

$$B_\ell(\mathbf{M}) = \min_{\mathbf{x}\neq 0}\{\mathsf{hw}(\mathbf{x}) + \mathsf{hw}(\mathbf{M}^\intercal\mathbf{x})\},$$
$$B_d(\mathbf{M}) = \min_{\mathbf{x}\neq 0}\{\mathsf{hw}(\mathbf{x}) + \mathsf{hw}(\mathbf{M}\mathbf{x})\}$$

respectively, where hw denotes the word-wise hamming weight function. It is easily seen that $2 \leq B_\ell(\mathbf{M}), B_d(\mathbf{M}) \leq v+1$ for an invertible matrix $\mathbf{M}$. A $v \times v$-matrix $\mathbf{M}$ is a *maximum distance separable* (MDS) matrix if $B_\ell(\mathbf{M}) = B_d(\mathbf{M}) = v+1$. It is also known that $B_\ell(\mathbf{M}) = v+1$ if and only if $B_d(\mathbf{M}) = v+1$ [67]. In this section, we give the branch number of the linear layer of Rubato for general v.

Theorem A.1.1. If $\mathbf{M}_v$ is $v \times v$-MDS matrix, then the linear and the differential branch numbers of

$$\mathsf{MixRows} \circ \mathsf{MixColumns}$$

are all $2v$ where MixColumns (resp. MixRows) multiplies the matrix $\mathbf{M}_v$ to each column (resp. row) of the state.

Proof. We will prove that the differential branch number of MixRows∘MixColumns is $2v$. The linear branch number is computed similarly. We use the notations in Figure A.1. We define $r_i^{(x)}, c_i^{(x)}, r_i^{(y)}, c_i^{(y)}, r_i^{(z)}, c_i^{(z)}$ as follows:

$$\begin{aligned}
r_i^{(x)} &= \mathsf{hw}((x_{i,1}, x_{i,2}, \ldots, x_{i,v}));\\
c_i^{(x)} &= \mathsf{hw}((x_{1,i}, x_{2,i}, \ldots, x_{v,i}));\\
r_i^{(y)} &= \mathsf{hw}((y_{i,1}, y_{i,2}, \ldots, y_{i,v}));\\
c_i^{(y)} &= \mathsf{hw}((y_{1,i}, y_{2,i}, \ldots, y_{v,i}));\\
r_i^{(z)} &= \mathsf{hw}((z_{i,1}, z_{i,2}, \ldots, z_{i,v}));\\
c_i^{(z)} &= \mathsf{hw}((z_{1,i}, z_{2,i}, \ldots, z_{v,i})).
\end{aligned}$$

Let a be the number of non-zero columns in state X. By reordering the indices, one can get

$$\begin{aligned}
c_1^{(x)} \geq c_2^{(x)} \geq \ldots c_a^{(x)} > 0,\\
c_{a+1}^{(x)} = c_{a+2}^{(x)} = \cdots = c_v^{(x)} = 0,
\end{aligned}$$

which implies that $\mathsf{hw}(X) = \sum_{k=1}^{v} c_k^{(x)} \geq a \cdot c_a^{(x)}$.

Let b be the number of non-zero row in Y. Since the branch number of MixColumns is $v+1$, $c_a^{(y)} \geq v+1-c_a^{(x)}$ which implies $b \geq v+1-c_a^{(x)}$. Since the number of non-zero columns are a, $r_i^{(y)} \leq a$. By reordering the indices, one can get

$$a \geq r_1^{(y)} \geq r_2^{(y)} \cdots \geq r_b^{(y)} > 0,$$
$$r_{b+1}^{(y)} = r_{b+2}^{(y)} = \cdots = r_v^{(y)} = 0.$$

Then, we have

$$\begin{aligned}
\mathsf{hw}(Z) &= \sum_{k=1}^{v} r_k^{(z)} = \sum_{k=1}^{b} r_k^{(z)} + \sum_{k=b+1}^{v} r_k^{(z)} \\
&\geq \sum_{k=1}^{b} (v+1-r_k^{(y)}) \geq b \cdot (v+1-a) \\
&\geq \left(v+1-c_a^{(x)}\right) \cdot (v+1-a).
\end{aligned}$$

Combining the above inequalities, we have

$$\mathsf{hw}(X) + \mathsf{hw}(Z) \geq a \cdot c_a^{(x)} + (v+1-c_a^{(x)}) \cdot (v+1-a).$$

The right hand side is a linear expression with respect to each variable when the other one is fixed, so that it is enough to check

$$\left(a, c_a^{(x)}\right) \in \{(1,1), (1,v), (v,1), (v,v)\}$$

in order to find the minimum value. By substituting those values, we have

$$\mathsf{hw}(X) + \mathsf{hw}(Z) \geq \min(v^2+1, 2v) = 2v.$$

Finally, the following example in Figure A.2 shows that the branch number of MixColumns $\circ$ MixRows is $2v$. □

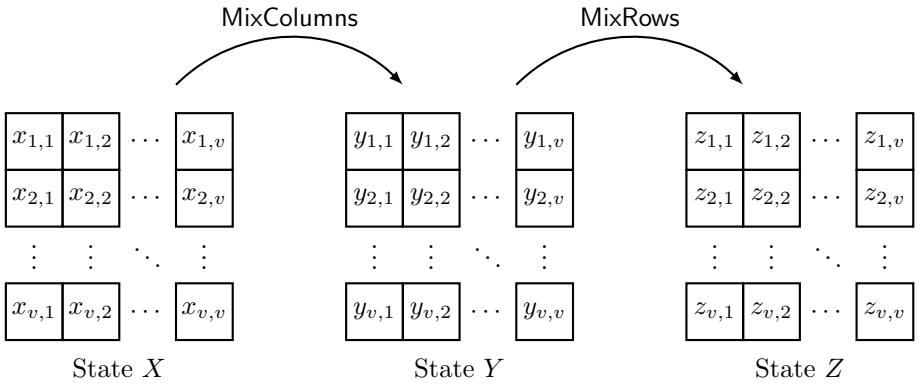


Figure A.1: Diagram of state change in Rubato.

□

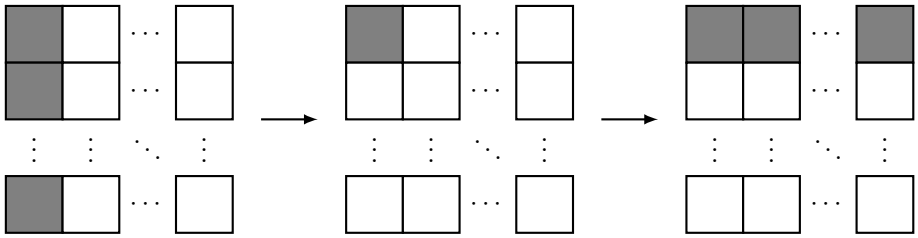


Figure A.2: Pictorial representation of an example satisfying $B_d = 2v$.

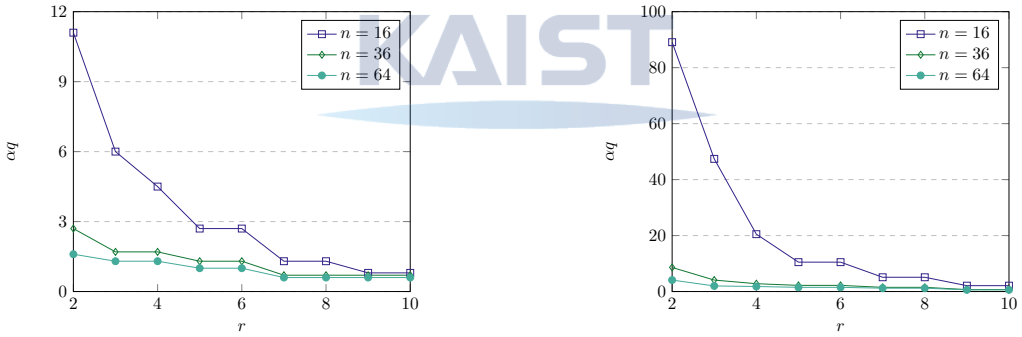
A.2 Additional Plots on Security Analysis

In this section, we give some additional plots on the security analysis of Rubato. In Figure A.3, we plot a trade-off relation between the number of rounds r and the width of the discrete Gaussian distribution αq. One can use a customized set of parameters for various applications.

In the Arora-Ge attack, the probability such that a noise sampled from a discrete Gaussian is denoted by

$$\varepsilon_t = \Pr\left[e \leftarrow D_{\alpha q} : |e| \leq t\alpha q\right].$$

The value $t\alpha q$ determines the success probability of the attacks and necessarily affects the total complexity of the attacks. We give a plot of the complexity of Arora-Ge attack, which is dominating attacks, according to $t\alpha q$ in Figure A.4.

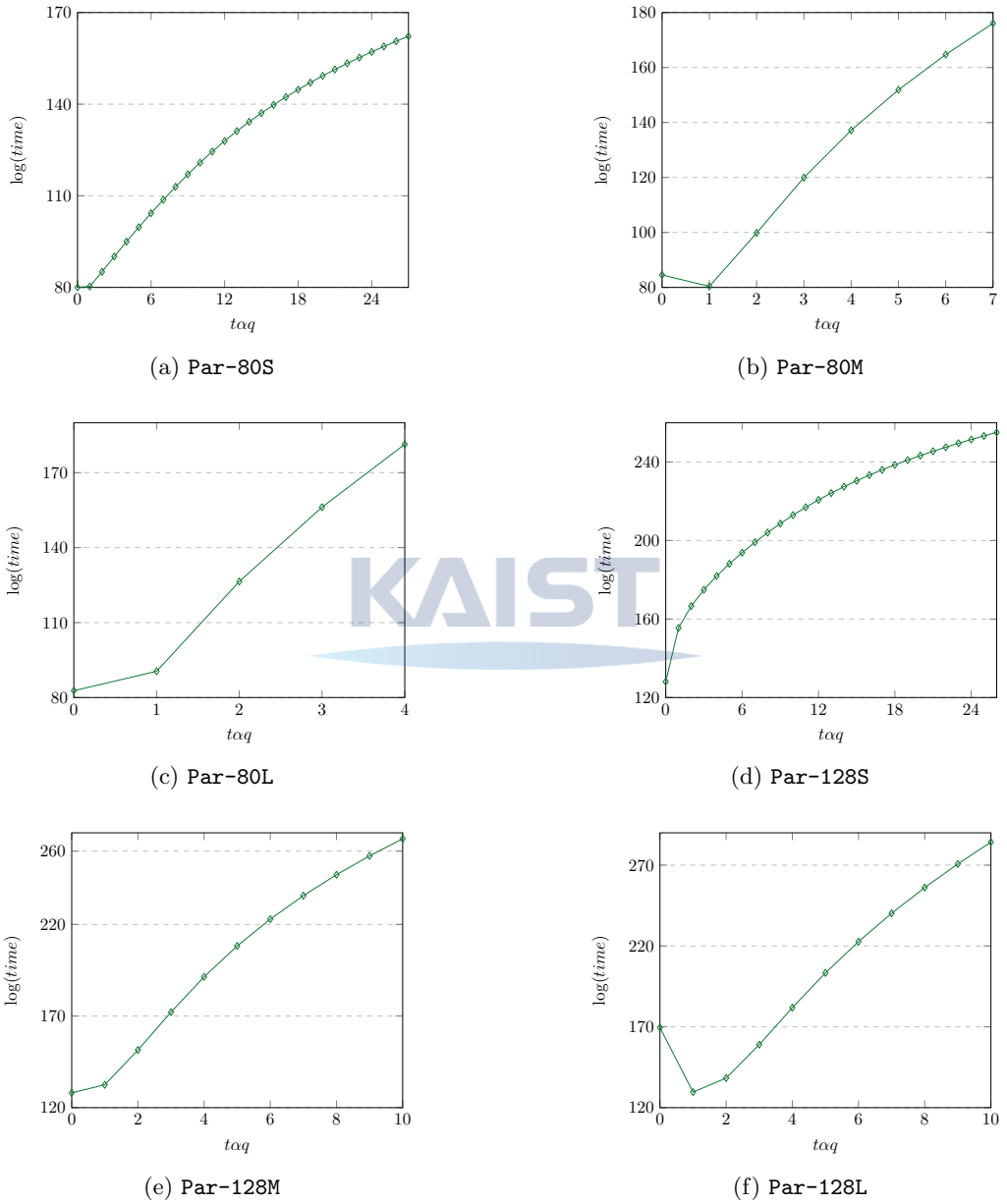


(a) 80-bit security

(b) 128-bit security

Figure A.3: The appropriate choice of the number of rounds r and the width of discrete Gaussian distribution αq for a given security level.

KAIST



(a) `Par-80S` (b) `Par-80M`

(c) `Par-80L` (d) `Par-128S`

(e) `Par-128M` (f) `Par-128L`

Figure A.4: The log of time complexity of the Arora-Ge attack as a function in $t\alpha q$.

KAIST